ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών



Δημιουργία Προηγμένου NFT Marketplace στο SUI Blockchain: Bids με NFTs από διαφορετικές συλλογές

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

του

Κωνσταντίνος Τριπαλιτάκης

Επιβλέπων: Μάριος Κόνιαρης

Αθήνα, Οκτώβριος 2026

. . . . . .

Κωνσταντίνος Τριπαλιτάκης

Copyright © Κωνσταντίνος Τριπαλιτάκης.
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.
Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Περίληψη

Τα NFTs (Non-Fungible Tokens) καθιέρωσαν την έννοια του μοναδικού ψηφιακού περιουσιακού στοιχείου σε δημόσια blockchains, όμως η αγορά τους παραμένει οργανωμένη σχεδόν αποκλειστικά γύρω από πωλήσεις έναντι κρυπτονομίσματος: σταθερές τιμές, δημοπρασίες και προσφορές. Η απευθείας ανταλλαγή NFT με NFT, ιδίως όταν τα δύο αντικείμενα ανήκουν σε διαφορετικές συλλογές, δεν εξυπηρετείται από τις υπάρχουσες πλατφόρμες και, όπου επιχειρείται, προϋποθέτει εμπιστοσύνη προς τον αντισυμβαλλόμενο ή προς έναν μεσάζοντα.

Η παρούσα διπλωματική εργασία σχεδιάζει και υλοποιεί ένα marketplace ανταλλαγών (barter) στο Sui blockchain, στο οποίο ένας χρήστης υποβάλλει προσφορά (bid) που αποτελείται από ένα ή περισσότερα NFTs οποιασδήποτε συλλογής και, προαιρετικά, ποσό SUI, με αντάλλαγμα ένα συγκεκριμένο NFT άλλου χρήστη. Τα προσφερόμενα στοιχεία δεσμεύονται (escrow) μέσα σε ένα κοινόχρηστο αντικείμενο Bid από τη στιγμή της υποβολής, ώστε η αποδοχή από τον κάτοχο του ζητούμενου NFT να ολοκληρώνει την ανταλλαγή ατομικά και χωρίς θεματοφύλακα. Το έξυπνο συμβόλαιο είναι γραμμένο στη γλώσσα Move και αξιοποιεί το αντικειμενοκεντρικό μοντέλο του Sui.

Το ανταλλακτικό εμπόριο υπήρξε ιστορικά η πρώτη μορφή οικονομικής συναλλαγής, πολύ πριν το χρήμα καθιερωθεί ως κοινό μέτρο αξίας, και δεν έπαψε ποτέ να έχει θέση εκεί όπου τα αγαθά είναι μοναδικά και η αξία τους υποκειμενική. Η ψηφιοποίηση πραγματικών αγαθών ως NFTs έχει ήδη περάσει από το πειραματικό στο θεσμικό στάδιο, με χαρακτηριστικό παράδειγμα την ψηφιοποίηση (tokenization) τίτλων ιδιοκτησίας ακινήτων σε blockchain από το Κτηματολόγιο του Ντουμπάι. Σε αυτό το πλαίσιο, η ύπαρξη ενός αξιόπιστου μηχανισμού απευθείας ανταλλαγής, που δεν επιβάλλει τη ρευστοποίηση κάθε αγαθού σε κρυπτονόμισμα ως ενδιάμεσο βήμα, παύει να αποτελεί συμπλήρωμα των υπαρχουσών αγορών και γίνεται αναγκαία υποδομή. Η παρούσα εργασία προτείνει έναν τέτοιο μηχανισμό, στον οποίο η ανταλλαγή εκτελείται ατομικά από το ίδιο το συμβόλαιο, ώστε κανένα από τα δύο μέρη να μη χρειάζεται να εμπιστευθεί το άλλο.

Λέξεις κλειδιά: blockchain, Sui, Move, NFT, marketplace, barter, bid, escrow, atomic swap, smart contract, dApp, cross-collection

Abstract

Non-fungible tokens (NFTs) established the notion of a unique digital asset on public blockchains, yet their markets remain organised almost exclusively around sales for cryptocurrency: fixed prices, auctions and offers. Direct NFT-for-NFT exchange, especially between items of different collections, is not served by existing platforms and, where it is attempted at all, requires trusting the counterparty or an intermediary.

This diploma thesis designs and implements a barter marketplace on the Sui blockchain, in which a user submits a bid consisting of one or more NFTs from any collection and, optionally, an amount of SUI, in exchange for a specific NFT held by another user. The offered assets are escrowed inside a shared `Bid` object at submission time, so that acceptance by the owner of the requested NFT completes the swap atomically and without a custodian. The smart contract is written in Move and exploits Sui's object-centric model.

Barter was historically the first form of economic exchange, long before money was established as a common measure of value, and it never ceased to have a place where goods are unique and their value subjective. The digitisation of real-world goods as NFTs has already moved from the experimental to the institutional stage, a notable example being the tokenisation of real-estate title deeds on a blockchain by the Dubai Land Department. In this context, a trustworthy mechanism for direct exchange, one that does not impose the liquidation of every asset into cryptocurrency as an intermediate step, ceases to be a complement to existing markets and becomes necessary infrastructure. This thesis proposes such a mechanism, in which the swap is executed atomically by the contract itself, so that neither party needs to trust the other.

Keywords: blockchain, Sui, Move, NFT, marketplace, barter, bid, escrow, atomic swap, smart contract, dApp, cross-collection

Ευχαριστίες

Ευχαριστώ θερμά την οικογένειά μου για τη στήριξη όλα αυτά τα χρόνια και τον κ. Κόνιαρη για την υπομονή και την καθοδήγησή του.

Περιεχόμενα

Περίληψη **ii**

Abstract **iii**

Ευχαριστίες **iv**

Κατάλογος σχημάτων **x**

Κατάλογος πινάκων **xii**

1 Εισαγωγή **1**

1.1 Κίνητρο 1

1.2 Αντικείμενο και στόχος 2

1.3 Μεθοδολογία 3

1.4 Δομή της εργασίας 3

2 Θεωρητικό υπόβαθρο **4**

2.1 Blockchain: βασικές έννοιες 4

2.1.1 Κατανεμημένο βιβλίο συναλλαγών 4

2.1.2 Συναίνεση 4

2.1.3 Έξυπνα συμβόλαια 5

2.2 To Sui blockchain 5

2.2.1 Αντικειμενοκεντρικό μοντέλο 5

2.2.2 Εκτέλεση συναλλαγών και συναίνεση 7

2.2.3 Κόμβοι του δικτύου 8

2.2.4 Programmable Transaction Blocks 9

2.2.5 Gas και αποθήκευση 10

2.2.6 Συμβάντα 10

2.2.7 Πρότυπα του οικοσυστήματος 10

2.2.8 Τα δίκτυα του Sui 11

2.3 Η γλώσσα Move . . . 12
2.3.1 Πόροι και ικανότητες . . . 12
2.3.2 Γενικοί τύποι . . . 13
2.3.3 Δυναμικά πεδία . . . 13
2.3.4 Αρχικοποίηση, one-time witness και έλεγχοι . . . 14
2.4 NFTs και αγορές . . . 14
2.4.1 Non-Fungible Tokens . . . 14
2.4.2 Μηχανισμοί διαπραγμάτευσης . . . 15
2.4.3 Το πρόβλημα της δίκαιης ανταλλαγής . . . 15

3 Απαιτήσεις και σχεδίαση του συστήματος 17
3.1 Η γενική ιδέα . . . 17
3.2 Ρόλοι χρηστών . . . 17
3.3 Λειτουργικές απαιτήσεις . . . 18
3.4 Μη λειτουργικές απαιτήσεις . . . 19
3.5 Κύκλος ζωής της προσφοράς . . . 20
3.6 Σχεδιαστικές αποφάσεις . . . 21
3.6.1 Η προσφορά ως κοινόχρηστο αντικείμενο . . . 21
3.6.2 Δέσμευση με δυναμικά πεδία αντικειμένων . . . 21
3.6.3 Γενικοί τύποι για τις συλλογές . . . 22
3.6.4 Στοχευμένες προσφορές . . . 22
3.6.5 Το νόμισμα ως μέρος της προσφοράς . . . 23
3.6.6 Ανακάλυψη μέσω συμβάντων . . . 23
3.7 Περιπτώσεις χρήσης . . . 23
3.7.1 ΠΧ1: Σύνδεση πορτοφολιού . . . 23
3.7.2 ΠΧ2: Δημιουργία NFT . . . 24
3.7.3 ΠΧ3: Υποβολή προσφοράς . . . 24
3.7.4 ΠΧ4: Αποδοχή προσφοράς . . . 25
3.7.5 ΠΧ5: Απόρριψη προσφοράς . . . 26
3.7.6 ΠΧ6: Ακύρωση προσφοράς . . . 27
3.8 Αρχιτεκτονική . . . 27
3.9 Σχεδίαση διεπαφής . . . 28

4 Υλοποίηση του έξυπνου συμβολαίου 30
4.1 Δομή του πακέτου . . . 30
4.1.1 Συμβάσεις κώδικα . . . 31
4.2 Μοντέλο δεδομένων . . . 31
4.2.1 Η δομή Bid . . . 31

4.2.2 Ο τύπος NFT της εφαρμογής . . . 33
4.3 Αρχικοποίηση και Display . . . 34
4.4 Συναρτήσεις . . . 35
4.4.1 Δημιουργία NFT . . . 35
4.4.2 Υποβολή προσφοράς . . . 36
4.4.3 Αποδοχή . . . 38
4.4.4 Ακύρωση και απόρριψη . . . 41
4.5 Συμβάντα και κωδικοί σφαλμάτων . . . 42
4.6 Ανάλυση ασφάλειας . . . 43
4.6.1 Εξουσιοδότηση . . . 44
4.6.2 Διατήρηση των δεσμευμένων στοιχείων . . . 44
4.6.3 Ατομικότητα και ταυτόχρονη πρόσβαση . . . 45
4.6.4 Απουσία επανεισόδου . . . 45
4.6.5 Παρατηρήσεις και γνωστά όρια . . . 45
4.7 Έλεγχοι . . . 46
4.7.1 Το σκηνικό των ελέγχων . . . 46
4.7.2 Τι καλύπτεται . . . 47
4.8 Ανάπτυξη στο δίκτυο . . . 49
4.8.1 Ροή ανάπτυξης ανά δίκτυο . . . 49
4.8.2 Το τοπικό δίκτυο στην πράξη . . . 50

5 Υλοποίηση της αποκεντρωμένης εφαρμογής 52
5.1 Επισκόπηση και τεχνολογίες . . . 52
5.2 Σύνδεση με το δίκτυο και το πορτοφόλι . . . 54
5.2.1 Το στιγμιότυπο dApp Kit . . . 54
5.2.2 Πελάτης ανά δίκτυο . . . 54
5.3 Ανάγνωση της κατάστασης χωρίς indexer . . . 55
5.3.1 NFTs του χρήστη . . . 55
5.3.2 Ανακάλυψη μέσω συμβάντων . . . 56
5.3.3 Μεταδεδομένα και προσωρινή αποθήκευση . . . 60
5.4 Σύνθεση και εκτέλεση συναλλαγών . . . 60
5.4.1 Κατασκευή των PTBs . . . 60
5.4.2 Εκτέλεση, αναμονή και ανανέωση . . . 62
5.5 Υλοποίηση της διεπαφής . . . 63
5.6 Δυσκολίες και εναλλακτικές προσεγγίσεις . . . 65
5.6.1 Το ListEvents στο τοπικό δίκτυο . . . 65
5.6.2 Λοιπές δυσκολίες . . . 65
5.7 Επαλήθευση . . . 66

6 Επίδειξη λειτουργίας **68**
6.1 Αρχική σελίδα και σύνδεση πορτοφολιού 68
6.2 Δημιουργία NFT 70
6.3 Περιήγηση σε συλλογή 71
6.4 Υποβολή προσφοράς 71
6.5 Εισερχόμενες προσφορές 73
6.6 Αποδοχή, απόρριψη και ακύρωση 75
6.7 Η εφαρμογή σε κινητή οθόνη 77

7 Αξιολόγηση **80**
7.1 Κόστος συναλλαγών 80
7.1.1 Μεθοδολογία 80
7.1.2 Αποτελέσματα 81
7.1.3 Ανάλυση 82
7.1.4 Χρόνος απόκρισης 83
7.2 Δοκιμές αντοχής και κλιμάκωσης 84
7.2.1 Μεθοδολογία και επιλογή δικτύου 84
7.2.2 Μέγεθος προσφοράς 84
7.2.3 Παρτίδες δημιουργίας 86
7.2.4 Ταυτόχρονοι χρήστες 86
7.2.5 Ανταγωνισμός για την ίδια προσφορά 87
7.2.6 Κόστος ανακάλυψης με το ιστορικό 87
7.3 Προβολή στο mainnet 88
7.4 Αποτελέσματα των σεναρίων 90
7.5 Κάλυψη απαιτήσεων 92
7.6 Περιορισμοί 92

8 Συμπεράσματα και μελλοντικές επεκτάσεις **94**
8.1 Σύνοψη 94
8.2 Συμπεράσματα 95
8.3 Μελλοντικές επεκτάσεις 96

Βιβλιογραφία **98**

Συντομογραφίες **102**

Α′ Πηγαίος κώδικας του έξυπνου συμβολαίου **103**
Α′.1 marketplace_nft.move 103
Α′.2 barter.move 105

Α′.3 barter_fixture.move 112

Κατάλογος σχημάτων

2.1 Μορφές ιδιοκτησίας αντικειμένων στο Sui. Η δεξιά διάταξη είναι ακριβώς εκείνη που χρησιμοποιεί η εργασία: ένα κοινόχρηστο Bid που κρατά τα προσφερόμενα NFTs ως δυναμικά πεδία αντικειμένων (DOF). 7
2.2 Το PTB της υποβολής προσφοράς: τρεις εντολές, μία υπογραφή, ατομική εκτέλεση. 9

3.1 Διάγραμμα περιπτώσεων χρήσης. Οι δύο ρόλοι ορίζονται ανά προσφορά· ο ίδιος χρήστης αναλαμβάνει και τους δύο. 18
3.2 Μηχανή καταστάσεων της προσφοράς. Σε κάθε τερματική κατάσταση το αντικείμενο Bid έχει διαγραφεί από την αλυσίδα. 21
3.3 Διάγραμμα ακολουθίας της υποβολής προσφοράς. 25
3.4 Διάγραμμα ακολουθίας της αποδοχής. Τα τρία πρώτα μηνύματα δείχνουν πώς η εφαρμογή ανακαλύπτει τις εισερχόμενες προσφορές χωρίς indexer. . 26
3.5 Αρχιτεκτονική του συστήματος. 27
3.6 Οθόνες της εφαρμογής και μεταβάσεις. Με συνεχές περίγραμμα οι σελίδες, με διακεκομμένο τα παράθυρα διαλόγου. 29

4.1 Οι δομές κατάστασης των δύο modules. Το `Bid` δεν περιέχει τα προσφερόμενα NFTs ως πεδίο· αυτά προσαρτώνται ως δυναμικά πεδία αντικειμένων με κλειδιά `EscrowKey(0)`, `EscrowKey(1)`, κ.ο.κ. 32

5.1 Ανακάλυψη των εισερχόμενων προσφορών ενός χρήστη από δεδομένα της αλυσίδας. 57

6.1 Η αρχική σελίδα πριν από τη σύνδεση πορτοφολιού. 69
6.2 Αριστερά: επιλογή πορτοφολιού. Δεξιά: η αρχική σελίδα με συνδεδεμένο πορτοφόλι και NFTs ήδη στην αγορά. 69
6.3 Η σελίδα δημιουργίας NFT. 70
6.4 Αριστερά: εκκρεμής δημιουργία. Δεξιά: ειδοποίηση επιτυχίας. 70

6.5 Η σελίδα της συλλογής Cars. Πρώτη αριστερά η κάρτα NFT του ίδιου του χρήστη. 71
6.6 Σύνθεση προσφοράς, βήμα 2: επιλογή NFTs. Κάτω αριστερά η σύνοψη «δίνεις ⇄ παίρνεις». 72
6.7 Αριστερά: βήμα 1, ποσό SUI. Δεξιά: βήμα 3, επισκόπηση. 72
6.8 Αριστερά: ειδοποίηση επιτυχούς υποβολής, πάνω από τη συλλογή. Δεξιά: η προσφορά στην καρτέλα «Placed». 73
6.9 Τα NFTs του χρήστη: τα δύο πρώτα με ένδειξη εισερχόμενων προσφορών, τα δύο επόμενα δεσμευμένα στη δική του προσφορά («In a bid»). 74
6.10 Οι εισερχόμενες προσφορές για το Rare Mansion. 75
6.11 Η καρτέλα «Received» με τρεις εισερχόμενες προσφορές, η τρίτη ανεπτυγμένη. 76
6.12 Αριστερά: αποδοχή σε εξέλιξη. Δεξιά: η ανταλλαγή ολοκληρώθηκε. 76
6.13 Αριστερά: απόρριψη εισερχόμενης προσφοράς. Δεξιά: ακύρωση υποβληθείσας προσφοράς. 77
6.14 Κινητή οθόνη: αρχική σελίδα, συλλογή, επισκόπηση προσφοράς. 78
6.15 Κινητή οθόνη: δημιουργία, εισερχόμενες προσφορές, σελίδα προσφορών. . 79

7.1 Κόστος υποβολής προσφοράς ως συνάρτηση του πλήθους δεσμευμένων NFTs (λογαριθμικοί άξονες). Το καθαρό κόστος είναι γραμμικό· ο υπολογισμός μένει στο ελάχιστο κλιμάκιο έως τα 50 και εκτοξεύεται πάνω από τα 100. 85
7.2 Ο χρόνος του ερωτήματος εισερχόμενων προσφορών αυξάνει γραμμικά με το πλήθος των προσφορών στην αλυσίδα, περίπου 58 ms ανά χίλιες προσφορές σε τοπικό κόμβο. 88

Κατάλογος πινάκων

2.1 NFT στο μοντέλο λογαριασμών και στο αντικειμενοκεντρικό μοντέλο . . . 7
2.2 Τα δίκτυα του Sui . . . 11
2.3 Οι ικανότητες της Move . . . 12

3.1 Λειτουργικές απαιτήσεις . . . 19
3.2 Μη λειτουργικές απαιτήσεις . . . 20

4.1 Πεδία της δομής `Bid` . . . 33
4.2 Δημόσιες συναρτήσεις του πακέτου . . . 35
4.3 Συμβάντα του πακέτου . . . 43
4.4 Κωδικοί σφαλμάτων του συμβολαίου . . . 43
4.5 Εξουσιοδότηση ανά συνάρτηση . . . 44
4.6 Οι έλεγχοι του module `barter` . . . 48
4.7 Οι έλεγχοι του module `marketplace_nft` . . . 49

5.1 Τεχνολογίες της εφαρμογής . . . 53
5.2 Ερωτήματα της εφαρμογής και ακύρωση της προσωρινής μνήμης . . . 60
5.3 Τα PTBs της εφαρμογής . . . 61

7.1 Κόστος gas ανά λειτουργία, σε SUI . . . 81
7.2 Κόστος αποθήκευσης ανά αντικείμενο, όπως προκύπτει από τις μετρήσεις . 82
7.3 Καθαρό κόστος ανά κύκλο ζωής προσφοράς με ένα NFT, σε SUI . . . 83
7.4 Κόστος και χρόνος προσφοράς με n δεσμευμένα NFTs, σε SUI (τοπικό δίκτυο) 85
7.5 Δημιουργία n NFTs σε μία συναλλαγή, σε SUI (τοπικό δίκτυο) . . . 86
7.6 Εκατό ταυτόχρονες συναλλαγές ανά ροή (τοπικό δίκτυο) . . . 87
7.7 k ταυτόχρονες αποδοχές της ίδιας προσφοράς, τρεις γύροι ανά k . . . 87
7.8 Χρόνος του ερωτήματος εισερχόμενων προσφορών ως συνάρτηση του ιστορικού . . . 88
7.9 Παράμετροι gas ανά δίκτυο (16 Οκτωβρίου 2026) . . . 89
7.10 Προβολή του καθαρού κόστους ανά λειτουργία στο mainnet . . . 89

7.11 Σενάρια συμβολαίου στο τοπικό δίκτυο που υπερβαίνουν τους ελέγχους μονάδας 91
7.12 Αντιστοίχιση απαιτήσεων σε ελέγχους 92

Κατάλογος αποσπασμάτων κώδικα

2.1 Ένα αντικείμενο με ικανότητες `key` και `store` 13
4.1 Δομή του αποθετηρίου 30
4.2 Η δομή της προσφοράς και το κλειδί δέσμευσης 31
4.3 Το one-time witness, ο τύπος `MarketplaceNFT` και το συμβάν δημιουργίας 33
4.4 Η συνάρτηση `init` του `marketplace_nft` 34
4.5 Η συνάρτηση `mint` 35
4.6 Η συνάρτηση `place_bid` 36
4.7 Η συνάρτηση `accept_bid` 38
4.8 Οι ιδιωτικές φάσεις του κύκλου ζωής 39
4.9 Οι συναρτήσεις `cancel_bid` και `reject_bid` 41
4.10 Ένας έλεγχος αποδοχής με το σκηνικό 46
5.1 Δομή του καταλόγου `app/src` 53
5.2 Το στιγμιότυπο dApp Kit 54
5.3 Επιλογή πελάτη ανά δίκτυο 54
5.4 Ανάγνωση όλων των συμβάντων ενός τύπου 56
5.5 Το ερώτημα εισερχόμενων προσφορών 56
5.6 Ανάγνωση των δεσμευμένων NFTs μιας προσφοράς 59
5.7 Το PTB της υποβολής προσφοράς 60
5.8 Δημιουργία παρτίδας NFTs 61
5.9 Κοινός μηχανισμός εκτέλεσης συναλλαγών 62
A′.1 Το module `marketplace_nft` 103
A′.2 Το module `barter` 105
A′.3 Το σκηνικό των ελέγχων 112

Κεφάλαιο 1

Εισαγωγή

1.1 Κίνητρο

Τα Non-Fungible Tokens (NFTs) είναι ψηφιακά περιουσιακά στοιχεία των οποίων η μοναδικότητα και η κυριότητα πιστοποιούνται από ένα δημόσιο blockchain. Από την τυποποίηση του ERC-721 στο Ethereum [1] έως τα σύγχρονα αντικειμενοκεντρικά blockchains όπως το Sui [2], τα NFTs χρησιμοποιούνται για ψηφιακή τέχνη, συλλεκτικά αντικείμενα, εισιτήρια, αντικείμενα παιχνιδιών και τίτλους ιδιοκτησίας. Η δευτερογενής αγορά τους, ωστόσο, οργανώθηκε εξαρχής γύρω από ένα και μόνο πρότυπο συναλλαγής: την πώληση έναντι κρυπτονομίσματος. Πλατφόρμες όπως το OpenSea [3] και το Blur [4] στο Ethereum, ή το Tradeport [5] και το BlueMove [6] στο Sui, προσφέρουν καταχωρήσεις σταθερής τιμής, δημοπρασίες και προσφορές σε νόμισμα. Η μονάδα μέτρησης της αξίας είναι πάντοτε το ETH ή το SUI.

Υπάρχει όμως μια ολόκληρη κατηγορία συναλλαγών που το πρότυπο αυτό δεν εξυπηρετεί: η *ανταλλαγή* (barter). Το πρόβλημα γίνεται φανερό με ένα απλό παράδειγμα. Έστω ότι κάποιος έχει ένα μήλο και ένας φίλος του ένα πορτοκάλι, και ο καθένας θέλει το φρούτο του άλλου. Η φυσική λύση είναι ο πρώτος να δώσει το μήλο και ο δεύτερος το πορτοκάλι. Σε μια αγορά που γνωρίζει μόνο πωλήσεις, όμως, η ανταλλαγή αυτή δεν μπορεί καν να εκφραστεί: πρέπει πρώτα ο καθένας να πουλήσει το φρούτο του έναντι του νομίσματος της χώρας και έπειτα να αγοράσει από τον άλλον το φρούτο που τον ενδιαφέρει, με δύο ανεξάρτητες συναλλαγές, διπλά τέλη και τον κίνδυνο να μεταβληθεί η τιμή στο ενδιάμεσο. Ακριβώς σε αυτή τη θέση βρίσκεται σήμερα ένας συλλέκτης που κατέχει ένα αντικείμενο μιας συλλογής και επιθυμεί ένα αντικείμενο διαφορετικής συλλογής: είτε πουλά και αγοράζει σε δύο βήματα, είτε διαπραγματεύεται εκτός αλυσίδας μια απευθείας ανταλλαγή. Η δεύτερη επιλογή αναδεικνύει το κλασικό πρόβλημα της δίκαιης ανταλλαγής: κάποιος από τους δύο πρέπει να στείλει πρώτος, εμπιστευόμενος ότι ο άλλος θα ανταποκριθεί, ή πρέπει και οι δύο να εμπιστευθούν έναν μεσάζοντα-θεματοφύλακα.

Το Sui προσφέρει τεχνικές προϋποθέσεις που καθιστούν το πρόβλημα αυτό ασυνήθιστα

ευχερές να λυθεί σωστά. Κάθε NFT είναι ένα αντικείμενο πρώτης τάξης με δικό του αναγνωριστικό και ιδιοκτήτη, όχι μια εγγραφή σε έναν χάρτη μέσα σε ένα συμβόλαιο. Η γλώσσα Move [7] διασφαλίζει στατικά ότι ένα τέτοιο αντικείμενο δεν μπορεί να αντιγραφεί, να χαθεί ή να δημιουργηθεί από το πουθενά. Τα Programmable Transaction Blocks επιτρέπουν σε μία συναλλαγή να συνθέσει πολλαπλές κλήσεις, και τα *κοινόχρηστα αντικείμενα* (shared objects) επιτρέπουν σε δύο άγνωστους μεταξύ τους χρήστες να αλληλεπιδράσουν με την ίδια δομή δεδομένων χωρίς κεντρικό διαχειριστή. Με άλλα λόγια, μια προσφορά ανταλλαγής μπορεί να γίνει η ίδια ένα αντικείμενο που κρατά δεσμευμένα τα προσφερόμενα στοιχεία, και η αποδοχή της μια ατομική συναλλαγή που τα μεταβιβάζει.

1.2 Αντικείμενο και στόχος

Αντικείμενο της εργασίας είναι η σχεδίαση και η υλοποίηση ενός NFT marketplace ανταλλαγών στο Sui, με κεντρικό χαρακτηριστικό τις *προσφορές με NFTs από διαφορετικές συλλογές*. Συγκεκριμένα, το σύστημα πρέπει να επιτρέπει σε έναν χρήστη:

- να υποβάλει προσφορά (bid) που αποτελείται από ένα ή περισσότερα NFTs οποιασδήποτε συλλογής και, προαιρετικά, από ποσό SUI, με αντάλλαγμα ένα συγκεκριμένο NFT που ανήκει σε άλλον χρήστη·
- να δεσμεύει τα προσφερόμενα στοιχεία κατά την υποβολή, ώστε ο αποδέκτης να γνωρίζει ότι η προσφορά είναι έγκυρη και εκτελέσιμη·
- ως κάτοχος του ζητούμενου NFT, να αποδεχθεί την προσφορά, οπότε η ανταλλαγή ολοκληρώνεται σε μία ατομική συναλλαγή, ή να την απορρίψει, οπότε τα δεσμευμένα στοιχεία επιστρέφουν στον προσφέροντα·
- ως προσφέρων, να ακυρώσει την προσφορά του οποιαδήποτε στιγμή πριν από την αποδοχή της.

Το σύστημα δεν διαθέτει διαχειριστή, θεματοφύλακα ή προνομιακά κλειδιά: η ορθότητα της ανταλλαγής διασφαλίζεται αποκλειστικά από το έξυπνο συμβόλαιο και το μοντέλο εκτέλεσης του Sui. Η υποστήριξη διαφορετικών συλλογών δεν επιτυγχάνεται με λίστες επιτρεπόμενων τύπων ή με μετατροπή σε κάποιο κοινό ενδιάμεσο token, αλλά με γενικούς τύπους (generics) στο επίπεδο του συμβολαίου: μια προσφορά έχει τύπο `Bid<Offered, Requested>`, όπου οι δύο παράμετροι μπορούν να είναι οποιοιδήποτε τύποι αντικειμένων με τις κατάλληλες ικανότητες. Έτσι ο ίδιος κώδικας εξυπηρετεί ανταλλαγές εντός της ίδιας συλλογής, μεταξύ συλλογών, ή ακόμη και μεταξύ εντελώς διαφορετικών εφαρμογών, με τον έλεγχο τύπων να γίνεται από την ίδια την εικονική μηχανή.

1.3 Μεθοδολογία

Η εργασία ακολούθησε τέσσερις φάσεις, αντίστοιχες με εκείνες που περιγράφουν οι [8] και [9]. Στην πρώτη φάση μελετήθηκαν το μοντέλο αντικειμένων του Sui και η γλώσσα Move, ώστε να εντοπισθούν οι δυνατότητες που απαντούν στο πρόβλημα που περιγράφηκε παραπάνω. Στη δεύτερη φάση προσδιορίσθηκαν οι απαιτήσεις και σχεδιάσθηκε το πρωτόκολλο: οι ρόλοι, ο κύκλος ζωής της προσφοράς, οι αναλλοίωτες συνθήκες ασφάλειας και η δομή των δεδομένων στην αλυσίδα. Στην τρίτη φάση υλοποιήθηκε το έξυπνο συμβόλαιο με ανάπτυξη καθοδηγούμενη από ελέγχους, και στη συνέχεια η dApp· και τα δύο αναθεωρήθηκαν αργότερα με βάση δημοσιευμένα πρότυπα κώδικα και σχεδίασης διεπαφών. Στην τέταρτη φάση το σύστημα αναπτύχθηκε σε τοπικό δίκτυο (και, σε πρώιμη έκδοση, σε devnet), επαληθεύθηκε από άκρη σε άκρη και αξιολογήθηκε ως προς το κόστος των συναλλαγών και την αντοχή του.

1.4 Δομή της εργασίας

Η εργασία οργανώνεται ως εξής.

- Το Κεφάλαιο 2 παρουσιάζει το θεωρητικό υπόβαθρο: τις βασικές έννοιες των blockchains, το αντικειμενοκεντρικό μοντέλο του Sui, τη γλώσσα Move και τα NFTs.
- Το Κεφάλαιο 3 καταγράφει τις απαιτήσεις και τη σχεδίαση του συστήματος, με περιπτώσεις χρήσης, διαγράμματα UML και τη μηχανή καταστάσεων της προσφοράς.
- Το Κεφάλαιο 4 περιγράφει την υλοποίηση του έξυπνου συμβολαίου σε Move, την ανάλυση ασφάλειας και τους ελέγχους.
- Το Κεφάλαιο 5 περιγράφει την αρχιτεκτονική και την υλοποίηση της dApp.
- Το Κεφάλαιο 6 επιδεικνύει τη λειτουργία του συστήματος μέσα από τις οθόνες της εφαρμογής.
- Το Κεφάλαιο 7 αξιολογεί το σύστημα ως προς το κόστος, την αντοχή και την κάλυψη των απαιτήσεων.
- Το Κεφάλαιο 8 συνοψίζει τα συμπεράσματα και προτείνει μελλοντικές επεκτάσεις.
- Στο παράρτημα παρατίθεται ο πλήρης πηγαίος κώδικας των δύο modules του έξυπνου συμβολαίου και του σκηνικού των ελέγχων του.

Κεφάλαιο 2

Θεωρητικό υπόβαθρο

Το κεφάλαιο αυτό συγκεντρώνει τις έννοιες που απαιτούνται για την κατανόηση των επόμενων. Ξεκινά από τις γενικές αρχές των blockchains και των έξυπνων συμβολαίων, συνεχίζει με το Sui και τη γλώσσα Move, στα οποία στηρίζεται η υλοποίηση, και κλείνει με τα NFTs και το πρόβλημα της δίκαιης ανταλλαγής, το οποίο η εργασία καλείται να λύσει.

2.1 Blockchain: βασικές έννοιες

2.1.1 Κατανεμημένο βιβλίο συναλλαγών

Ένα blockchain είναι ένα κατανεμημένο βιβλίο συναλλαγών (distributed ledger): μια δομή δεδομένων στην οποία καταγράφονται συναλλαγές και η οποία τηρείται ταυτόχρονα από πολλούς κόμβους ενός δικτύου, χωρίς κάποιος από αυτούς να έχει προνομιακό ρόλο [10]. Οι συναλλαγές ομαδοποιούνται σε blocks, και κάθε block περιέχει τη σύνοψη (hash) του προηγούμενου. Η αλυσίδα που σχηματίζεται έχει την ιδιότητα ότι η τροποποίηση ενός παλαιότερου block αλλάζει τη σύνοψή του και, κατ' επέκταση, ακυρώνει όλα τα επόμενα. Σε συνδυασμό με το γεγονός ότι κάθε κόμβος διατηρεί δικό του αντίγραφο, η ιδιότητα αυτή καθιστά το ιστορικό πρακτικά αμετάβλητο.

Κάθε συναλλαγή υπογράφεται ψηφιακά από τον αποστολέα της. Η ταυτότητα ενός χρήστη στο δίκτυο είναι ένα ζεύγος κλειδιών: το δημόσιο κλειδί, από το οποίο παράγεται η διεύθυνσή του, και το ιδιωτικό, με το οποίο υπογράφει. Δεν υπάρχει μητρώο χρηστών ούτε κωδικοί πρόσβασης. Όποιος κατέχει το ιδιωτικό κλειδί μιας διεύθυνσης μπορεί να διαθέσει ό,τι ανήκει σε αυτήν, και κανείς άλλος.

2.1.2 Συναίνεση

Επειδή δεν υπάρχει κεντρική αρχή, οι κόμβοι πρέπει να συμφωνήσουν μεταξύ τους για το ποιες συναλλαγές είναι έγκυρες και με ποια σειρά εκτελούνται. Ο μηχανισμός με τον

οποίο επιτυγχάνεται αυτή η συμφωνία ονομάζεται πρωτόκολλο συναίνεσης (consensus). Το Bitcoin εισήγαγε το Proof of Work, στο οποίο το δικαίωμα πρότασης ενός block κερδίζεται με υπολογιστική εργασία [10]. Τα νεότερα δίκτυα χρησιμοποιούν κατά κανόνα Proof of Stake: οι επικυρωτές (validators) δεσμεύουν κεφάλαιο ως εγγύηση καλής συμπεριφοράς, και η συμφωνία επιτυγχάνεται με πρωτόκολλα ανεκτικά σε βυζαντινά σφάλματα (Byzantine Fault Tolerant), τα οποία εγγυώνται ορθότητα εφόσον λιγότερο από το ένα τρίτο της συνολικής δέσμευσης ελέγχεται από κακόβουλους συμμετέχοντες. Το Sui ανήκει στη δεύτερη κατηγορία.

2.1.3 Έξυπνα συμβόλαια

Ο όρος έξυπνο συμβόλαιο (smart contract) προτάθηκε από τον Szabo για να περιγράψει συμφωνίες των οποίων οι όροι επιβάλλονται από λογισμικό αντί από τρίτο μέρος [11]. Στο πλαίσιο των blockchains, καθιερώθηκε με το Ethereum [12]: ένα έξυπνο συμβόλαιο είναι πρόγραμμα αποθηκευμένο στην αλυσίδα, το οποίο εκτελείται από όλους τους κόμβους με ντετερμινιστικό τρόπο όταν το καλεί μια συναλλαγή. Η εκτέλεση είναι *ατομική*: είτε ολοκληρώνεται και όλες οι αλλαγές που επέφερε καταγράφονται, είτε αποτυγχάνει και δεν καταγράφεται καμία. Η ατομικότητα αυτή είναι το θεμέλιο πάνω στο οποίο στηρίζεται κάθε πρωτόκολλο ανταλλαγής χωρίς μεσάζοντα, όπως θα φανεί στην Ενότητα 2.4.3.

Η εκτέλεση έχει κόστος, το οποίο καταβάλλει ο αποστολέας της συναλλαγής στο εγγενές νόμισμα του δικτύου και ονομάζεται gas. Το gas αποζημιώνει τους επικυρωτές και ταυτόχρονα αποτρέπει την κατάχρηση των κοινόχρηστων υπολογιστικών πόρων.

2.2 Το Sui blockchain

Το Sui είναι ένα δημόσιο blockchain Proof of Stake που αναπτύχθηκε από τη Mysten Labs και τέθηκε σε λειτουργία το 2023 [2, 13]. Διαφέρει από τα περισσότερα δίκτυα της γενιάς του Ethereum σε δύο θεμελιώδεις επιλογές: η κατάσταση της αλυσίδας οργανώνεται γύρω από *αντικείμενα* αντί για λογαριασμούς, και τα έξυπνα συμβόλαια γράφονται στη γλώσσα Move. Και οι δύο επιλογές επηρεάζουν άμεσα τον σχεδιασμό της παρούσας εργασίας.

2.2.1 Αντικειμενοκεντρικό μοντέλο

Στο Ethereum, η κατάσταση είναι ένας χάρτης από διευθύνσεις σε λογαριασμούς, και κάθε συμβόλαιο διαθέτει δικό του ιδιωτικό αποθηκευτικό χώρο. Ένα NFT του προτύπου ERC-721 δεν υπάρχει ως αυτοτελής οντότητα· είναι μια εγγραφή της μορφής `tokenId` → `owner` μέσα στον χώρο του συμβολαίου της συλλογής [1]. Η «μεταβίβαση» είναι μια κλήση που αλλάζει την εγγραφή αυτή.

Στο Sui, η βασική μονάδα αποθήκευσης είναι το *αντικείμενο* (object) [14]. Κάθε αντικείμενο έχει:

- ένα μοναδικό αναγνωριστικό (object ID) 32 bytes,
- έναν τύπο, που ορίζεται από κάποιο module της Move,
- έναν αριθμό έκδοσης, που αυξάνεται σε κάθε τροποποίηση,
- έναν ιδιοκτήτη, και
- τα πεδία δεδομένων που προβλέπει ο τύπος του.

Ένα NFT είναι λοιπόν ένα αντικείμενο πρώτης τάξης. Η διεύθυνση του κατόχου του δεν βρίσκεται μέσα σε κάποιο συμβόλαιο αλλά στα μεταδεδομένα του ίδιου του αντικειμένου, και η μεταβίβασή του είναι εγγενής λειτουργία του συστήματος.

Ο ιδιοκτήτης ενός αντικειμένου καθορίζει ποιος μπορεί να το χρησιμοποιήσει ως είσοδο σε συναλλαγή. Διακρίνονται τέσσερις περιπτώσεις, που απεικονίζονται στο Σχήμα 2.1:

Αντικείμενο διεύθυνσης (address-owned). Ανήκει σε μία διεύθυνση και μόνον αυτή μπορεί να το χρησιμοποιήσει. Έτσι κρατούνται τα NFTs και τα νομίσματα στα πορτοφόλια των χρηστών.

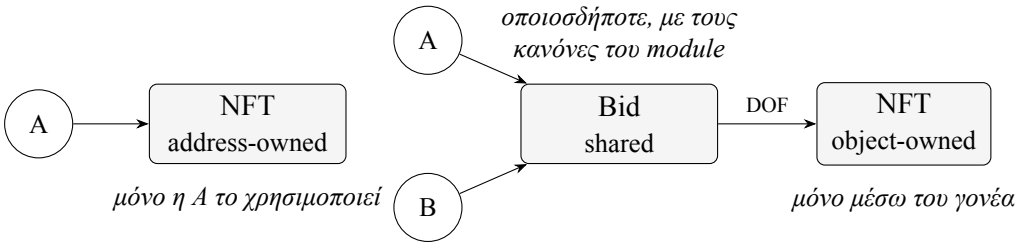
Κοινόχρηστο αντικείμενο (shared). Δεν ανήκει σε κανέναν και μπορεί να το χρησιμοποιήσει οποιοσδήποτε, υπό τους κανόνες που επιβάλλει το module που το όρισε. Χρησιμοποιείται όταν πολλοί ανεξάρτητοι χρήστες πρέπει να αλληλεπιδράσουν με την ίδια κατάσταση.

Αμετάβλητο αντικείμενο (immutable). Δεν μεταβάλλεται ποτέ ξανά και είναι αναγνώσιμο από όλους. Τα δημοσιευμένα πακέτα κώδικα ανήκουν σε αυτή την κατηγορία.

Αντικείμενο αντικειμένου (object-owned). Ανήκει σε άλλο αντικείμενο, για παράδειγμα ως δυναμικό πεδίο του. Είναι προσβάσιμο μόνο μέσω του γονέα του.

Στο σύστημα της εργασίας εμφανίζονται και οι τέσσερις κατηγορίες. Ένα NFT στο πορτοφόλι ενός χρήστη είναι αντικείμενο με ιδιοκτήτη διεύθυνση. Η προσφορά Bid είναι κοινόχρηστο αντικείμενο, ώστε να μπορεί ο προσφέρων να την ακυρώσει και ο κάτοχος του ζητούμενου NFT να την αποδεχθεί ή να την απορρίψει. Το δημοσιευμένο πακέτο του συμβολαίου είναι αμετάβλητο αντικείμενο. Και τα NFTs που δεσμεύονται μέσα σε μια προσφορά γίνονται, για όσο διαρκεί η δέσμευση, αντικείμενα που ανήκουν στο Bid: δεν εμφανίζονται στο πορτοφόλι κανενός και απελευθερώνονται μόνο όταν η προσφορά κλείσει.

Ο Πίνακας 2.1 συνοψίζει τις διαφορές των δύο μοντέλων ως προς τα NFTs.



Σχήμα 2.1: Μορφές ιδιοκτησίας αντικειμένων στο Sui. Η δεξιά διάταξη είναι ακριβώς εκείνη που χρησιμοποιεί η εργασία: ένα κοινόχρηστο Bid που κρατά τα προσφερόμενα NFTs ως δυναμικά πεδία αντικειμένων (DOF).

Πίνακας 2.1: NFT στο μοντέλο λογαριασμών και στο αντικειμενοκεντρικό μοντέλο

	Ethereum (ERC-721)	Sui
Τι είναι το NFT	Εγγραφή στον χώρο αποθήκευσης του συμβολαίου της συλλογής	Αυτοτελές αντικείμενο με δικό του ID
Πού καταγράφεται ο κάτοχος	Σε mapping μέσα στο συμβόλαιο	Στο πεδίο owner του αντικειμένου
Μεταβίβαση	Κλήση συνάρτησης του συμβολαίου	Εγγενής λειτουργία (`transfer`)
Χρήση από τρίτο συμβόλαιο	Με προηγούμενη έγκριση (approve)	Το αντικείμενο περνά ως όρισμα, κατά τιμή ή αναφορά
«Συλλογή»	Ένα συμβόλαιο	Ένας τύπος Move

Η τέταρτη γραμμή του πίνακα έχει ιδιαίτερη σημασία για τα marketplaces. Στο Ethereum, για να διαπραγματευθεί ένα marketplace ένα NFT εκ μέρους του χρήστη, ο χρήστης πρέπει να του έχει παραχωρήσει διαρκή έγκριση διαχείρισης. Η έγκριση αυτή παραμένει ενεργή και μετά τη συναλλαγή και αποτελεί γνωστή επιφάνεια επίθεσης. Στο Sui δεν υπάρχει αντίστοιχη έννοια: ένα συμβόλαιο αποκτά πρόσβαση σε ένα αντικείμενο μόνο όταν ο κάτοχός του το δώσει ρητά ως όρισμα στη συγκεκριμένη συναλλαγή.

2.2.2 Εκτέλεση συναλλαγών και συναίνεση

Κάθε συναλλαγή στο Sui δηλώνει ρητά τα αντικείμενα που χρησιμοποιεί ως εισόδους. Αυτό επιτρέπει στους επικυρωτές να γνωρίζουν εκ των προτέρων ποιες συναλλαγές είναι ανεξάρτητες μεταξύ τους και να τις εκτελούν παράλληλα. Επιτρέπει επίσης μια διάκριση που δεν υπάρχει στα περισσότερα blockchains [15]:

- Οι συναλλαγές που αγγίζουν μόνο αντικείμενα διεύθυνσης δεν ανταγωνίζονται καμία άλλη, αφού μόνο ο κάτοχος μπορεί να τις υποβάλει. Γι' αυτές αρκεί, κατά κανόνα, η επικύρωση από την απαιτούμενη πλειοψηφία των επικυρωτών, χωρίς να χρειάζεται ολική διάταξη ως προς τις υπόλοιπες συναλλαγές του δικτύου.

- Οι συναλλαγές που αγγίζουν έστω και ένα κοινόχρηστο αντικείμενο πρέπει να διαταχθούν, διότι δύο χρήστες μπορεί να επιχειρήσουν ταυτόχρονα να το τροποποιήσουν. Η διάταξη γίνεται από το πρωτόκολλο συναίνεσης, που στην τρέχουσα μορφή του δικτύου είναι το Mysticeti [16], εξέλιξη των Narwhal και Bullshark [17].

Η συνέπεια για τον σχεδιαστή είναι ότι τα κοινόχρηστα αντικείμενα έχουν κόστος και πρέπει να χρησιμοποιούνται μόνο όπου είναι αναγκαία. Στην περίπτωσή μας είναι: μια προσφορά πρέπει να μπορεί να την αποδεχθεί ένας χρήστης και να την ακυρώσει ένας άλλος, άρα δεν μπορεί να ανήκει αποκλειστικά σε κανέναν από τους δύο. Η συναίνεση προσφέρει επιπλέον ακριβώς την εγγύηση που χρειάζεται ένα πρωτόκολλο ανταλλαγής: αν η αποδοχή και η ακύρωση υποβληθούν ταυτόχρονα, μία από τις δύο θα διαταχθεί πρώτη και θα εκτελεστεί, ενώ η άλλη θα αποτύχει επειδή το αντικείμενο της προσφοράς δεν θα υπάρχει πλέον.

2.2.3 Κόμβοι του δικτύου

Οι «κόμβοι» που αναφέρθηκαν μέχρι εδώ δεν είναι όλοι ίδιοι. Στο Sui διακρίνονται τρεις ρόλοι με διαφορετικές αρμοδιότητες [18].

Επικυρωτές (validators). Είναι οι κόμβοι που παράγουν την αλυσίδα. Σε κάθε εποχή (epoch), διάρκειας περίπου 24 ωρών, μια επιτροπή επικυρωτών, καθορισμένη από το δεσμευμένο κεφάλαιο, παραλαμβάνει τις συναλλαγές, ελέγχει την εγκυρότητά τους, τις εκτελεί, τις διατάσσει μέσω του πρωτοκόλλου συναίνεσης όπου χρειάζεται, και υπογράφει τα αποτελέσματα. Οι επικυρωτές ομαδοποιούν επίσης τις εκτελεσμένες συναλλαγές σε σημεία ελέγχου (checkpoints), τα οποία αποτελούν την επίσημη, πλήρως διατεταγμένη ιστορία του δικτύου. Δεν εξυπηρετούν ερωτήματα εφαρμογών· ο ρόλος τους είναι η επικύρωση, όχι η διάθεση δεδομένων.

Πλήρεις κόμβοι (full nodes). Παρακολουθούν τους επικυρωτές, παραλαμβάνουν τα σημεία ελέγχου, επαληθεύουν τις υπογραφές τους και αναπαράγουν τοπικά κάθε συναλλαγή, ώστε να διατηρούν δικό τους πλήρες αντίγραφο της κατάστασης και του ιστορικού. Δεν συμμετέχουν στη συναίνεση και δεν χρειάζονται άδεια ή δεσμευμένο κεφάλαιο: οποιοσδήποτε μπορεί να λειτουργήσει έναν. Είναι οι κόμβοι με τους οποίους επικοινωνούν τα πορτοφόλια, οι εξερευνητές και οι αποκεντρωμένες εφαρμογές, μέσω των διεπαφών JSON-RPC και gRPC: δέχονται νέες συναλλαγές και τις προωθούν στους επικυρωτές, και απαντούν σε ερωτήματα όπως «ποια αντικείμενα ανήκουν σε αυτή τη διεύθυνση» ή «ποια συμβάντα αυτού του τύπου εκπέμφθηκαν», για τα οποία διατηρούν τα αντίστοιχα ευρετήρια. Η Mysten Labs προσφέρει δημόσιους πλήρεις κόμβους για κάθε δίκτυο, και αυτούς χρησιμοποιεί η εφαρμογή της εργασίας.

Indexers. Προαιρετικές υπηρεσίες που διαβάζουν τα σημεία ελέγχου από έναν πλήρη κόμβο και τα αποθηκεύουν σε σχεσιακή βάση δεδομένων, ώστε να απαντούν σε ερωτήματα που

ένας πλήρης κόμβος δεν υποστηρίζει, όπως «όλα τα αντικείμενα ενός τύπου» ή συγκεντρωτικά στατιστικά, συνήθως μέσω GraphQL. Απαιτούν υποδομή και εισάγουν μια πρόσθετη πηγή αλήθειας που πρέπει να παραμένει συγχρονισμένη με την αλυσίδα. Η εφαρμογή της εργασίας σχεδιάσθηκε ώστε να μην τους χρειάζεται.

Η διάκριση έχει πρακτική σημασία για ό,τι ακολουθεί. Όταν στη συνέχεια λέμε ότι «ο κόμβος ευρετηριάζει τα συμβάντα» ή ότι «η εφαρμογή ρωτά τον κόμβο», εννοούμε πάντοτε έναν πλήρη κόμβο. Το τοπικό δίκτυο ανάπτυξης της Ενότητας 2.2.8 εκκινεί έναν επικυρωτή και έναν πλήρη κόμβο μέσα στην ίδια διεργασία, οπότε η ίδια διάκριση ισχύει και εκεί.

2.2.4 Programmable Transaction Blocks

Μια συναλλαγή στο Sui δεν περιορίζεται σε μία κλήση συνάρτησης. Έχει τη μορφή Programmable Transaction Block (PTB): μιας ακολουθίας εντολών που εκτελούνται με τη σειρά, μοιράζονται εισόδους και μπορούν να χρησιμοποιούν η μία τα αποτελέσματα της άλλης [19]. Το PTB εκτελείται ατομικά ως σύνολο. Οι εντολές που χρησιμοποιεί η εργασία είναι:

`MoveCall` Κλήση δημόσιας συνάρτησης ενός module.

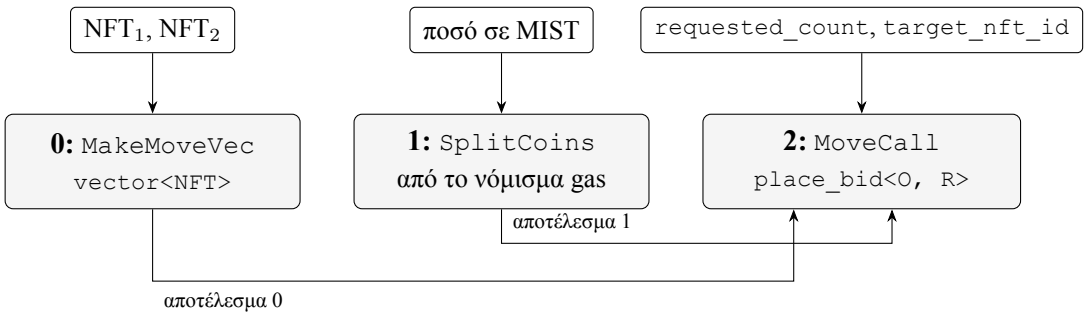
`SplitCoins` Αποκοπή ποσού από ένα νόμισμα σε νέο νόμισμα.

`TransferObjects` Μεταβίβαση αντικειμένων σε διεύθυνση.

`MakeMoveVec` Κατασκευή διανύσματος (`vector<T>`) από επιμέρους αντικείμενα, ώστε να δοθεί ως όρισμα σε συνάρτηση.

`Publish` Δημοσίευση πακέτου.

Το Σχήμα 2.2 δείχνει το PTB με το οποίο η εφαρμογή υποβάλλει μια προσφορά: κατασκευάζεται ένα διάνυσμα από τα προσφερόμενα NFTs, αποκόπτεται από το νόμισμα του gas το προσφερόμενο ποσό SUI, και τα δύο αποτελέσματα δίνονται ως ορίσματα στη συνάρτηση `place_bid`. Ο χρήστης υπογράφει μία φορά.



Σχήμα 2.2: Το PTB της υποβολής προσφοράς: τρεις εντολές, μία υπογραφή, ατομική εκτέλεση.

2.2.5 Gas και αποθήκευση

Το κόστος μιας συναλλαγής στο Sui έχει δύο σκέλη [20]. Το *κόστος υπολογισμού* εξαρτάται από τις πράξεις που εκτελούνται και από την τιμή αναφοράς του gas της τρέχουσας εποχής. Το *κόστος αποθήκευσης* είναι ανάλογο του μεγέθους των αντικειμένων που δημιουργούνται ή μεγαλώνουν και λειτουργεί ως εγγύηση: όταν ένα αντικείμενο διαγραφεί, το μεγαλύτερο μέρος του ποσού που είχε καταβληθεί για την αποθήκευσή του (99% με τις τρέχουσες παραμέτρους του δικτύου) επιστρέφεται (storage rebate) σε όποιον υποβάλλει τη συναλλαγή της διαγραφής, όχι κατ' ανάγκην σε όποιον το είχε δημιουργήσει. Όλα τα ποσά εκφράζονται σε MIST, με 1 SUI $= 10^9$ MIST. Ο μηχανισμός της επιστροφής θα μας απασχολήσει στην αξιολόγηση (Κεφάλαιο 7), διότι μια προσφορά είναι ακριβώς ένα αντικείμενο που δημιουργείται για να διαγραφεί.

2.2.6 Συμβάντα

Ένα module μπορεί να εκπέμπει *συμβάντα* (events) κατά την εκτέλεση. Τα συμβάντα δεν αποθηκεύονται στην κατάσταση της αλυσίδας και δεν είναι αναγνώσιμα από άλλα συμβόλαια· καταγράφονται στα αποτελέσματα της συναλλαγής και ευρετηριάζονται από τους πλήρεις κόμβους (Ενότητα 2.2.3), ώστε οι εφαρμογές να μπορούν να τα αναζητούν ανά τύπο. Μαζί με κάθε συμβάν ο κόμβος καταγράφει τον αποστολέα και το ψηφιακό αποτύπωμα της συναλλαγής που το εξέπεμψε, οπότε ένα ωφέλιμο φορτίο δεν χρειάζεται να επαναλαμβάνει αυτά τα στοιχεία. Η σημασία τους είναι μεγάλη, επειδή ένας πλήρης κόμβος δεν προσφέρει ερώτημα της μορφής «όλα τα αντικείμενα τύπου T». Μπορεί να απαντήσει ποια αντικείμενα ανήκουν σε μια διεύθυνση, όχι όμως ποια κοινόχρηστα αντικείμενα ενός τύπου υπάρχουν. Τα συμβάντα είναι επομένως ο τυπικός τρόπος με τον οποίο μια εφαρμογή ανακαλύπτει τι υπάρχει στην αλυσίδα χωρίς δικό της indexer.

2.2.7 Πρότυπα του οικοσυστήματος

Δύο πρότυπα του Sui αφορούν άμεσα την εργασία.

Display. Τα πεδία ενός αντικειμένου είναι αυθαίρετα· ένα πορτοφόλι δεν γνωρίζει ποιο από αυτά είναι το όνομα ή η εικόνα. Το πρότυπο Object Display [21] επιτρέπει στον εκδότη ενός τύπου να ορίσει ένα σύνολο προτύπων (templates), π.χ. `name` → `"{name}"`, τα οποία οι πλήρεις κόμβοι εφαρμόζουν στα πεδία του αντικειμένου για να παραγάγουν τα μεταδεδομένα προβολής του. Ο ορισμός απαιτεί το αντικείμενο `Publisher`, που αποδεικνύει ότι ο καλών είναι ο εκδότης του πακέτου.

Wallet Standard. Το Sui Wallet Standard [22] ορίζει τη διεπαφή μέσω της οποίας μια εφαρμογή ιστού ανακαλύπτει τα εγκατεστημένα πορτοφόλια και τους ζητά να υπογράψουν συναλλαγές. Χάρη σε αυτό, η εφαρμογή της εργασίας λειτουργεί με οποιοδήποτε συμβατό πορτοφόλι χωρίς ειδικό κώδικα για το καθένα.

2.2.8 Τα δίκτυα του Sui

Το Sui δεν είναι ένα δίκτυο αλλά τέσσερα, με το ίδιο λογισμικό και διαφορετικό σκοπό [23, 24]. Η διάκριση έχει σημασία για την εργασία, διότι καθορίζει πού αναπτύχθηκε, πού μετρήθηκε και πού δημοσιεύεται τελικά το σύστημα (Ενότητα 4.8). Ο Πίνακας 2.2 τα συνοψίζει.

Πίνακας 2.2: Τα δίκτυα του Sui

Δίκτυο	Σκοπός	Διατήρηση δεδομένων	Απόκτηση SUI	Ρόλος στην εργασία
Mainnet	Παραγωγή· πραγματικές συναλλαγές	Μόνιμη	Αγορά· πραγματική αξία	Καμία (εκτός εμβέλειας)
Testnet	Δοκιμή πριν την παραγωγή	Μακροχρόνια· σπάνιες, προαναγγελλόμενες διαγραφές	Faucet με όριο ρυθμού· χωρίς αξία	Τελική δημοσίευση και χρήση με πραγματικό πορτοφόλι
Devnet	Νέες, ασταθείς δυνατότητες	Διαγράφεται εβδομαδιαίως	Faucet με όριο ρυθμού· χωρίς αξία	Πρώτη δοκιμαστική δημοσίευση
Localnet	Ανάπτυξη στον υπολογιστή του προγραμματιστή	Όσο ζει η διεργασία· νέα γένεση σε κάθε εκκίνηση	Τοπικό faucet χωρίς όριο	Όλη η ανάπτυξη, οι έλεγχοι και οι μετρήσεις

Το mainnet είναι το δίκτυο παραγωγής: λειτουργείται από ανεξάρτητους επικυρωτές, τα δεδομένα του διατηρούνται μόνιμα και το SUI του έχει χρηματική αξία. Το testnet τρέχει το ίδιο λογισμικό ως περιβάλλον δοκιμής πριν την παραγωγή· τα νομίσματά του δεν έχουν αξία και διανέμονται δωρεάν από ένα faucet με όριο ρυθμού, ενώ τα δεδομένα του διατηρούνται μακροχρόνια, με σπάνιες διαγραφές που προαναγγέλλονται. Το devnet φιλοξενεί τις νεότερες, ενδεχομένως ασταθείς δυνατότητες του πρωτοκόλλου και διαγράφεται κάθε εβδομάδα, άρα ό,τι δημοσιεύεται εκεί έχει ημερομηνία λήξης. Το localnet, τέλος, δεν είναι δημόσιο δίκτυο αλλά μια πλήρης αλυσίδα που εκκινεί ο προγραμματιστής στον δικό του υπολογιστή

με την εντολή `sui start`, με έναν επικυρωτή, πλήρη κόμβο και τοπικό faucet χωρίς όριο. Κάθε εκκίνηση με την επιλογή `--force-regenesis` δημιουργεί μια νέα, άδεια αλυσίδα με νέο αναγνωριστικό, οπότε κάθε δημοσιευμένο πακέτο και κάθε αντικείμενο παύουν να υπάρχουν.

Δύο συνέπειες της διάκρισης επανέρχονται στα επόμενα κεφάλαια. Πρώτον, το κόστος σε gas είναι ντετερμινιστικό και κοινό σε όλα τα δίκτυα με τις ίδιες παραμέτρους πρωτοκόλλου, ενώ οι χρόνοι απόκρισης ενός τοπικού δικτύου με έναν επικυρωτή δεν αντιπροσωπεύουν δημόσιο δίκτυο (Ενότητα 7.1). Δεύτερον, τα πορτοφόλια επέκτασης του οικοσυστήματος δηλώνουν ποια δίκτυα υποστηρίζουν, και κατά κανόνα δεν περιλαμβάνουν το τοπικό· γι' αυτό η ανάπτυξη στο localnet χρησιμοποιεί ένα προσωρινό πορτοφόλι μέσα στον φυλλομετρητή, και η δοκιμή με πραγματικό πορτοφόλι γίνεται στο testnet (Κεφάλαιο 5).

2.3 Η γλώσσα Move

Η Move σχεδιάστηκε αρχικά για το blockchain Diem με ρητό στόχο την ασφαλή αναπαράσταση ψηφιακών περιουσιακών στοιχείων [7]. Το Sui χρησιμοποιεί μια παραλλαγή της προσαρμοσμένη στο αντικειμενοκεντρικό μοντέλο [25]. Ο κώδικας οργανώνεται σε *modules*, τα οποία ομαδοποιούνται σε *πακέτα* (packages) και δημοσιεύονται στην αλυσίδα ως αμετάβλητα αντικείμενα.

2.3.1 Πόροι και ικανότητες

Η κεντρική ιδέα της Move είναι ότι οι τιμές ενός τύπου δεν μπορούν εξ ορισμού ούτε να αντιγραφούν ούτε να απορριφθούν. Μια τιμή πρέπει να καταλήξει κάπου: να αποθηκευθεί, να μεταβιβασθεί, ή να αποδομηθεί ρητά από το module που όρισε τον τύπο της. Ο μεταγλωττιστής απορρίπτει κάθε πρόγραμμα που αφήνει μια τέτοια τιμή «στον αέρα». Οι εξαιρέσεις δηλώνονται με *ικανότητες* (abilities) πάνω στον τύπο, όπως δείχνει ο Πίνακας 2.3.

Πίνακας 2.3: Οι ικανότητες της Move

Ικανότητα	Τι επιτρέπει
`copy`	Η τιμή μπορεί να αντιγραφεί.
`drop`	Η τιμή μπορεί να απορριφθεί σιωπηρά στο τέλος της εμβέλειάς της.
`key`	Η τιμή είναι αντικείμενο του Sui: έχει πεδίο `id: UID` και μπορεί να υπάρχει αυτοτελώς στην αλυσίδα.
`store`	Η τιμή μπορεί να αποθηκευθεί μέσα σε άλλο αντικείμενο και να μεταβιβασθεί ελεύθερα εκτός του module που την όρισε.

Ένα NFT δηλώνεται τυπικά ως `has key, store`. Η απουσία του `copy` σημαίνει ότι δεν

μπορεί να διπλασιασθεί, και η απουσία του `drop` ότι δεν μπορεί να χαθεί. Αυτές δεν είναι συμβάσεις που ο προγραμματιστής οφείλει να τηρεί αλλά ιδιότητες που ελέγχονται κατά τη μεταγλώττιση και επαληθεύονται εκ νέου από τον bytecode verifier κατά τη δημοσίευση. Για ένα πρωτόκολλο δέσμευσης, η συνέπεια είναι ισχυρή: μια συνάρτηση που παραλαμβάνει ένα διάνυσμα NFTs δεν μεταγλωττίζεται αν δεν αποφασίσει ρητά πού θα καταλήξει το καθένα.

```
public struct Sword has key, store {
    id: UID,
    strength: u64,
}

public fun forge(strength: u64, ctx: &mut TxContext): Sword {
    Sword { id: object::new(ctx), strength }
}

// Δενμεταγλωττίζεται : το `sword` δενέχει  `drop`
// καιδενμεταβιβάζεταιούτεαποδομείται    .
public fun lose(sword: Sword) { }
```

Απόσπασμα κώδικα 2.1: Ένα αντικείμενο με ικανότητες `key` και `store`

2.3.2 Γενικοί τύποι

Η Move υποστηρίζει γενικούς τύπους (generics) με περιορισμούς ικανοτήτων. Η δήλωση `fun wrap<T: key + store>(item: T)` δέχεται οποιονδήποτε τύπο αντικειμένου που μπορεί να αποθηκευθεί, ανεξάρτητα από το ποιο πακέτο τον όρισε. Οι παράμετροι τύπου επιλύονται κατά την κλήση, και η εικονική μηχανή ελέγχει ότι τα ορίσματα έχουν πράγματι τους δηλωμένους τύπους.

Μια παράμετρος τύπου που δεν εμφανίζεται σε κανένα πεδίο της δομής δηλώνεται ως `phantom`. Χρησιμεύει ως ετικέτα σε επίπεδο τύπων: δύο τιμές `Bid<Sword, Shield>` και `Bid<Shield, Sword>` έχουν ίδια πεδία αλλά είναι διαφορετικοί τύποι, και μια συνάρτηση που αναμένει τον έναν δεν δέχεται τον άλλον. Στην τεχνική αυτή στηρίζεται η υποστήριξη διαφορετικών συλλογών στην εργασία (Ενότητα 4.2).

2.3.3 Δυναμικά πεδία

Τα πεδία μιας δομής ορίζονται στατικά. Όταν ένα αντικείμενο πρέπει να κρατήσει μεταβλητό πλήθος τιμών, ή τιμές ετερογενών τύπων, χρησιμοποιούνται τα *δυναμικά πεδία* (dynamic fields) [26]: ζεύγη ονόματος-τιμής που προσαρτώνται σε ένα αντικείμενο κατά την εκτέλεση. Υπάρχουν δύο παραλλαγές:

- Στο απλό δυναμικό πεδίο (`dynamic_field`) η τιμή *περιτυλίσσεται* (wrapped) στο πεδίο. Αν είναι αντικείμενο, παύει να είναι προσβάσιμο με το ID του από εργαλεία εκτός αλυσίδας.
- Στο δυναμικό πεδίο αντικειμένου (`dynamic_object_field`) η τιμή πρέπει να είναι αντικείμενο και *παραμένει* αυτοτελές αντικείμενο με το δικό του ID, του οποίου ιδιοκτήτης γίνεται το γονικό αντικείμενο. Οι explorers και τα πορτοφόλια συνεχίζουν να το βλέπουν.

Η εργασία χρησιμοποιεί τη δεύτερη παραλλαγή για τα δεσμευμένα NFTs, ώστε ένα NFT που βρίσκεται μέσα σε προσφορά να παραμένει ορατό και επαληθεύσιμο από οποιονδήποτε.

2.3.4 Αρχικοποίηση, one-time witness και έλεγχοι

Ένα module μπορεί να ορίσει συνάρτηση `init`, που εκτελείται μία μόνο φορά, κατά τη δημοσίευση του πακέτου. Αν η πρώτη της παράμετρος είναι μια δομή με το όνομα του module σε κεφαλαία και μόνη ικανότητα το `drop`, το σύστημα κατασκευάζει και της παραδίδει μια μοναδική τιμή αυτού του τύπου, το λεγόμενο one-time witness (OTW). Επειδή κανείς άλλος δεν μπορεί να κατασκευάσει τέτοια τιμή, το OTW αποδεικνύει ότι ο κώδικας εκτελείται κατά τη δημοσίευση, και με αυτό το module διεκδικεί το αντικείμενο `Publisher` που χρειάζεται για το Display.

Οι αποτυχίες σηματοδοτούνται με `assert!` και έναν αριθμητικό κωδικό (abort code). Στην έκδοση 2024 της γλώσσας ο κωδικός δηλώνεται ως σταθερά με την επισημείωση `#[error]`, η οποία επιτρέπει να συνοδεύεται από περιγραφικό μήνυμα κειμένου που ενσωματώνεται στο μεταγλωττισμένο πακέτο. Μια αποτυχία ακυρώνει ολόκληρη τη συναλλαγή. Τέλος, η Move διαθέτει ενσωματωμένο πλαίσιο ελέγχων μονάδας: το module `test_scenario` προσομοιώνει διαδοχικές συναλλαγές από διαφορετικούς αποστολείς, και η επισημείωση `#[expected_failure]` επιτρέπει να ελεγχθεί ότι μια κλήση αποτυγχάνει με τον αναμενόμενο κωδικό και στο αναμενόμενο module.

2.4 NFTs και αγορές

2.4.1 Non-Fungible Tokens

Ένα token είναι *ανταλλάξιμο* (fungible) όταν κάθε μονάδα του είναι ισοδύναμη με κάθε άλλη, όπως συμβαίνει με τα νομίσματα. Ένα Non-Fungible Token είναι αντίθετα μοναδικό: αναπαριστά ένα συγκεκριμένο στοιχείο με δική του ταυτότητα [27]. Στο Ethereum η έννοια τυποποιήθηκε με το ERC-721 [1] και αργότερα με το ERC-1155 [28], και γνώρισε ευρεία διάδοση από το 2021 [29]. Οι χρήσεις περιλαμβάνουν ψηφιακή τέχνη, συλλεκτικά αντικείμενα, αντικείμενα παιχνιδιών, εισιτήρια και πιστοποιητικά.

Τα NFTs οργανώνονται σε *συλλογές*: σύνολα αντικειμένων κοινής προέλευσης και κοινών χαρακτηριστικών. Εντός μιας συλλογής, τα αντικείμενα διαφέρουν συχνά ως προς τη *σπανιότητα* (rarity), που επηρεάζει την αξία που τους αποδίδει η αγορά. Στο Sui, όπως φάνηκε στον Πίνακα 2.1, μια συλλογή αντιστοιχεί φυσιολογικά σε έναν τύπο Move. Δύο NFTs διαφορετικών συλλογών είναι τιμές διαφορετικών τύπων, και ένα γενικό πρωτόκολλο ανταλλαγής οφείλει να μπορεί να χειρισθεί τύπους που δεν γνωρίζει εκ των προτέρων.

2.4.2 Μηχανισμοί διαπραγμάτευσης

Οι αγορές NFT έχουν αναπτύξει μια μικρή οικογένεια μηχανισμών, που συναντώνται με μικρές παραλλαγές σε όλες τις μεγάλες πλατφόρμες:

- *καταχώρηση σταθερής τιμής* (listing): ο κάτοχος ορίζει τιμή και ο πρώτος που την καταβάλλει αποκτά το αντικείμενο·
- *δημοπρασία* (auction): οι ενδιαφερόμενοι πλειοδοτούν εντός προθεσμίας·
- *προσφορά* (offer ή bid): ο ενδιαφερόμενος προτείνει ένα τίμημα για ένα συγκεκριμένο αντικείμενο ή για οποιοδήποτε αντικείμενο μιας συλλογής, και ο κάτοχος αποφασίζει αν θα το δεχθεί·
- *ανταλλαγή* (swap ή barter): τα δύο μέρη ανταλλάσσουν αντικείμενα μεταξύ τους, ενδεχομένως με συμπληρωματικό ποσό σε νόμισμα.

Στους τρεις πρώτους το τίμημα είναι νόμισμα. Η παρούσα εργασία συνδυάζει τον τρίτο με τον τέταρτο: μια προσφορά της οποίας το τίμημα είναι NFTs, νόμισμα, ή και τα δύο.

2.4.3 Το πρόβλημα της δίκαιης ανταλλαγής

Όταν δύο μέρη που δεν εμπιστεύονται το ένα το άλλο θέλουν να ανταλλάξουν ψηφιακά αγαθά, ανακύπτει το πρόβλημα της *δίκαιης ανταλλαγής* (fair exchange): πρέπει είτε να λάβουν και οι δύο αυτό που αναμένουν, είτε κανείς. Αν ένας στείλει πρώτος, ο άλλος μπορεί να μην ανταποδώσει. Είναι γνωστό ότι το πρόβλημα δεν λύνεται ντετερμινιστικά χωρίς έμπιστο τρίτο μέρος [30], και τα κλασικά πρωτόκολλα στηρίζονται σε έναν τέτοιο τρίτο, που παρεμβαίνει τουλάχιστον σε περίπτωση διαφωνίας [31].

Το blockchain μπορεί να παίξει τον ρόλο αυτού του τρίτου χωρίς να είναι κανείς συγκεκριμένος. Η ατομικότητα των συναλλαγών εγγυάται ότι οι δύο μεταβιβάσεις είτε εκτελούνται μαζί είτε καθόλου, ιδέα που έχει μελετηθεί εκτενώς στο πλαίσιο των atomic swaps [32]. Απομένει ένα πρακτικό ζήτημα: οι δύο πλευρές σπάνια είναι ταυτόχρονα παρούσες. Ο προσφέρων υποβάλλει την πρότασή του τώρα, και ο αποδέκτης θα τη δει αργότερα. Στο μεσοδιάστημα, η προσφορά πρέπει να παραμένει εκτελέσιμη.

Υπάρχουν δύο τρόποι να επιτευχθεί αυτό. Στον πρώτο, ο προσφέρων υπογράφει εκτός αλυσίδας μια εντολή και παραχωρεί στο συμβόλαιο της αγοράς έγκριση να μετακινήσει τα

στοιχεία του όταν βρεθεί αντισυμβαλλόμενος. Τα στοιχεία παραμένουν στο πορτοφόλι του, με συνέπεια η προσφορά να μπορεί να καταστεί ανεκτέλεστη αν στο μεταξύ τα μετακινήσει. Στον δεύτερο, τα προσφερόμενα στοιχεία *δεσμεύονται* (escrow) στην αλυσίδα κατά την υποβολή, υπό τον έλεγχο του συμβολαίου. Η προσφορά είναι τότε εγγυημένα εκτελέσιμη, με τίμημα ότι τα στοιχεία δεν είναι διαθέσιμα στον προσφέροντα μέχρι να αποσύρει την προσφορά. Το μοντέλο αντικειμένων του Sui, στο οποίο δεν υπάρχουν διαρκείς εγκρίσεις, ευνοεί τον δεύτερο τρόπο, και αυτόν ακολουθεί η εργασία.

Κεφάλαιο 3

Απαιτήσεις και σχεδίαση του συστήματος

Το κεφάλαιο περιγράφει το σύστημα πριν από την υλοποίησή του: ποιοι το χρησιμοποιούν, τι οφείλει να κάνει, ποιες ιδιότητες πρέπει να διατηρεί, και ποιες σχεδιαστικές αποφάσεις ελήφθησαν για να τα επιτύχει. Ακολουθείται η μεθοδολογία της [9]: ρόλοι, απαιτήσεις, περιπτώσεις χρήσης με διαγράμματα UML [33], αρχιτεκτονική και σχεδίαση διεπαφής.

3.1 Η γενική ιδέα

Το σύστημα είναι ένα marketplace στο οποίο δεν υπάρχουν τιμές. Κάθε χρήστης βλέπει τα NFTs των υπολοίπων και, αν κάποιο τον ενδιαφέρει, υποβάλλει στον κάτοχό του μια *προσφορά*: ένα σύνολο δικών του NFTs, ένα ποσό SUI, ή συνδυασμό των δύο. Ο κάτοχος βλέπει τις προσφορές που έχει λάβει και για καθεμία επιλέγει αν θα την αποδεχθεί ή θα την απορρίψει. Η αποδοχή ολοκληρώνει την ανταλλαγή αμέσως. Όσο η προσφορά εκκρεμεί, τα προσφερόμενα στοιχεία βρίσκονται δεσμευμένα στην αλυσίδα, και ο προσφέρων μπορεί να την ακυρώσει οποτεδήποτε για να τα πάρει πίσω.

Η εφαρμογή επίδειξης περιλαμβάνει τρεις συλλογές (Cars, Houses, Tickets), η καθεμία με τρεις βαθμίδες σπανιότητας (common, uncommon, rare), και επιτρέπει σε οποιονδήποτε να δημιουργήσει δωρεάν NFTs για να πειραματισθεί. Το πρωτόκολλο όμως δεν γνωρίζει τίποτε για τις συγκεκριμένες συλλογές: λειτουργεί με οποιονδήποτε τύπο αντικειμένου.

3.2 Ρόλοι χρηστών

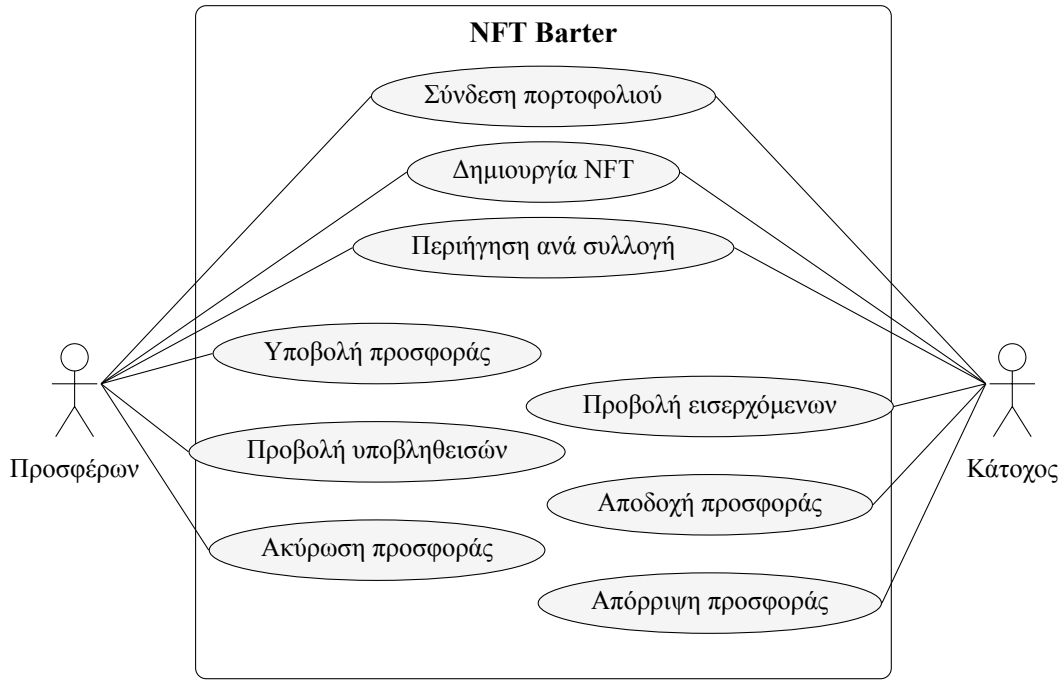
Το σύστημα δεν διακρίνει κατηγορίες λογαριασμών. Κάθε διεύθυνση μπορεί να εκτελέσει κάθε λειτουργία, και οι ρόλοι που ακολουθούν ορίζονται ως προς μια συγκεκριμένη προσφορά.

Προσφέρων (bidder). Ο χρήστης που υποβάλλει μια προσφορά. Είναι ο μόνος που μπορεί

να την ακυρώσει.

Κάτοχος (owner). Ο χρήστης που κατέχει το NFT στο οποίο στοχεύει η προσφορά. Μπορεί να την αποδεχθεί ή να την απορρίψει.

Ο ίδιος χρήστης είναι συνήθως προσφέρων σε κάποιες προσφορές και κάτοχος σε άλλες. Αξίζει να τονισθεί η απουσία τρίτου ρόλου: δεν υπάρχει διαχειριστής. Το Σχήμα 3.1 δείχνει το διάγραμμα περιπτώσεων χρήσης.



Σχήμα 3.1: Διάγραμμα περιπτώσεων χρήσης. Οι δύο ρόλοι ορίζονται ανά προσφορά· ο ίδιος χρήστης αναλαμβάνει και τους δύο.

3.3 Λειτουργικές απαιτήσεις

Οι λειτουργικές απαιτήσεις συγκεντρώνονται στον Πίνακα 3.1. Οι ΛΑ1 έως ΛΑ6 αφορούν το πρωτόκολλο και υλοποιούνται στο έξυπνο συμβόλαιο· οι ΛΑ7 έως ΛΑ10 αφορούν την εφαρμογή.

Πίνακας 3.1: Λειτουργικές απαιτήσεις

Κωδ.	Απαίτηση
ΛΑ1	Ο χρήστης δημιουργεί (mint) NFT με όνομα, περιγραφή, εικόνα, συλλογή και σπανιότητα.
ΛΑ2	Ο χρήστης υποβάλλει προσφορά που περιέχει μηδέν ή περισσότερα NFTs ενός τύπου και μηδέν ή περισσότερο SUI, με τουλάχιστον ένα από τα δύο μη μηδενικό.
ΛΑ3	Η προσφορά μπορεί να στοχεύει συγκεκριμένο NFT, οπότε μόνο με αυτό μπορεί να γίνει αποδεκτή.
ΛΑ4	Ο κάτοχος αποδέχεται προσφορά παραδίδοντας τα ζητούμενα NFTs· λαμβάνει όλα τα προσφερόμενα στοιχεία και ο προσφέρων λαμβάνει τα ζητούμενα, στην ίδια συναλλαγή.
ΛΑ5	Ο κάτοχος του στοχευόμενου NFT απορρίπτει προσφορά· όλα τα προσφερόμενα στοιχεία επιστρέφουν στον προσφέροντα.
ΛΑ6	Ο προσφέρων ακυρώνει προσφορά του· όλα τα προσφερόμενα στοιχεία επιστρέφουν σε αυτόν.
ΛΑ7	Η εφαρμογή παρουσιάζει τα διαθέσιμα NFTs ανά συλλογή, με κάτοχο και σπανιότητα, και επιτρέπει φιλτράρισμα ανά βαθμίδα σπανιότητας.
ΛΑ8	Η εφαρμογή παρουσιάζει στον χρήστη τα NFTs που κατέχει και, για καθένα, τις προσφορές που έχει λάβει, καθώς και όσα έχει δεσμεύσει σε δικές του προσφορές.
ΛΑ9	Η εφαρμογή παρουσιάζει στον χρήστη τις προσφορές που έχει υποβάλει και εκκρεμούν.
ΛΑ10	Για κάθε προσφορά η εφαρμογή δείχνει ρητά τι δίνει και τι λαμβάνει ο χρήστης.

3.4 Μη λειτουργικές απαιτήσεις

Όπως παρατηρεί και η [9], σε εφαρμογές αυτού του είδους είναι οι μη λειτουργικές απαιτήσεις που καθορίζουν την αρχιτεκτονική. Συγκεντρώνονται στον Πίνακα 3.2.

Πίνακας 3.2: Μη λειτουργικές απαιτήσεις

Κωδ.	Ιδιότητα	Απαίτηση
ΜΛΑ1	Ατομικότητα	Η ανταλλαγή είτε ολοκληρώνεται πλήρως και για τα δύο μέρη είτε δεν συμβαίνει καθόλου.
ΜΛΑ2	Διατήρηση στοιχείων	Κάθε δεσμευμένο στοιχείο καταλήγει είτε στον αντισυμβαλλόμενο (αποδοχή) είτε στον προσφέροντα (απόρριψη, ακύρωση). Δεν υπάρχει διαδρομή στην οποία χάνεται ή παραμένει εγκλωβισμένο.
ΜΛΑ3	Εξουσιοδότηση	Μόνο ο προσφέρων ακυρώνει· μόνο ο κάτοχος του στοχευόμενου NFT αποδέχεται ή απορρίπτει.
ΜΛΑ4	Απουσία έμπιστου τρίτου	Δεν υπάρχει θεματοφύλακας, διαχειριστής ή προνομιακό κλειδί, ούτε τέλη πέραν του gas.
ΜΛΑ5	Γενικότητα	Το πρωτόκολλο λειτουργεί με οποιονδήποτε τύπο αντικειμένου, χωρίς καταχώρηση συλλογών.
ΜΛΑ6	Ορθότητα υπό ταυτόχρονη πρόσβαση	Ταυτόχρονες ενέργειες στην ίδια προσφορά (π.χ. αποδοχή και ακύρωση) έχουν καλά ορισμένο αποτέλεσμα: εκτελείται ακριβώς μία.
ΜΛΑ7	Διαφάνεια	Η κατάσταση κάθε προσφοράς και τα δεσμευμένα στοιχεία είναι επαληθεύσιμα από οποιονδήποτε με δεδομένα της αλυσίδας.
ΜΛΑ8	Ευχρηστία	Κάθε ενέργεια απαιτεί μία υπογραφή. Η εφαρμογή λειτουργεί με κάθε πορτοφόλι του Wallet Standard, σε επιτραπέζια και κινητή οθόνη.

3.5 Κύκλος ζωής της προσφοράς

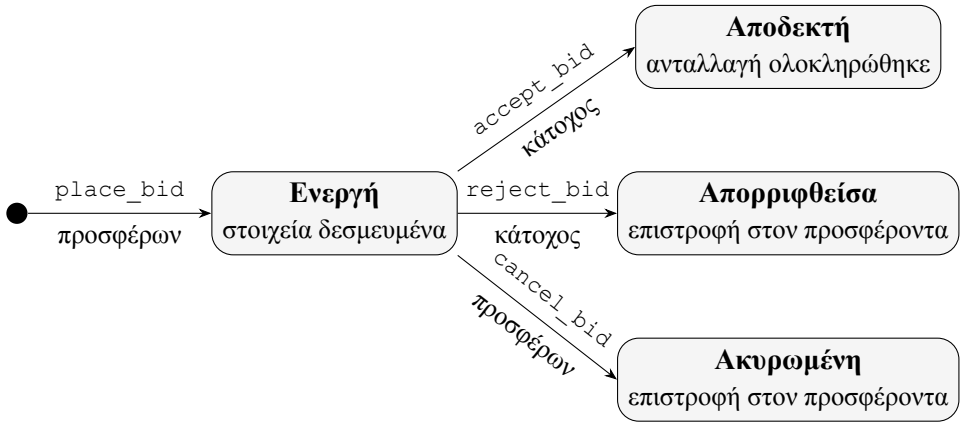
Μια προσφορά έχει μία μόνο ενεργή κατάσταση και τρεις τερματικές, όπως δείχνει το Σχήμα 3.2. Δεν υπάρχουν ενδιάμεσες καταστάσεις, διότι κάθε μετάβαση είναι μία ατομική συναλλαγή.

Ο κύκλος ζωής συνοδεύεται από τρεις αναλλοίωτες συνθήκες, τις οποίες η υλοποίηση οφείλει να διατηρεί και οι οποίες θα ελεγχθούν στο Κεφάλαιο 4:

Α1. Όσο η προσφορά είναι ενεργή, τα δεσμευμένα στοιχεία ανήκουν στο αντικείμενο της προσφοράς και σε κανέναν χρήστη.

Α2. Κάθε μετάβαση σε τερματική κατάσταση αδειάζει πλήρως τη δέσμευση και διαγράφει το αντικείμενο της προσφοράς. Καμία προσφορά δεν μπορεί να εκτελεστεί δύο φορές.

Α3. Τα στοιχεία που εξέρχονται από τη δέσμευση είναι ακριβώς εκείνα που εισήλθαν.



Σχήμα 3.2: Μηχανή καταστάσεων της προσφοράς. Σε κάθε τερματική κατάσταση το αντικείμενο Bid έχει διαγραφεί από την αλυσίδα.

3.6 Σχεδιαστικές αποφάσεις

Οι απαιτήσεις επιδέχονται περισσότερες από μία υλοποιήσεις. Η ενότητα καταγράφει τις κύριες αποφάσεις, τις εναλλακτικές που εξετάσθηκαν και τους λόγους της επιλογής.

3.6.1 Η προσφορά ως κοινόχρηστο αντικείμενο

Η προσφορά πρέπει να είναι προσβάσιμη από δύο διαφορετικούς χρήστες: τον προσφέροντα, για την ακύρωση, και τον κάτοχο, για την αποδοχή ή την απόρριψη. Αν ήταν αντικείμενο του προσφέροντος, ο κάτοχος δεν θα μπορούσε να τη χρησιμοποιήσει σε συναλλαγή του. Αν μεταβιβαζόταν στον κάτοχο, ο προσφέρων θα έχανε τη δυνατότητα ακύρωσης και θα εξαρτιόταν από την καλή θέληση του αποδέκτη για να πάρει πίσω τα στοιχεία του, κάτι που παραβιάζει τη ΜΛΑ4. Η μόνη μορφή ιδιοκτησίας που ικανοποιεί και τους δύο είναι το κοινόχρηστο αντικείμενο, με τους ελέγχους εξουσιοδότησης να γίνονται από τον κώδικα.

Εξετάσθηκε και η εναλλακτική ενός ενιαίου κοινόχρηστου *μητρώου* που θα περιείχε όλες τις προσφορές. Απορρίφθηκε διότι κάθε συναλλαγή του συστήματος θα έπρεπε τότε να κλειδώσει το ίδιο αντικείμενο, και άσχετες μεταξύ τους προσφορές θα ανταγωνίζονταν για διάταξη. Με ένα κοινόχρηστο αντικείμενο *ανά προσφορά*, ανταγωνισμός υπάρχει μόνο μεταξύ ενεργειών στην ίδια προσφορά, δηλαδή ακριβώς εκεί όπου η διάταξη είναι απαραίτητη (ΜΛΑ6). Το κόστος της επιλογής είναι ότι οι προσφορές δεν είναι απαριθμήσιμες από κάποιο κεντρικό σημείο· το ζήτημα αντιμετωπίζεται με συμβάντα, όπως εξηγείται παρακάτω.

3.6.2 Δέσμευση με δυναμικά πεδία αντικειμένων

Τα προσφερόμενα NFTs πρέπει να αποθηκευθούν μέσα στην προσφορά. Η απλούστερη λύση, ένα πεδίο `vector<Offered>`, θα περιτύλιγε τα αντικείμενα: θα έπαυαν να εμφανίζονται ως αυτοτελή αντικείμενα και ένας τρίτος δεν θα μπορούσε εύκολα να επαληθεύσει

τι περιέχει η προσφορά. Προτιμήθηκαν τα δυναμικά πεδία αντικειμένων (Ενότητα 2.3.3), με τα οποία κάθε δεσμευμένο NFT διατηρεί το ID του και εμφανίζεται στους εξερευνητές της αλυσίδας ως παιδί της προσφοράς (ΜΛΑ7). Ως κλειδί χρησιμοποιείται ένας αύξων αριθμός, περιτυλιγμένος σε ειδικό τύπο ώστε να μην συγκρούεται με πεδία που θα μπορούσε να προσθέσει άλλος κώδικας.

3.6.3 Γενικοί τύποι για τις συλλογές

Η υποστήριξη διαφορετικών συλλογών (ΜΛΑ5) θα μπορούσε να επιτευχθεί με ένα μητρώο επιτρεπόμενων τύπων, το οποίο όμως προϋποθέτει διαχειριστή. Επιλέχθηκε η παραμετροποίηση της προσφοράς με δύο παραμέτρους τύπου, `Bid<Offered, Requested>`. Ο προσφέρων καθορίζει και τους δύο κατά την υποβολή, και οι συναρτήσεις αποδοχής, απόρριψης και ακύρωσης απαιτούν τους ίδιους τύπους για να δεχθούν το αντικείμενο. Ο έλεγχος γίνεται από την εικονική μηχανή: είναι αδύνατο να γίνει αποδεκτή μια προσφορά για `Requested = Shield` παραδίδοντας αντικείμενο άλλου τύπου, διότι η κλήση δεν θα περάσει καν τον έλεγχο τύπων.

Η επιλογή έχει έναν περιορισμό που πρέπει να δηλωθεί: τα προσφερόμενα NFTs μιας προσφοράς έχουν όλα τον ίδιο τύπο `Offered`. Ο προσφέρων μπορεί να δώσει αντικείμενα τύπου Α για ένα αντικείμενο τύπου Β, όχι όμως αντικείμενα τύπων Α και Γ μαζί στην ίδια προσφορά. Η άρση του περιορισμού συζητείται στο Κεφάλαιο 8.

3.6.4 Στοχευμένες προσφορές

Μια προσφορά μπορεί να φέρει προαιρετικά το ID του NFT που ζητά. Όταν το φέρει, η αποδοχή απαιτεί να παραδοθεί ακριβώς αυτό το αντικείμενο. Όταν δεν το φέρει, η προσφορά είναι *ανοικτή*: μπορεί να την αποδεχθεί οποιοσδήποτε παραδώσει το ζητούμενο πλήθος αντικειμένων του τύπου `Requested`. Το συμβόλαιο υποστηρίζει και τις δύο μορφές. Η εφαρμογή χρησιμοποιεί αποκλειστικά τη στοχευμένη, που αντιστοιχεί στο σενάριο «θέλω αυτό το συγκεκριμένο αντικείμενο».

Η απόρριψη έχει νόημα μόνο για στοχευμένες προσφορές, αφού μια ανοικτή δεν απευθύνεται σε κανέναν συγκεκριμένα. Η εξουσιοδότησή της θέτει ένα ενδιαφέρον ερώτημα: πώς αποδεικνύει ο καλών ότι κατέχει το στοχευόμενο NFT; Στο Sui δεν χρειάζεται να ερωτηθεί κάποιο μητρώο. Αρκεί η συνάρτηση να ζητήσει ως όρισμα μια *αναφορά* στο αντικείμενο: μόνο ο κάτοχος ενός αντικειμένου διεύθυνσης μπορεί να το περάσει ως είσοδο σε συναλλαγή, άρα η ίδια η ικανότητα να κληθεί η συνάρτηση αποτελεί την απόδειξη.

Αξίζει επίσης να σημειωθεί ότι η προσφορά στοχεύει το *αντικείμενο* και όχι τον κάτοχό του. Αν το NFT αλλάξει χέρια όσο η προσφορά εκκρεμεί, ο νέος κάτοχος είναι εκείνος που μπορεί πλέον να την αποδεχθεί ή να την απορρίψει.

3.6.5 Το νόμισμα ως μέρος της προσφοράς

Το προσφερόμενο SUI αποθηκεύεται ως `Balance<SUI>` σε πεδίο της προσφοράς. Η συνάρτηση υποβολής δέχεται πάντοτε ένα `Coin<SUI>`, ενδεχομένως μηδενικής αξίας, ώστε να υπάρχει μία υπογραφή συνάρτησης για όλους τους συνδυασμούς. Η μόνη απαίτηση είναι να προσφέρεται *κάτι*: τουλάχιστον ένα NFT ή θετικό ποσό.

3.6.6 Ανακάλυψη μέσω συμβάντων

Εφόσον δεν υπάρχει μητρώο προσφορών και ένας πλήρης κόμβος δεν απαριθμεί κοινόχρηστα αντικείμενα ανά τύπο (Ενότητα 2.2.6), η εφαρμογή χρειάζεται άλλον τρόπο να βρει τις προσφορές. Το συμβόλαιο εκπέμπει ένα συμβάν για κάθε μετάβαση του κύκλου ζωής, καθώς και για κάθε δημιουργία NFT. Η εφαρμογή διαβάζει τα συμβάντα υποβολής, συλλέγει τα IDs των προσφορών, και ανακτά τα αντίστοιχα αντικείμενα· όσα δεν υπάρχουν πλέον έχουν φτάσει σε τερματική κατάσταση. Η προσέγγιση δεν απαιτεί καμία υποδομή πέραν ενός πλήρους κόμβου. Τα όριά της ως προς την κλιμάκωση συζητούνται στην Ενότητα 7.6.

3.7 Περιπτώσεις χρήσης

3.7.1 ΠΧ1: Σύνδεση πορτοφολιού

ΠΧ1 — Σύνδεση πορτοφολιού	
Ηθοποιός	Οποιοσδήποτε χρήστης
Προϋποθέσεις	Υπάρχει εγκατεστημένο πορτοφόλι συμβατό με το Wallet Standard.
Κύρια ροή	1. Ο χρήστης επιλέγει «Connect Wallet». 2. Η εφαρμογή εμφανίζει τα διαθέσιμα πορτοφόλια. 3. Ο χρήστης επιλέγει ένα και εγκρίνει τη σύνδεση. 4. Η εφαρμογή εμφανίζει τη διεύθυνσή του και ενεργοποιεί τις λειτουργίες που απαιτούν υπογραφή.
Εναλλακτικές	Ο χρήστης απορρίπτει τη σύνδεση: η εφαρμογή παραμένει σε κατάσταση ανάγνωσης.
Μετασυνθήκες	Η επιλογή αποθηκεύεται τοπικά, ώστε η σύνδεση να αποκατασταθεί αυτόματα στην επόμενη επίσκεψη.

Χωρίς συνδεδεμένο πορτοφόλι ο χρήστης μπορεί να περιηγηθεί στις συλλογές, όχι όμως να υποβάλει προσφορά ή να δει τις σελίδες που αφορούν δικά του στοιχεία.

3.7.2 ΠΧ2: Δημιουργία NFT

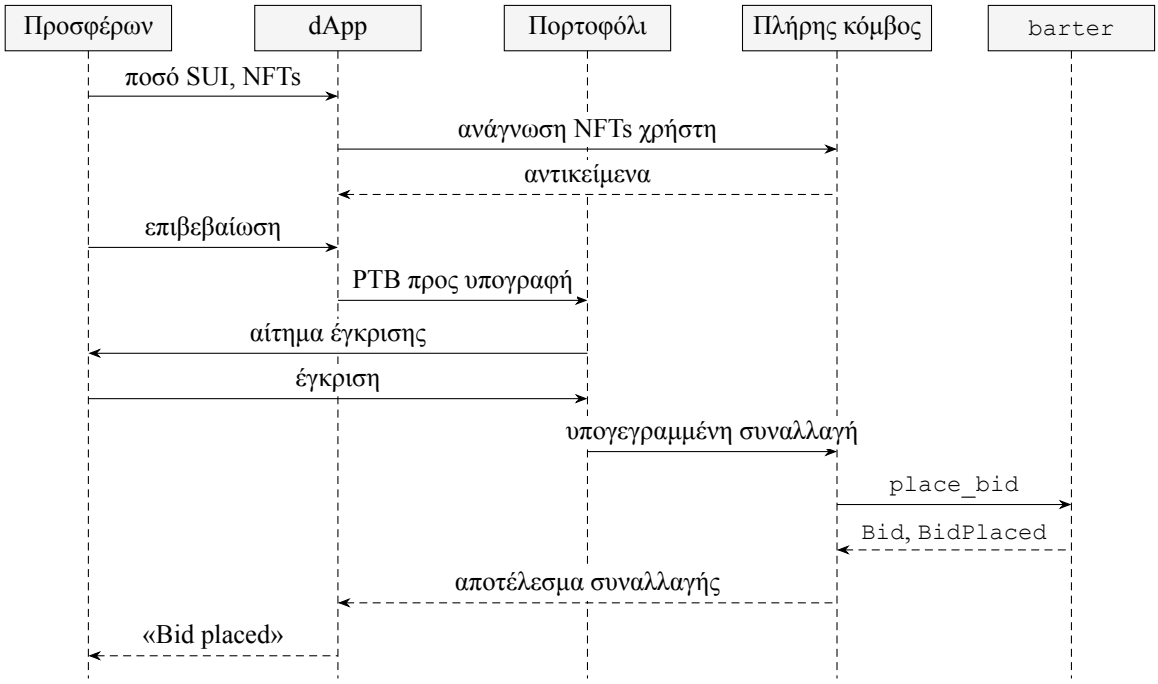
ΠΧ2 — Δημιουργία NFT	
Ηθοποιός	Συνδεδεμένος χρήστης
Κύρια ροή	1. Ο χρήστης επιλέγει ένα από τα εννέα είδη του καταλόγου και ποσότητα. 2. Η εφαρμογή συνθέτει ένα PTB με μία κλήση `mint` ανά τεμάχιο και μία τελική εντολή που μεταβιβάζει τα αποτελέσματα στη διεύθυνσή του. 3. Ο χρήστης υπογράφει. 4. Τα νέα NFTs βρίσκονται στο πορτοφόλι του.
Εναλλακτικές	Ο χρήστης απορρίπτει την υπογραφή ή δεν έχει SUI για gas: εμφανίζεται μήνυμα αποτυχίας και δεν αλλάζει τίποτε. Σε δίκτυο δοκιμών μπορεί να ζητήσει SUI από το κουμπί «Faucet» της εφαρμογής. Αν ζητήσει δεύτερη δημιουργία ενώ η πρώτη εκκρεμεί, η δεύτερη μπαίνει σε ουρά και εκτελείται μετά την πρώτη.
Μετασυνθήκες	Εκπέμπεται συμβάν `NftMinted` ανά τεμάχιο. Τα NFTs εμφανίζονται στη σελίδα της συλλογής τους.

3.7.3 ΠΧ3: Υποβολή προσφοράς

ΠΧ3 — Υποβολή προσφοράς	
Ηθοποιός	Προσφέρων
Προϋποθέσεις	Συνδεδεμένο πορτοφόλι. Το NFT-στόχος ανήκει σε άλλον χρήστη.
Κύρια ροή	1. Ο χρήστης επιλέγει «Place a bid» σε ένα NFT. 2. Ορίζει ποσό SUI ή παραλείπει το βήμα («Skip»). 3. Επιλέγει προαιρετικά NFTs από το πορτοφόλι του. 4. Ελέγχει τη σύνοψη «δίνεις / παίρνεις» και επιβεβαιώνει. 5. Η εφαρμογή συνθέτει το PTB του Σχήματος 2.2 και ο χρήστης υπογράφει. 6. Δημιουργείται κοινόχρηστο αντικείμενο `Bid` με τα στοιχεία δεσμευμένα.
Εναλλακτικές	3α. Δεν έχει επιλεγεί ούτε SUI ούτε NFT: η εφαρμογή δεν επιτρέπει τη μετάβαση στην επισκόπηση. 5α. Ο χρήστης απορρίπτει την υπογραφή: δεν δεσμεύεται τίποτε.
Μετασυνθήκες	Τα προσφερόμενα στοιχεία δεν βρίσκονται πλέον στο πορτοφόλι του προσφέροντος. Εκπέμπεται `BidPlaced`. Η προσφορά εμφανίζεται στον κάτοχο ως εισερχόμενη.

Το Σχήμα 3.3 δείχνει τη ροή μηνυμάτων. Αξίζει να παρατηρηθεί ότι η εφαρμογή δεν αποστέλλει η ίδια τη συναλλαγή: την παραδίδει στο πορτοφόλι, το οποίο την παρουσιάζει

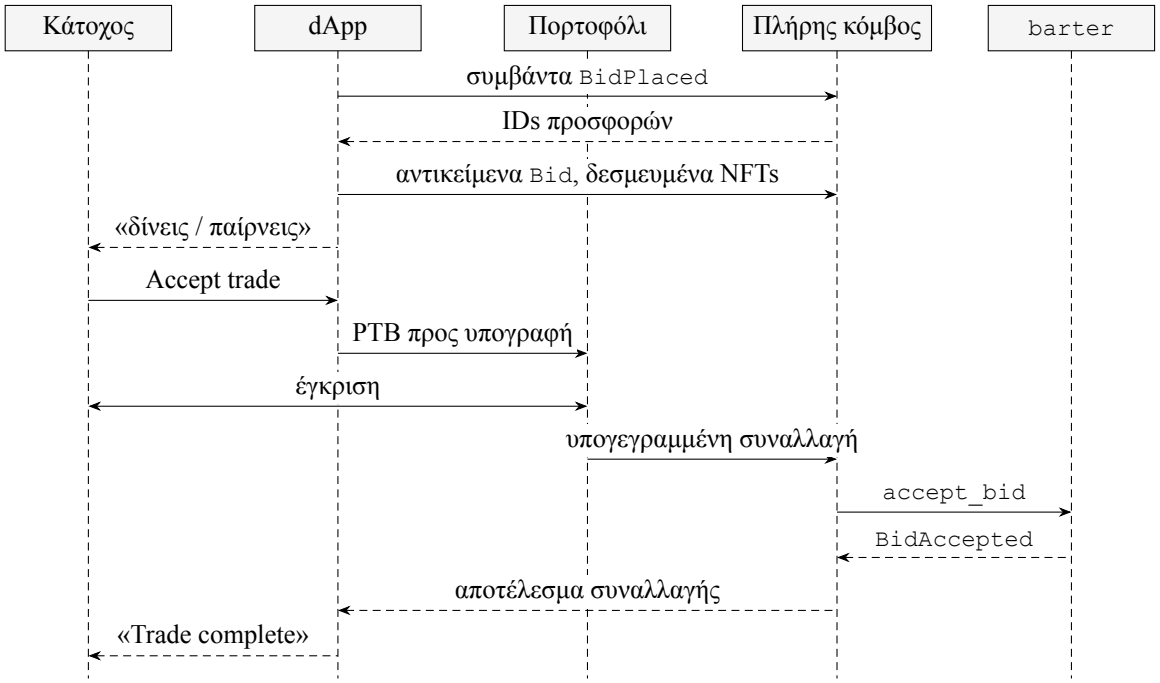
στον χρήστη, την υπογράφει και την υποβάλλει.



Σχήμα 3.3: Διάγραμμα ακολουθίας της υποβολής προσφοράς.

3.7.4 ΠΧ4: Αποδοχή προσφοράς

ΠΧ4 — Αποδοχή προσφοράς	
Ηθοποιός	Κάτοχος
Προϋποθέσεις	Υπάρχει ενεργή προσφορά που στοχεύει NFT του χρήστη.
Κύρια ροή	1. Ο χρήστης ανοίγει τις εισερχόμενες προσφορές, από το NFT στη σελίδα «My NFTs» ή από την καρτέλα «Received» της σελίδας «My bids». 2. Εξετάζει τι δίνει και τι λαμβάνει. 3. Επιλέγει «Accept trade» και υπογράφει. 4. Στην ίδια συναλλαγή το NFT του μεταβιβάζεται στον προσφέροντα και τα δεσμευμένα στοιχεία σε αυτόν.
Εναλλακτικές	3α. Η προσφορά ακυρώθηκε στο μεταξύ από τον προσφέροντα: η συναλλαγή απορρίπτεται πριν την υπογραφή, εμφανίζεται μήνυμα αποτυχίας, και η λίστα ανανεώνεται στην επόμενη ανάγνωση. 3β. Το στοχευόμενο NFT άλλαξε κάτοχο: η προσφορά παύει να εμφανίζεται στον παλιό κάτοχο και εμφανίζεται στον νέο.
Μετασυνθήκες	Το αντικείμενο `Bid` έχει διαγραφεί. Εκπέμπεται `BidAccepted`.



Σχήμα 3.4: Διάγραμμα ακολουθίας της αποδοχής. Τα τρία πρώτα μηνύματα δείχνουν πώς η εφαρμογή ανακαλύπτει τις εισερχόμενες προσφορές χωρίς indexer.

3.7.5 ΠΧ5: Απόρριψη προσφοράς

ΠΧ5 — Απόρριψη προσφοράς	
Ηθοποιός	Κάτοχος
Προϋποθέσεις	Υπάρχει ενεργή στοχευμένη προσφορά για NFT του χρήστη.
Κύρια ροή	1. Ο χρήστης επιλέγει «Reject» σε μια εισερχόμενη προσφορά και υπογράφει. 2. Το συμβόλαιο επαληθεύει ότι ο καλών παρουσιάζει το στοχευόμενο NFT. 3. Όλα τα δεσμευμένα στοιχεία επιστρέφουν στον προσφέροντα.
Μετασυνθήκες	Το NFT του κατόχου δεν μετακινείται. Το `Bid` διαγράφεται. Εκπέμπεται `BidRejected`.

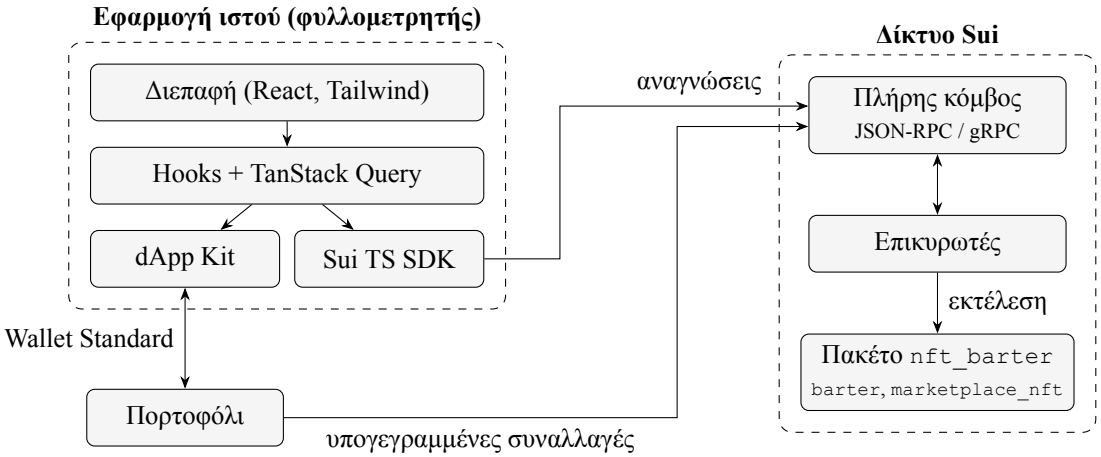
Η απόρριψη δεν είναι απαραίτητη για την ορθότητα· ο κάτοχος θα μπορούσε απλώς να αγνοήσει μια προσφορά. Προσφέρει όμως δύο πράγματα: καθαρίζει τη λίστα του κατόχου, και αποδεσμεύει άμεσα τα στοιχεία του προσφέροντος χωρίς να χρειαστεί εκείνος να ενεργήσει.

3.7.6 ΠΧ6: Ακύρωση προσφοράς

ΠΧ6 — Ακύρωση προσφοράς	
Ηθοποιός	Προσφέρων
Προϋποθέσεις	Ο χρήστης έχει υποβάλει προσφορά που είναι ακόμη ενεργή.
Κύρια ροή	1. Ο χρήστης ανοίγει την καρτέλα «Placed» της σελίδας «My bids». 2. Επιλέγει «Cancel bid» και υπογράφει. 3. Όλα τα δεσμευμένα στοιχεία επιστρέφουν στο πορτοφόλι του.
Εναλλακτικές	2α. Η προσφορά έγινε αποδεκτή στο μεταξύ: η συναλλαγή απορρίπτεται πριν την υπογραφή, εμφανίζεται μήνυμα αποτυχίας, και η λίστα ανανεώνεται στην επόμενη ανάγνωση.
Μετασυνθήκες	Το `Bid` διαγράφεται. Εκπέμπεται `BidCancelled`.

3.8 Αρχιτεκτονική

Το σύστημα αποτελείται από δύο μόνο μέρη που αναπτύχθηκαν στο πλαίσιο της εργασίας: το πακέτο Move που δημοσιεύεται στην αλυσίδα, και την εφαρμογή ιστού που εκτελείται εξ ολοκλήρου στον φυλλομετρητή. Δεν υπάρχει διακομιστής εφαρμογής ούτε βάση δεδομένων. Το Σχήμα 3.5 δείχνει τα μέρη και τις μεταξύ τους εξαρτήσεις.



Σχήμα 3.5: Αρχιτεκτονική του συστήματος.

Η εφαρμογή επικοινωνεί με το δίκτυο με δύο τρόπους. Οι *αναγνώσεις* (αντικείμενα, συμβάντα) γίνονται απευθείας από τον φυλλομετρητή προς έναν πλήρη κόμβο, μέσω του Sui TypeScript SDK. Οι *εγγραφές* περνούν πάντοτε από το πορτοφόλι: η εφαρμογή κατασκευάζει τη συναλλαγή, το πορτοφόλι την υπογράφει με το κλειδί του χρήστη και την υποβάλλει. Το ιδιωτικό κλειδί δεν γίνεται ποτέ προσβάσιμο στην εφαρμογή.

Συνέπεια της αρχιτεκτονικής είναι ότι η εφαρμογή δεν αποτελεί σημείο εμπιστοσύνης. Ένας χρήστης που δεν την εμπιστεύεται μπορεί να καλέσει το συμβόλαιο απευθείας από τη γραμμή εντολών ή να χρησιμοποιήσει διαφορετική διεπαφή. Όλες οι εγγυήσεις των ΜΛΑ1 έως ΜΛΑ7 παρέχονται από το συμβόλαιο.

3.9 Σχεδίαση διεπαφής

Η διεπαφή σχεδιάσθηκε εκ νέου σε δεύτερη φάση της εργασίας, αφού η πρώτη υλοποίηση, αν και λειτουργικά πλήρης, είχε τη μορφή γενικού πίνακα διαχείρισης και δεν ανεδείκνυε τη φύση της συναλλαγής. Οι αρχές του επανασχεδιασμού ήταν οι εξής.

«Δίνεις ⇄ παίρνεις». Κάθε προσφορά παρουσιάζεται, σε όλες τις οθόνες, ως δύο πλευρές μιας ανταλλαγής από την οπτική του χρήστη που την κοιτάζει, και ποτέ ως φόρμα ή ως γραμμή πίνακα. Η ίδια προσφορά διαβάζεται αντίστροφα από τον προσφέροντα και από τον κάτοχο (ΛΑ10).

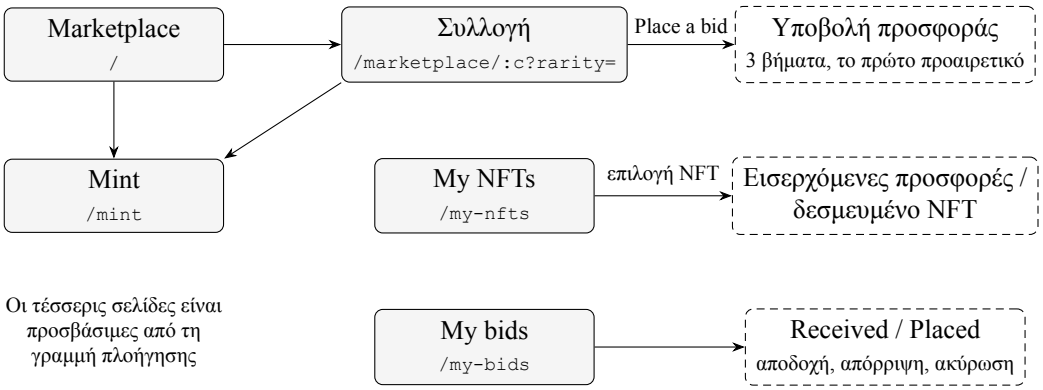
Χρώμα με σημασία. Τρία χρώματα έχουν σταθερό ρόλο: ένα για τις ενέργειες του χρήστη, ένα για ό,τι αφορά προσφορές και ανταλλαγές, και ένα για ό,τι παραπέμπει στην αλυσίδα (διευθύνσεις, σύνδεσμοι προς τον εξερευνητή).

Σπανιότητα ως badge. Οι τρεις βαθμίδες αποδίδονται ως χαλκός, άργυρος και χρυσός, σε τρεις μορφές (ετικέτα, κουκκίδα, πλαίσιο), ώστε να διαβάζονται σε κάθε μέγεθος, από την κάρτα έως τη μικρογραφία.

Σύνθεση προσφοράς σε βήματα. Η υποβολή γίνεται σε τρία βήματα πλήρους οθόνης (SUI, NFTs, επισκόπηση), με τη σύνοψη της ανταλλαγής μόνιμα ορατή.

Ορατή εκκρεμότητα. Κάθε συναλλαγή διαρκεί μερικά δευτερόλεπτα. Το κουμπί που την εκκίνησε δείχνει την πρόοδο, και οι ενέργειες που θα συγκρούονταν μαζί της είτε απενεργοποιούνται έως ότου ολοκληρωθεί είτε, στη δημιουργία NFT, μπαίνουν σε ουρά. Κάθε ολοκληρωμένη συναλλαγή συνοδεύεται από σύνδεσμο προς τον εξερευνητή της αλυσίδας.

Το Σχήμα 3.6 δείχνει τις οθόνες και τις μεταβάσεις μεταξύ τους. Οι μακέτες υψηλής πιστότητας σχεδιάστηκαν ως διαδραστικό πρωτότυπο HTML για επιτραπέζια και κινητή οθόνη και υλοποιήθηκαν όπως περιγράφεται στο Κεφάλαιο 5. Οι τελικές οθόνες, σε πλάτη 1280 και 390 εικονοστοιχείων, παρουσιάζονται στο Κεφάλαιο 6.



Σχήμα 3.6: Οθόνες της εφαρμογής και μεταβάσεις. Με συνεχές περίγραμμα οι σελίδες, με διακεκομμένο τα παράθυρα διαλόγου.

Κεφάλαιο 4

Υλοποίηση του έξυπνου συμβολαίου

Το κεφάλαιο περιγράφει το πακέτο Move που υλοποιεί το πρωτόκολλο του Κεφαλαίου 3. Η παρουσίαση ακολουθεί τη δομή της αντίστοιχης ενότητας της [9]: μοντέλο δεδομένων, συναρτήσεις, έλεγχοι και ανάπτυξη στο δίκτυο, με την προσθήκη μιας ανάλυσης ασφάλειας. Ο πλήρης πηγαίος κώδικας παρατίθεται στο Παράρτημα Α′· τα αποσπάσματα του κεφαλαίου αντλούνται απευθείας από τα ίδια αρχεία και διατηρούν την αρίθμηση γραμμών τους.

4.1 Δομή του πακέτου

Το πακέτο `nft_barter` αποτελείται από δύο modules, που χωρίζονται με κριτήριο ποιες αναλλοίωτες συνθήκες προστατεύει το καθένα: το `marketplace_nft` (85 γραμμές) ορίζει τον τύπο NFT της εφαρμογής και τη δημιουργία του, ενώ το `barter` (238 γραμμές) ορίζει την προσφορά, τη δέσμευση και τον κύκλο ζωής της. Το πρώτο δεν γνωρίζει τίποτε για προσφορές και το δεύτερο τίποτε για τον συγκεκριμένο τύπο NFT. Η σουίτα ελέγχων (515 γραμμές) ακολουθεί την ίδια διαίρεση. Η δομή του αποθετηρίου είναι η ακόλουθη:

```
marketplace/
  Move.toml                        # ορισμόςπακέτου , έκδοσηγλώσσας  2024
  Published.toml                   # διευθύνσειςδημοσίευσηςανάδίκτυο
  sources/marketplace_nft.move     # τύπος NFT, Display, mint
  sources/barter.move              # Bid, δέσμευση, accept / cancel /
     reject
  tests/barter_fixture.move        # σκηνικόελέγχωνκαιδύοψευδοτύποι    -
  tests/barter_tests.move          # 18 έλεγχοιτουκύκλουζωής
  tests/marketplace_nft_tests.move
  scripts/                         # deploy.sh, dev.sh, sync-env.sh, ...
  app/                             # ηεφαρμογήιστούΚεφάλαιο   ( 5)
```

Απόσπασμα κώδικα 4.1: Δομή του αποθετηρίου

Το `Move.toml` δηλώνει το όνομα του πακέτου, τη σταθερή έκδοση 2024 της γλώσσας και τα δίκτυα στα οποία δημοσιεύεται. Δεν δηλώνονται ρητές εξαρτήσεις· η βασική βιβλιοθήκη (`std`) και το πλαίσιο του Sui (`sui`) παρέχονται σιωπηρά από τη σύγχρονη αλυσίδα εργαλείων. Το πακέτο δεν εξαρτάται από κανένα πακέτο τρίτων.

4.1.1 Συμβάσεις κώδικα

Ο κώδικας ακολουθεί ένα σύνολο δημοσιευμένων κανόνων ύφους και αρχιτεκτονικής για Sui Move [34], που συνοψίζονται εδώ επειδή εξηγούν αρκετές από τις επιλογές που ακολουθούν.

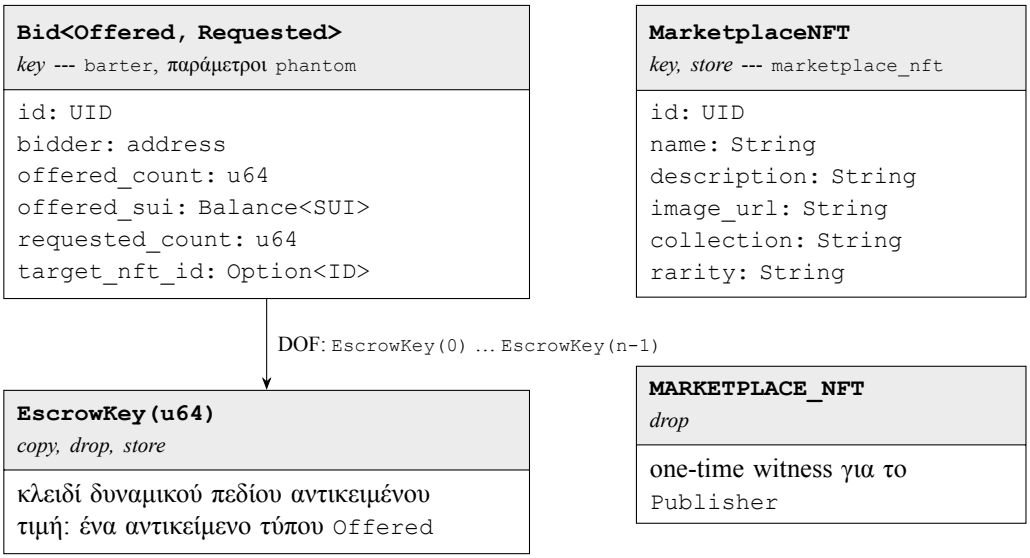
- **Ένα module ανά ιδιοκτήτη αναλλοίωτων.** Ο κώδικας που μοιράζεται την ίδια εξουσιοδότηση, τον ίδιο κύκλο ζωής και τον ίδιο λόγο αλλαγής μένει μαζί· η διαίρεση γίνεται μόνο σε πραγματικά όρια, όχι για να μικρύνουν τα αρχεία.
- **Σταθερή διάταξη ενοτήτων.** Κάθε αρχείο έχει τις ίδιες επικεφαλίδες με την ίδια σειρά: Errors, Public Structs, Private Initialization, Public Functions, Private Functions, Test-Only Functions.
- **Περιγραφικά σφάλματα.** Οι κωδικοί σφαλμάτων δηλώνονται με την επισημείωση `#[error(code = N)]` και φέρουν μήνυμα κειμένου (Ενότητα 4.5).
- **Μικρές συναρτήσεις με ορατή σειρά φάσεων.** Κάθε δημόσια συνάρτηση είναι μια σύντομη ακολουθία: έλεγχοι, μετακίνηση στοιχείων, διαγραφή, συμβάν. Οι φάσεις που επαναλαμβάνονται ζουν σε ιδιωτικές βοηθητικές συναρτήσεις.
- **Σύνταξη δέκτη και μακροεντολές της βιβλιοθήκης.** Χρησιμοποιείται η σύνταξη `bid.escrow(…)` αντί για `escrow(&mut bid, …)`, και οι μακροεντολές `do!` και `any!` της `std` αντί για χειρόγραφους βρόχους.
- **Έλεγχοι ως προτάσεις συμπεριφοράς.** Οι έλεγχοι ονομάζονται από αυτό που αποδεικνύουν και μοιράζονται ένα κοινό σκηνικό (Ενότητα 4.7).

4.2 Μοντέλο δεδομένων

Τα δύο modules ορίζουν τέσσερις δομές κατάστασης και πέντε δομές συμβάντων. Το Σχήμα 4.1 απεικονίζει τις πρώτες σε μορφή διαγράμματος κλάσεων.

4.2.1 Η δομή Bid

```
27 /// An open offer of `offered_count` NFTs of type `Offered` plus `
       offered_sui`,
28 /// escrowed as dynamic object fields, for `requested_count` NFTs of type
       `Requested`.
```



Σχήμα 4.1: Οι δομές κατάστασης των δύο modules. Το `Bid` δεν περιέχει τα προσφερόμενα NFTs ως πεδίο· αυτά προσαρτώνται ως δυναμικά πεδία αντικειμένων με κλειδιά `EscrowKey(0)`, `EscrowKey(1)`, κ.ο.κ.

```
29 public struct Bid<phantom Offered: key + store, phantom Requested: key +
       store> has key {
30     id: UID,
31     bidder: address,
32     offered_count: u64,
33     offered_sui: Balance<SUI>,
34     requested_count: u64,
35     /// When set, the counter offer must include this NFT and only its
           owner may reject.
36     target_nft_id: Option<ID>,
37 }
38
39 /// Dynamic object field key of the `n`-th escrowed NFT.
40 public struct EscrowKey(u64) has copy, drop, store;
```

Απόσπασμα κώδικα 4.2: Η δομή της προσφοράς και το κλειδί δέσμευσης

Η δομή `Bid` είναι παραμετρική ως προς δύο τύπους. Και οι δύο περιορίζονται σε `key + store`, δηλαδή σε αντικείμενα που μπορούν να αποθηκευθούν και να μεταβιβασθούν ελεύθερα, ιδιότητες που έχει κάθε τυπικό NFT. Και οι δύο είναι `phantom`, αφού κανένα πεδίο δεν έχει τύπο `Offered` ή `Requested`: οι παράμετροι υπάρχουν μόνο για να διακρίνουν, σε επίπεδο τύπων, μια προσφορά «σπαθιών για ασπίδα» από μια προσφορά «ασπίδων για σπαθί». Ο Πίνακας 4.1 εξηγεί τα πεδία.

Πίνακας 4.1: Πεδία της δομής `Bid`

Πεδίο	Ρόλος
`id`	Το μοναδικό αναγνωριστικό του αντικειμένου. Σε αυτό προσαρτώνται τα δυναμικά πεδία.
`bidder`	Η διεύθυνση του προσφέροντος. Χρησιμοποιείται για την εξουσιοδότηση της ακύρωσης και ως παραλήπτης σε κάθε διαδρομή επιστροφής.
`offered_count`	Το πλήθος των δεσμευμένων NFTs, άρα και το εύρος των κλειδιών `EscrowKey`.
`offered_sui`	Το δεσμευμένο ποσό SUI, ως `Balance` (ενδεχομένως μηδενικό).
`requested_count`	Το πλήθος αντικειμένων τύπου `Requested` που ζητούνται.
`target_nft_id`	Προαιρετικά, το ID του συγκεκριμένου NFT που ζητείται.

Το `Bid` έχει μόνο την ικανότητα `key`: προορίζεται αποκλειστικά για κοινόχρηστο αντικείμενο κορυφαίου επιπέδου και κοινοποιείται με την `transfer::share_object`. Η απουσία του `store` σημαίνει ότι κανένα άλλο πακέτο δεν μπορεί να το περιτυλίξει ή να το μεταβιβάσει· ο μόνος τρόπος να φύγει μια προσφορά από την αλυσίδα είναι οι τρεις συναρτήσεις τερματισμού του module. Αξίζει επίσης να σημειωθεί τι *δεν* αποθηκεύεται: τα ονόματα των δύο τύπων. Αυτά περιέχονται ήδη στον πλήρη τύπο του αντικειμένου, όπως τον αναφέρει κάθε κόμβος, και η αντιγραφή τους σε πεδία θα ήταν πλεονασμός. Το κλειδί `EscrowKey` είναι μια δομή-περιτύλιγμα ενός ακεραίου. Η χρήση ιδιαίτερου τύπου αντί για γυμνό `u64` εξασφαλίζει ότι τα πεδία δέσμευσης δεν θα συγκρουσθούν ποτέ με δυναμικά πεδία άλλης προέλευσης, διότι το κλειδί ενός δυναμικού πεδίου περιλαμβάνει και τον τύπο του.

4.2.2 Ο τύπος NFT της εφαρμογής

```
10 /// One-time witness; claims the `Publisher` that owns the `Display`.
11 public struct MARKETPLACE_NFT has drop {}
12
13 /// A tradeable collectible. Any `key + store` type can be bartered; this
       one
14 /// exists so the marketplace has something to mint.
15 public struct MarketplaceNFT has key, store {
16     id: UID,
17     name: String,
18     description: String,
19     image_url: String,
20     collection: String,
```

```
    rarity: String,
}

/// A new NFT exists. Lets clients discover NFTs without an indexer.
public struct NftMinted has copy, drop {
    nft_id: ID,
}
```

Απόσπασμα κώδικα 4.3: Το one-time witness, ο τύπος `MarketplaceNFT` και το συμβάν δημιουργίας

Ο `MarketplaceNFT` είναι ο τύπος που δημιουργεί η εφαρμογή επίδειξης. Η συλλογή και η σπανιότητα είναι πεδία κειμένου, ώστε ένας μόνο τύπος να καλύπτει και τις τρεις συλλογές της εφαρμογής. Πρέπει να τονισθεί ότι το πρωτόκολλο προσφορών *δεν* εξαρτάται από αυτόν τον τύπο: το `barter` δεν τον εισάγει καν, και οι έλεγχοι της Ενότητας 4.7 ασκούν τις προσφορές με δύο άσχετους μεταξύ τους τύπους.

4.3 Αρχικοποίηση και Display

```
fun init(otw: MARKETPLACE_NFT, ctx: &mut TxContext) {
    let publisher = package::claim(otw, ctx);

    let mut nft_display = display::new_with_fields<MarketplaceNFT>(
        &publisher,
        vector[b"name", b"description", b"image_url", b"collection", b"
            rarity"].map!(|key| key.to_string()),
        vector[b"{name}", b"{description}", b"{image_url}", b"{collection}
            ", b"{rarity}"].map!(
            |template| template.to_string(),
        ),
        ctx,
    );
    nft_display.update_version();

    transfer::public_transfer(publisher, ctx.sender());
    transfer::public_transfer(nft_display, ctx.sender());
}
```

Απόσπασμα κώδικα 4.4: Η συνάρτηση `init` του `marketplace_nft`

Η `init` εκτελείται μία φορά, κατά τη δημοσίευση. Παραλαμβάνει το one-time witness `MARKETPLACE_NFT` (Ενότητα 2.3.4) και με αυτό διεκδικεί το αντικείμενο `Publisher`.

Έπειτα ορίζει το Display του `MarketplaceNFT`: πέντε πρότυπα που αντιστοιχίζουν τα τυπικά ονόματα προβολής στα ομώνυμα πεδία της δομής, με τις δύο λίστες να παράγονται από τη μακροεντολή `map!` πάνω σε κυριολεκτικά διανύσματα. Τέλος, μεταβιβάζει το `Publisher` και το `Display` στον εκδότη, ώστε να μπορεί να τροποποιήσει τα πρότυπα στο μέλλον. Κανένα από τα δύο αντικείμενα δεν παρέχει δικαιώματα επί των προσφορών· ο εκδότης του πακέτου δεν έχει κανένα προνόμιο στο πρωτόκολλο (ΜΛΑ4). Το `barter` δεν έχει `init`: δεν χρειάζεται καμία αρχική κατάσταση.

4.4 Συναρτήσεις

Ο Πίνακας 4.2 συνοψίζει τις δημόσιες συναρτήσεις. Είναι μόνο πέντε, όλες `public`, άρα καλούνται τόσο απευθείας από PTB όσο και από άλλα πακέτα. Δεν υπάρχουν συναρτήσεις ανάγνωσης πεδίων: οι εφαρμογές διαβάζουν τα αντικείμενα απευθείας από τον κόμβο, και ένα ελάχιστο δημόσιο ABI είναι ευκολότερο να διατηρηθεί συμβατό σε μελλοντικές αναβαθμίσεις.

Πίνακας 4.2: Δημόσιες συναρτήσεις του πακέτου

Συνάρτηση	Ορίσματα	Αποτέλεσμα
module `marketplace_nft`		
`mint`	όνομα, περιγραφή, εικόνα, συλλογή, σπανιότητα	*Επιστρέφει* το νέο `MarketplaceNFT`· συμβάν `NftMinted`.
module `barter`		
`place_bid<O,R>`	`vector<O>`, `Coin<SUI>`, πλήθος ζητουμένων, `Option<ID>`	Νέο κοινόχρηστο `Bid<O,R>` με τα στοιχεία δεσμευμένα· `BidPlaced`.
`accept_bid<O,R>`	`Bid<O,R>`, `vector<R>`	Ανταλλαγή και διαγραφή του `Bid`· `BidAccepted`.
`cancel_bid<O,R>`	`Bid<O,R>`	Επιστροφή στον προσφέροντα και διαγραφή· `BidCancelled`.
`reject_bid<O,R>`	`Bid<O,R>`, `&R`	Επιστροφή στον προσφέροντα και διαγραφή· `BidRejected`.

4.4.1 Δημιουργία NFT

```
50 /// Mints an NFT and returns it; the transaction decides where it goes, so
       a
51 /// PTB can mint and bid in one go.
52 public fun mint(
```

```
    name: String,
    description: String,
    image_url: String,
    collection: String,
    rarity: String,
    ctx: &mut TxContext,
): MarketplaceNFT {
    let nft = MarketplaceNFT {
        id: object::new(ctx),
        name,
        description,
        image_url,
        collection,
        rarity,
    };

    event::emit(NftMinted { nft_id: nft.id.to_inner() });

    nft
}
```

Απόσπασμα κώδικα 4.5: Η συνάρτηση `mint`

Η `mint` είναι ανοικτή σε όλους και δεν επικυρώνει τις τιμές συλλογής και σπανιότητας. Η επιλογή είναι συνειδητή για μια εφαρμογή επίδειξης, στην οποία ο τύπος NFT υπάρχει για να τροφοδοτεί το πρωτόκολλο προσφορών με υλικό. Σε πραγματική συλλογή η δημιουργία θα περιοριζόταν με κάποιο αντικείμενο-δικαίωμα (capability).

Η συνάρτηση *επιστρέφει* το αντικείμενο αντί να το μεταβιβάσει στον καλούντα. Η διαφορά είναι ουσιώδης για τη σύνθεση: μέσα σε ένα PTB, το αποτέλεσμα της `mint` μπορεί να τροφοδοτήσει απευθείας την `place_bid`, ώστε δημιουργία και προσφορά να γίνουν σε μία συναλλαγή, ή να παραδοθεί στον αποστολέα με την εντολή `TransferObjects`. Ο έλεγχος 3 του Πίνακα 4.7 ασκεί ακριβώς αυτή τη ροή. Το PTB είναι υποχρεωμένο να καταναλώσει την επιστρεφόμενη τιμή, αφού ο τύπος δεν έχει `drop`· μια συναλλαγή που θα την άφηνε αχρησιμοποίητη απορρίπτεται πριν την εκτέλεση.

4.4.2 Υποβολή προσφοράς

```
/// Escrows `offered` and `offered_coin` in a new shared `Bid` owned by
    the sender.
public fun place_bid<Offered: key + store, Requested: key + store>(
```

```
72      offered: vector<Offered>,
73      offered_coin: Coin<SUI>,
74      requested_count: u64,
75      target_nft_id: Option<ID>,
76      ctx: &mut TxContext,
77  ) {
78      assert!(!offered.is_empty() || offered_coin.value() > 0, EEmptyBid);
79      assert!(requested_count > 0, EInvalidRequestedCount);
80
81      let mut bid = Bid<Offered, Requested> {
82          id: object::new(ctx),
83          bidder: ctx.sender(),
84          offered_count: offered.length(),
85          offered_sui: offered_coin.into_balance(),
86          requested_count,
87          target_nft_id,
88      };
89      bid.escrow(offered);
90
91      event::emit(BidPlaced {
92          bid_id: bid.id.to_inner(),
93          offered_type: type_name::with_defining_ids<Offered>().into_string
                (),
94          requested_type: type_name::with_defining_ids<Requested>().
                into_string(),
95          offered_count: bid.offered_count,
96          offered_sui: bid.offered_sui.value(),
97          requested_count,
98          target_nft_id,
99      });
100
101     transfer::share_object(bid);
102 }
```

Απόσπασμα κώδικα 4.6: Η συνάρτηση `place_bid`

Η συνάρτηση εκτελεί τέσσερα βήματα.

1. **Έλεγχοι εγκυρότητας** (γραμμές 78 και 79). Η προσφορά πρέπει να περιέχει τουλάχιστον ένα NFT ή θετικό ποσό (`EEmptyBid`) και να ζητά τουλάχιστον ένα αντικείμενο (`EInvalidRequestedCount`). Η σειρά των δύο ελέγχων είναι μέρος της διεπαφής και επαληθεύεται από δικό της έλεγχο.

2. **Κατασκευή και δέσμευση** (81 έως 89). Το νόμισμα μετατρέπεται σε `Balance` και αποθηκεύεται στο πεδίο `offered_sui`. Έπειτα η ιδιωτική `escrow` (Απόσπασμα 4.8) προσαρτά κάθε στοιχείο του διανύσματος στο `Bid` ως δυναμικό πεδίο αντικειμένου με κλειδί `EscrowKey(i)`.
3. **Συμβάν** (91 έως 99). Εκπέμπεται `BidPlaced` με όλα τα στοιχεία που χρειάζεται μια εφαρμογή για να εντοπίσει την προσφορά, συμπεριλαμβανομένων των ονομάτων των δύο τύπων, που διαφορετικά θα χάνονταν μαζί με το αντικείμενο.
4. **Κοινοποίηση** (101). Το αντικείμενο γίνεται κοινόχρηστο.

Η μακροεντολή `do!` της γραμμής 174 αξίζει ιδιαίτερης μνείας. Ο τύπος του διανύσματος, `vector<Offered>`, δεν έχει την ικανότητα `drop`, αφού δεν την έχει ούτε ο `Offered`. Η `do!` καταναλώνει το διάνυσμα στοιχείο προς στοιχείο και το καταστρέφει μόνο όταν αδειάσει· ένας χειρόγραφος βρόχος που θα άφηνε κάποιο στοιχείο εκτός δεν θα μεταγλωττιζόταν καν. Με άλλα λόγια, το ίδιο το σύστημα τύπων εγγυάται ότι *κάθε* NFT που εισήλθε στη συνάρτηση έχει δεσμευθεί.

Αξίζει επίσης να παρατηρηθεί τι *δεν* ελέγχει η συνάρτηση: ότι το στοχευόμενο NFT υπάρχει, ή ότι είναι πράγματι τύπου `Requested`. Ο έλεγχος είναι περιττός σε αυτό το σημείο. Μια προσφορά με ανύπαρκτο ή λανθασμένο στόχο απλώς δεν μπορεί να γίνει αποδεκτή από κανέναν, και ο προσφέρων μπορεί πάντοτε να την ακυρώσει.

4.4.3 Αποδοχή

```
104 /// Settles the swap: the escrow goes to the sender and `counter_nfts` go
        to the bidder.
105 public fun accept_bid<Offered: key + store, Requested: key + store>(
106     mut self: Bid<Offered, Requested>,
107     counter_nfts: vector<Requested>,
108     ctx: &mut TxContext,
109 ) {
110     self.assert_counter_offer(&counter_nfts);
111
112     let bid_id = self.id.to_inner();
113     let bidder = self.bidder;
114     let acceptor = ctx.sender();
115     self.release_escrow(acceptor);
116     counter_nfts.do!(|nft| transfer::public_transfer(nft, bidder));
117     self.destroy(acceptor, ctx);
118
119     event::emit(BidAccepted { bid_id });
120 }
```

Απόσπασμα κώδικα 4.7: Η συνάρτηση `accept_bid`

Η `accept_bid` παραλαμβάνει το `Bid` *κατά τιμή*, με το όνομα `self` όπως ορίζει η σύμβαση δέκτη της γλώσσας. Αυτό σημαίνει ότι η συνάρτηση αναλαμβάνει την κυριότητά του και οφείλει, πριν επιστρέψει, να το έχει αποδομήσει πλήρως. Το σώμα της είναι μια σύνοψη πέντε φάσεων, καθεμία σε μία γραμμή, και οι λεπτομέρειες ζουν στις ιδιωτικές συναρτήσεις του Αποσπάσματος 4.8.

```
155 // === Private Functions ===
156
157 fun assert_counter_offer<Offered: key + store, Requested: key + store>(
158     self: &Bid<Offered, Requested>,
159     counter_nfts: &vector<Requested>,
160 ) {
161     assert!(counter_nfts.length() == self.requested_count,
            EInvalidCounterCount);
162     if (self.target_nft_id.is_none()) return;
163
164     let target_nft_id = *self.target_nft_id.borrow();
165     assert!(counter_nfts.any!(|nft| object::id(nft) == target_nft_id),
            ETargetNftNotProvided);
166 }
167
168 /// Attaches each offered NFT under `EscrowKey(n)`, in offer order.
169 fun escrow<Offered: key + store, Requested: key + store>(
170     self: &mut Bid<Offered, Requested>,
171     offered: vector<Offered>,
172 ) {
173     let mut index = 0;
174     offered.do!(|nft| {
175         dof::add(&mut self.id, EscrowKey(index), nft);
176         index = index + 1;
177     });
178 }
179
180 /// Detaches every escrowed NFT and transfers it to `recipient`.
181 fun release_escrow<Offered: key + store, Requested: key + store>(
182     self: &mut Bid<Offered, Requested>,
183     recipient: address,
184 ) {
```

```
185     self.offered_count.do!(|index| {
186         let nft = dof::remove<EscrowKey, Offered>(&mut self.id, EscrowKey(
                index));
187         transfer::public_transfer(nft, recipient);
188     });
189 }
190 
191 /// Deletes the emptied bid and pays its SUI, if any, to `recipient`.
192 fun destroy<Offered: key + store, Requested: key + store>(
193     self: Bid<Offered, Requested>,
194     recipient: address,
195     ctx: &mut TxContext,
196 ) {
197     let Bid { id, offered_sui, .. } = self;
198     id.delete();
199 
200     if (offered_sui.value() > 0) {
201         transfer::public_transfer(offered_sui.into_coin(ctx), recipient);
202     } else {
203         offered_sui.destroy_zero();
204     }
205 }
```

Απόσπασμα κώδικα 4.8: Οι ιδιωτικές φάσεις του κύκλου ζωής

Πρώτα η `assert_counter_offer` επαληθεύει ότι ο καλών παραδίδει ακριβώς `requested_count` αντικείμενα (`EInvalidCounterCount`) και, αν η προσφορά είναι στοχευμένη, ότι ένα από αυτά έχει το ζητούμενο ID (`ETargetNftNotProvided`), με τη μακροεντολή `any!`. Ο τύπος των αντικειμένων δεν χρειάζεται έλεγχο: η παράμετρος είναι `vector<Requested>` και το `Bid` είναι `Bid<Offered, Requested>`, άρα η ταύτιση έχει ήδη επιβληθεί από την εικονική μηχανή.

Έπειτα η `release_escrow` αφαιρεί τα δεσμευμένα NFTs ένα προς ένα από τα δυναμικά πεδία και τα μεταβιβάζει στον αποδέκτη, και η `do!` μεταβιβάζει τα αντικείμενα του αποδέκτη στον προσφέροντα. Η `destroy` αποδομεί το `Bid`, διαγράφει το `UID` του και καταβάλλει το υπόλοιπο SUI στον αποδέκτη. Εδώ το σύστημα τύπων επιβάλλει μια ακόμη εγγύηση: το πεδίο `offered_sui` έχει τύπο `Balance<SUI>`, που δεν έχει `drop`. Η αποδόμηση παράγει μια τιμή `Balance` η οποία *πρέπει* να καταλήξει κάπου, είτε ως νόμισμα στον παραλήπτη είτε με ρητή καταστροφή αν είναι μηδενική, αλλιώς ο κώδικας δεν μεταγλωττίζεται. Το συμβάν εκπέμπεται τελευταίο, όταν κάθε μετακίνηση έχει ολοκληρωθεί.

Αξίζει να προσεχθεί ένα σημείο στο οποίο ο μεταγλωττιστής *δεν* βοηθά. Η διαγραφή

ενός `UID` δεν αποτυγχάνει αν το αντικείμενο έχει ακόμη δυναμικά πεδία· τα πεδία αυτά απλώς καθίστανται απρόσιτα. Η ορθότητα της αποδέσμευσης εξαρτάται λοιπόν από το ότι η `release_escrow` αφαιρεί ακριβώς `offered_count` πεδία και ότι τόσα ακριβώς πρόσθεσε η `escrow`. Η ισότητα αυτή ισχύει εκ κατασκευής, αφού το `offered_count` υπολογίζεται από το μήκος του διανύσματος και δεν τροποποιείται ποτέ, και επαληθεύεται από τους ελέγχους με πολλαπλά δεσμευμένα NFTs.

4.4.4 Ακύρωση και απόρριψη

```
122 /// Returns the escrow to the bidder. Only the bidder may cancel.
123 public fun cancel_bid<Offered: key + store, Requested: key + store>(
124     mut self: Bid<Offered, Requested>,
125     ctx: &mut TxContext,
126 ) {
127     assert!(self.bidder == ctx.sender(), ENotBidder);
128 
129     let bid_id = self.id.to_inner();
130     let bidder = self.bidder;
131     self.release_escrow(bidder);
132     self.destroy(bidder, ctx);
133 
134     event::emit(BidCancelled { bid_id });
135 }
136 
137 /// Declines a targeted bid and returns the escrow to the bidder.
138 /// Presenting `target_nft` proves the sender owns it.
139 public fun reject_bid<Offered: key + store, Requested: key + store>(
140     mut self: Bid<Offered, Requested>,
141     target_nft: &Requested,
142     ctx: &mut TxContext,
143 ) {
144     assert!(self.target_nft_id.is_some(), EBidNotTargeted);
145     assert!(self.target_nft_id.contains(&object::id(target_nft)),
        ENotTargetNftOwner);
146 
147     let bid_id = self.id.to_inner();
148     let bidder = self.bidder;
149     self.release_escrow(bidder);
150     self.destroy(bidder, ctx);
```

```
151
152     event::emit(BidRejected { bid_id });
153 }
```

Απόσπασμα κώδικα 4.9: Οι συναρτήσεις `cancel_bid` και `reject_bid`

Οι δύο συναρτήσεις έχουν το ίδιο σώμα με την αποδοχή, χωρίς τη φάση των αντικειμένων του αποδέκτη, και διαφέρουν μόνο στην εξουσιοδότηση. Η ακύρωση επιτρέπεται μόνο στον προσφέροντα, και ο έλεγχος είναι μια απλή σύγκριση της αποθηκευμένης διεύθυνσης με τον αποστολέα της συναλλαγής (`ENotBidder`).

Η απόρριψη εφαρμόζει τον μηχανισμό εξουσιοδότησης που περιγράφηκε στην Ενότητα 3.6. Η συνάρτηση δέχεται μια αμετάβλητη αναφορά `&Requested` και ελέγχει ότι το ID του αντικειμένου ταυτίζεται με τον στόχο της προσφοράς (`ENotTargetNftOwner`), με τη συνάρτηση `contains` του `Option`. Ο αποστολέας δεν συγκρίνεται με καμία διεύθυνση. Η απόδειξη της κυριότητας είναι το ίδιο το γεγονός ότι ο καλών μπόρεσε να δώσει το αντικείμενο ως είσοδο στη συναλλαγή. Το NFT δεν μετακινείται· μετά τη συναλλαγή παραμένει στον κάτοχό του. Μια μη στοχευμένη προσφορά δεν μπορεί να απορριφθεί (`EBidNotTargeted`).

Μια συνέπεια που αξίζει να γίνει ρητή: η προσφορά δεσμεύεται στο *αντικείμενο*, όχι στον κάτοχό του. Αν το στοχευόμενο NFT αλλάξει χέρια όσο η προσφορά εκκρεμεί, ο νέος κάτοχος μπορεί να την αποδεχθεί ή να την απορρίψει, και ο παλιός δεν μπορεί πια να κάνει τίποτε από τα δύο. Ο έλεγχος 18 του Πίνακα 4.6 επιβεβαιώνει τη συμπεριφορά.

4.5 Συμβάντα και κωδικοί σφαλμάτων

Το πακέτο εκπέμπει πέντε συμβάντα, ένα για κάθε μεταβολή κατάστασης (Πίνακας 4.3). Τα ωφέλιμα φορτία τους είναι σκόπιμα ελάχιστα: περιέχουν μόνο όσα δεν μπορούν να ανακτηθούν από αλλού. Ο αποστολέας κάθε συναλλαγής, που είναι ο προσφέρων στο `BidPlaced`, ο αποδέκτης στο `BidAccepted` και ο απορρίπτων στο `BidRejected`, καταγράφεται ήδη στα μεταδεδομένα του συμβάντος από τον κόμβο και δεν επαναλαμβάνεται. Τα ονόματα των τύπων περιλαμβάνονται στο `BidPlaced` επειδή το αντικείμενο `Bid` διαγράφεται στο τέλος του κύκλου ζωής του και δεν θα υπήρχε άλλη ανθεκτική πηγή τους. Η εφαρμογή χρησιμοποιεί τα δύο πρώτα συμβάντα για την ανακάλυψη NFTs και προσφορών· τα υπόλοιπα τρία επιτρέπουν την ανασύσταση του πλήρους ιστορικού από οποιονδήποτε.

Πίνακας 4.3: Συμβάντα του πακέτου

Συμβάν	Module	Πεδία
`NftMinted`	`marketplace_nft`	`nft_id`
`BidPlaced`	`barter`	`bid_id`, `offered_type`, `requested_type`, `offered_count`, `offered_sui`, `requested_count`, `target_nft_id`
`BidAccepted`	`barter`	`bid_id`
`BidCancelled`	`barter`	`bid_id`
`BidRejected`	`barter`	`bid_id`

Οι αποτυχίες σηματοδοτούνται με επτά κωδικούς (Πίνακας 4.4), που δηλώνονται με την επισημείωση `#[error(code = N)]` και συνοδεύονται από μήνυμα κειμένου. Ο κώδικας του σφάλματος είναι ο αριθμός που βλέπει η εφαρμογή και ο εξερευνητής, ενώ το μήνυμα, όπως «A bid must offer at least one NFT or some SUI», ενσωματώνεται στο μεταγλωττισμένο πακέτο και εμφανίζεται από τα εργαλεία που το υποστηρίζουν. Όλοι εγείρονται από το `barter`· το `marketplace_nft` δεν έχει καμία συνθήκη αποτυχίας. Η στήλη «Συνάρτηση» δείχνει πού εγείρεται ο καθένας.

Πίνακας 4.4: Κωδικοί σφαλμάτων του συμβολαίου

Κωδ.	Σταθερά	Σημασία	Συνάρτηση
0	`ENotBidder`	Ο καλών δεν είναι ο προσφέρων	`cancel_bid`
1	`EInvalidCounterCount`	Λάθος πλήθος παραδιδόμενων NFTs	`accept_bid`
2	`ETargetNftNotProvided`	Το στοχευόμενο NFT δεν παραδόθηκε	`accept_bid`
3	`EEmptyBid`	Προσφορά χωρίς NFT και χωρίς SUI	`place_bid`
4	`EInvalidRequestedCount`	Μηδενικό πλήθος ζητουμένων	`place_bid`
5	`EBidNotTargeted`	Απόρριψη μη στοχευμένης προσφοράς	`reject_bid`
6	`ENotTargetNftOwner`	Η αναφορά δεν είναι το στοχευόμενο NFT	`reject_bid`

4.6 Ανάλυση ασφάλειας

Η ενότητα εξετάζει κατά πόσο η υλοποίηση ικανοποιεί τις μη λειτουργικές απαιτήσεις και τις αναλλοίωτες συνθήκες του Κεφαλαίου 3, και καταγράφει τα όρια της ανάλυσης.

4.6.1 Εξουσιοδότηση

Ο Πίνακας 4.5 δείχνει ποιος μπορεί να καλέσει επιτυχώς κάθε συνάρτηση και με ποιον μηχανισμό επιβάλλεται ο περιορισμός (ΜΛΑ3).

Πίνακας 4.5: Εξουσιοδότηση ανά συνάρτηση

Συνάρτηση	Ποιος	Μηχανισμός
`place_bid`	Οποιοσδήποτε, με δικά του στοιχεία	Κυριότητα αντικειμένων: μόνο ο κάτοχος μπορεί να τα δώσει ως όρισμα κατά τιμή.
`accept_bid`	Ο κάτοχος του στόχου· σε ανοικτή προσφορά, όποιος κατέχει αντικείμενα τύπου `R`	Κυριότητα των παραδιδόμενων αντικειμένων και έλεγχος ID στόχου.
`cancel_bid`	Ο προσφέρων	Σύγκριση `self.bidder` με τον αποστολέα.
`reject_bid`	Ο κάτοχος του στόχου	Απαίτηση αναφοράς `&R` στο στοχευόμενο αντικείμενο.

4.6.2 Διατήρηση των δεσμευμένων στοιχείων

Η ΜΛΑ2 και οι αναλλοίωτες Α1 έως Α3 ελέγχονται εξετάζοντας κάθε διαδρομή.

Κατά την *υποβολή*, τα NFTs εξέρχονται από την κυριότητα του προσφέροντος και γίνονται παιδιά του `Bid`, και το SUI γίνεται πεδίο του. Από τη στιγμή που το `Bid` κοινοποιείται, κανείς δεν έχει άμεση πρόσβαση στα περιεχόμενά του: τα δυναμικά πεδία και το `Balance` είναι προσβάσιμα μόνο από κώδικα του module που όρισε τον τύπο, και το `barter` δεν διαθέτει άλλη συνάρτηση που τα αγγίζει πέραν των τριών συναρτήσεων τερματισμού (Α1).

Κατά τον *τερματισμό*, και οι τρεις συναρτήσεις περνούν από τις ίδιες δύο ιδιωτικές φάσεις: η `release_escrow` αφαιρεί όλα τα δυναμικά πεδία και η `destroy` αποδομεί το `Bid`, διαγράφει το `UID` και διαθέτει το `Balance`. Ο παραλήπτης είναι ο αποδέκτης στην αποδοχή και ο προσφέρων στις άλλες δύο. Εφόσον το `Bid` παραλαμβάνεται κατά τιμή και διαγράφεται, δεν υπάρχει μετά τη συναλλαγή αντικείμενο με το οποίο θα μπορούσε να επαναληφθεί η ενέργεια (Α2). Τα στοιχεία που εξέρχονται είναι εκείνα που εισήλθαν, αφού ανακτώνται με τα ίδια κλειδιά και κανένας κώδικας δεν μπορεί να τα αντικαταστήσει στο μεταξύ (Α3).

Σε όλες τις διαδρομές, το σύστημα τύπων της Move αποκλείει δύο ολόκληρες κατηγορίες σφαλμάτων: την παράλειψη διάθεσης ενός NFT που βρίσκεται σε τοπική μεταβλητή ή διάνυσμα, και την παράλειψη διάθεσης του `Balance`.

4.6.3 Ατομικότητα και ταυτόχρονη πρόσβαση

Η ατομικότητα (ΜΛΑ1) παρέχεται από το μοντέλο εκτέλεσης: αν οποιαδήποτε `assert!` αποτύχει, ολόκληρη η συναλλαγή αναιρείται. Δεν υπάρχει κατάσταση στην οποία ο ένας έχει παραλάβει και ο άλλος όχι.

Για την ταυτόχρονη πρόσβαση (ΜΛΑ6), ας υποτεθεί ότι ο προσφέρων υποβάλλει ακύρωση την ίδια στιγμή που ο κάτοχος υποβάλλει αποδοχή. Και οι δύο συναλλαγές δηλώνουν το ίδιο κοινόχρηστο αντικείμενο ως είσοδο, άρα διατάσσονται από τη συναίνεση (Ενότητα 2.2.2). Όποια εκτελεστεί πρώτη διαγράφει το `Bid`. Η δεύτερη αποτυγχάνει επειδή η είσοδός της δεν υπάρχει πλέον, χωρίς καμία παρενέργεια πέραν του gas. Το αποτέλεσμα είναι πάντοτε μία από τις δύο τερματικές καταστάσεις, ποτέ κάποιο μείγμα τους. Η συμπεριφορά αυτή επιβεβαιώθηκε πειραματικά στο τοπικό δίκτυο, με δύο λογαριασμούς να υποβάλλουν ταυτόχρονα αποδοχή της ίδιας προσφοράς (Ενότητα 7.4).

4.6.4 Απουσία επανεισόδου

Οι επιθέσεις επανεισόδου (reentrancy), που έχουν προκαλέσει σημαντικές απώλειες σε συμβόλαια του Ethereum, προϋποθέτουν ότι η μεταβίβαση ενός στοιχείου παραδίδει τον έλεγχο σε κώδικα του παραλήπτη. Στη Move του Sui αυτό δεν συμβαίνει. Η `transfer::public_transfer` απλώς αλλάζει τον ιδιοκτήτη του αντικειμένου· δεν καλεί κανέναν κώδικα. Η γλώσσα δεν διαθέτει δυναμική αποστολή κλήσεων, και το πακέτο δεν καλεί συναρτήσεις τρίτων πακέτων. Η κατηγορία αυτή επιθέσεων δεν έχει εφαρμογή.

4.6.5 Παρατηρήσεις και γνωστά όρια

Η ανάλυση ανέδειξε ορισμένα σημεία που δεν αποτελούν ευπάθειες αλλά πρέπει να είναι γνωστά σε όποιον χρησιμοποιήσει ή επεκτείνει το πρωτόκολλο.

- **Ανοικτές προσφορές.** Μια προσφορά χωρίς στόχο μπορεί να γίνει αποδεκτή από οποιονδήποτε παραδώσει αντικείμενα του ζητούμενου τύπου, όποια και αν είναι αυτά. Αυτή είναι η προβλεπόμενη σημασιολογία, και η εφαρμογή δεν τη χρησιμοποιεί.
- **Πολλαπλά ζητούμενα με στόχο.** Όταν `requested_count` > 1 και υπάρχει στόχος, ελέγχεται μόνο ότι ο στόχος περιλαμβάνεται· τα υπόλοιπα αντικείμενα μπορεί να είναι οποιαδήποτε του τύπου `R`.
- **Η απόρριψη προϋποθέτει αντικείμενο διεύθυνσης.** Ο μηχανισμός της αναφοράς αποδεικνύει κυριότητα μόνο για αντικείμενα που ανήκουν σε διεύθυνση. Αν ο τύπος `R` επέτρεπε κοινόχρηστα ή αμετάβλητα αντικείμενα, οποιοσδήποτε θα μπορούσε να παρουσιάσει αναφορά σε αυτά. Η συνέπεια θα ήταν μόνο ότι τρίτος θα μπορούσε να προκαλέσει επιστροφή των στοιχείων στον προσφέροντα, όχι απώλεια.

- **Απουσία λήξης.** Μια προσφορά παραμένει ενεργή επ' αόριστον, έως ότου κάποιος τη διεκπεραιώσει. Τα στοιχεία του προσφέροντος δεν κινδυνεύουν, αφού μπορεί πάντοτε να ακυρώσει, αλλά ένας κάτοχος μπορεί να αποδεχθεί μια παλιά προσφορά που ο προσφέρων έχει ξεχάσει.
- **Ανεπιθύμητες προσφορές.** Οποιοσδήποτε μπορεί να υποβάλει προσφορές για οποιοδήποτε NFT. Το κόστος το φέρει ο ίδιος, αφού δεσμεύει στοιχεία και καταβάλλει την εγγύηση αποθήκευσης, και ο κάτοχος μπορεί να τις απορρίψει ή να τις αγνοήσει.
- **Εμβέλεια της ανάλυσης.** Η ανάλυση είναι επιχειρηματολογική και στηρίζεται σε ελέγχους μονάδας. Δεν έχει γίνει τυπική επαλήθευση ούτε ανεξάρτητος έλεγχος ασφάλειας, που θα ήταν προαπαιτούμενα για χρήση με στοιχεία πραγματικής αξίας.

4.7 Έλεγχοι

Η σουίτα περιλαμβάνει 21 ελέγχους μονάδας γραμμένους με το `sui::test_scenario`, που προσομοιώνει διαδοχικές συναλλαγές διαφορετικών αποστολέων. Οι 18 ασκούν το `barter` με δύο τύπους που ορίζονται μόνο για τους ελέγχους, `Sword` και `Shield`, οι οποίοι δεν έχουν καμία σχέση μεταξύ τους ούτε με τον `MarketplaceNFT`. Με αυτόν τον τρόπο οι έλεγχοι επαληθεύουν ακριβώς τον ισχυρισμό του τίτλου της εργασίας: ότι η ανταλλαγή λειτουργεί μεταξύ αντικειμένων διαφορετικών τύπων. Οι υπόλοιποι τρεις ασκούν το `marketplace_nft`.

4.7.1 Το σκηνικό των ελέγχων

Οι δύο ψευδο-τύποι, το σκηνικό και όλες οι βοηθητικές συναρτήσεις ζουν σε ξεχωριστό module ελέγχων, το `barter_fixture`, ώστε κάθε έλεγχος να δηλώνει μόνο τη δική του διαφορά. Το σκηνικό είναι μια δομή χωρίς ικανότητες που περιτυλίγει το `Scenario`· η έλλειψη `drop` υποχρεώνει κάθε έλεγχο να τερματίσει ρητά με `end()`, άρα κανένα σενάριο δεν μένει μισοτελειωμένο. Δύο μακροεντολές, `next_tx!` και `ctx!`, είναι ο μόνος τρόπος προώθησης των συναλλαγών και πρόσβασης στο πλαίσιο, και μια σειρά μεθόδων όπως `swords(3)`, `sui(50)`, `take_bid()`, `sword_count()` και `sui_received()` εκφράζουν το κάθε βήμα στη γλώσσα του προβλήματος. Το Απόσπασμα 4.10 δείχνει έναν πλήρη έλεγχο.

```
84 #[test]
85 fun accept_bid_swaps_the_escrow_for_the_counter_nfts() {
86     let mut f = fixture::start(alice());
87     barter::place_bid<Sword, Shield>(f.swords(3), f.sui(0), 1, option::
       none(), f.ctx!());
88
89     f.next_tx!(bob(), |f| {
```

```
90          let bid = f.take_bid();
91          let bid_id = object::id(&bid);
92          let shield = f.new_shield();
93          bid.accept_bid(vector[shield], f.ctx!());
94          assert_eq!(event::events_by_type<BidAccepted>().map!(|e| e.
                bid_accepted_id()), vector[bid_id]);
95      });
96
97      f.next_tx!(bob(), |f| {
98          assert_eq!(f.sword_count(), 3);
99          assert!(!f.has_sui());
100     });
101     f.next_tx!(alice(), |f| assert_eq!(f.shield_count(), 1));
102     assert!(!f.has_bid());
103     f.end();
104 }
```

Απόσπασμα κώδικα 4.10: Ένας έλεγχος αποδοχής με το σκηνικό

4.7.2 Τι καλύπτεται

Ο Πίνακας 4.6 παραθέτει τους ελέγχους του `barter`, ομαδοποιημένους όπως στο αρχείο. Τα ονόματά τους είναι προτάσεις που περιγράφουν τη συμπεριφορά που αποδεικνύουν, όχι τη συνάρτηση που καλούν.

Πίνακας 4.6: Οι έλεγχοι του module `barter`

#	Έλεγχος	Σενάριο	Αναμένεται
Υποβολή			
1	`place_bid_escrows_the_offer_in_a_shared_bid`	2 σπαθιά· τα IDs τους βρίσκονται στα δυναμικά πεδία	επιτυχία
2	`place_bid_emits_the_full_offer`	όλα τα πεδία του `BidPlaced`	επιτυχία
3	`place_bid_rejects_an_empty_offer`	χωρίς NFT και χωρίς SUI	κωδικός 3
4	`place_bid_rejects_a_zero_requested_count`	ζητούνται 0 αντικείμενα	κωδικός 4
5	`place_bid_checks_the_offer_before_the_requested_count`	και τα δύο λάθος· προτεραιότητα	κωδικός 3
Αποδοχή			
6	`accept_bid_swaps_the_escrow_for_the_counter_nfts`	3 σπαθιά για 1 ασπίδα	επιτυχία
7	`accept_bid_pays_the_escrowed_sui_to_the_acceptor`	2 σπαθιά και 50 MIST	επιτυχία
8	`accept_bid_works_for_a_sui_only_offer`	μόνο SUI, στοχευμένη	επιτυχία
9	`accept_bid_delivers_every_requested_nft_to_the_bidder`	ζητούνται 2, στόχος και ένα ακόμη	επιτυχία
10	`accept_bid_rejects_the_wrong_number_of_counter_nfts`	2 αντί για 1	κωδικός 1
11	`accept_bid_rejects_a_counter_offer_without_the_target`	άλλη ασπίδα από τη ζητούμενη	κωδικός 2
12	`accept_bid_checks_the_count_before_the_target`	και τα δύο λάθος· προτεραιότητα	κωδικός 1
Ακύρωση			
13	`cancel_bid_returns_the_escrow_and_sui_to_the_bidder`	2 σπαθιά και 30 MIST πίσω	επιτυχία
14	`cancel_bid_rejects_anyone_but_the_bidder`	ακύρωση από τρίτο	κωδικός 0
Απόρριψη			
15	`reject_bid_returns_the_escrow_to_the_bidder`	3 σπαθιά και 5 MIST πίσω· η ασπίδα μένει	επιτυχία
16	`reject_bid_rejects_an_untargeted_bid`	απόρριψη ανοικτής προσφοράς	κωδικός 5
17	`reject_bid_rejects_a_different_nft_than_the_target`	αναφορά σε άλλη ασπίδα	κωδικός 6
Κύκλος ζωής με αλλαγή κατόχου			
18	`a_bid_follows_the_target_nft_to_its_new_owner`	ο στόχος μεταβιβάζεται· ο νέος κάτοχος αποδέχεται	επιτυχία

Πίνακας 4.7: Οι έλεγχοι του module `marketplace_nft`

#	Έλεγχος	Σενάριο
1	`mint_returns_an_nft_with_the_given_metadata`	και τα πέντε πεδία, συν το `NftMinted`
2	`init_hands_the_publisher_and_display_to_the_deployer`	`Publisher` και `Display` στον εκδότη
3	`two_minted_nfts_can_be_bartered_for_one_in_a_single_ptb_style_flow`	`mint` και `place_bid` στην ίδια συναλλαγή, έπειτα αποδοχή

Δώδεκα έλεγχοι καλύπτουν επιτυχείς διαδρομές και εννέα αποτυχίες. Κάθε αποτυχία δηλώνεται με `#[expected_failure(abort_code = …, location = …)]`, που απαιτεί να συμβεί με τον συγκεκριμένο κωδικό και στο συγκεκριμένο module. Και οι επτά κωδικοί σφαλμάτων έχουν τον δικό τους έλεγχο. Δύο έλεγχοι (5 και 12) παρέχουν ταυτόχρονα δύο άκυρες εισόδους και επαληθεύουν ποιος έλεγχος εγείρεται πρώτος, αφού η σειρά των ελέγχων είναι μέρος της συμπεριφοράς που βλέπουν οι εφαρμογές. Οι έλεγχοι συμβάντων δεν μετρούν απλώς τα συμβάντα· ο έλεγχος 2 διαβάζει και τα επτά πεδία του `BidPlaced`, και οι έλεγχοι 6, 13 και 15 επαληθεύουν ότι το `bid_id` του τερματικού συμβάντος είναι το αναγνωριστικό της προσφοράς που μόλις διαγράφηκε. Η σουίτα εκτελείται με την εντολή `sui move test`, ολοκληρώνεται επιτυχώς, και η μεταγλώττιση με `sui move build --lint` δεν παράγει καμία προειδοποίηση.

4.8 Ανάπτυξη στο δίκτυο

Το πακέτο δημοσιεύεται με την εντολή `sui client publish`. Η δημοσίευση δημιουργεί τέσσερα αντικείμενα: το ίδιο το πακέτο, που είναι αμετάβλητο· το `Publisher` και το `Display` που δημιούργησε η `init`· και το `UpgradeCap`, που επιτρέπει στον εκδότη να δημοσιεύσει νεότερη έκδοση. Η αλυσίδα εργαλείων καταγράφει τη διεύθυνση του πακέτου ανά δίκτυο στο αρχείο `Published.toml`.

4.8.1 Ροή ανάπτυξης ανά δίκτυο

Η ανάπτυξη ακολούθησε τα δίκτυα της Ενότητας 2.2.8 με σειρά αυξανόμενης μονιμότητας, ώστε κάθε στάδιο να κοστίζει όσο το δυνατόν λιγότερο σε χρόνο και σε εξάρτηση από εξωτερικούς παράγοντες.

1. **Έλεγχοι μονάδας, χωρίς δίκτυο.** Κάθε αλλαγή στο συμβόλαιο περνά πρώτα από τη σουίτα της Ενότητας 4.7, που εκτελείται σε δευτερόλεπτα και δεν χρειάζεται κόμβο.
2. **Localnet: το κύριο περιβάλλον εργασίας.** Ολόκληρη η ανάπτυξη του συμβολαίου και της εφαρμογής, τα σενάρια επαλήθευσης μέσω SDK, η αυτοματοποιημένη πλοήγηση της διεπαφής, οι μετρήσεις gas και τα στιγμιότυπα του Κεφαλαίου 6 έγιναν σε

τοπικό δίκτυο. Οι λόγοι είναι πρακτικοί: μηδενική καθυστέρηση, απεριόριστο SUI από το τοπικό faucet, δυνατότητα να ξεκινά κανείς από καθαρή αλυσίδα όποτε θέλει, και ανεξαρτησία από τη διαθεσιμότητα δημόσιων υπηρεσιών. Το τίμημα είναι ότι κάθε επανεκκίνηση αλλάζει τη διεύθυνση του πακέτου, και ότι τα πορτοφόλια επέκτασης δεν βλέπουν το τοπικό δίκτυο, οπότε χρησιμοποιείται το προσωρινό πορτοφόλι της εφαρμογής.

3. **Devnet: πρώτη δημόσια δοκιμή.** Μια πρώιμη έκδοση του πακέτου δημοσιεύθηκε στο devnet, για να επαληθευθεί η διαδικασία δημοσίευσης σε δημόσιο δίκτυο και η εμφάνιση των αντικειμένων στους δημόσιους εξερευνητές. Οι εβδομαδιαίες διαγραφές του δικτύου το καθιστούν ακατάλληλο για οτιδήποτε μονιμότερο, και δεν χρησιμοποιήθηκε περαιτέρω.
4. **Testnet: η τελική έκδοση.** Το τελικό πακέτο δημοσιεύεται στο testnet, και η εφαρμογή διαμορφώνεται με αυτό ως προεπιλεγμένο δίκτυο. Εκεί το σύστημα χρησιμοποιείται όπως θα χρησιμοποιούνταν στην παραγωγή: με πραγματικό πορτοφόλι επέκτασης, με δημόσιο εξερευνητή για κάθε συναλλαγή, και με SUI από το δημόσιο faucet, που έχει όριο ρυθμού και υπενθυμίζει ότι το gas δεν είναι δωρεάν. Το mainnet μένει εκτός εμβέλειας: μια εφαρμογή επίδειξης χωρίς ανεξάρτητο έλεγχο ασφάλειας δεν πρέπει να διαχειρίζεται στοιχεία πραγματικής αξίας (Ενότητα 4.6.5).

Η εφαρμογή δεν χρειάζεται αλλαγή κώδικα για τη μετάβαση από το ένα δίκτυο στο άλλο: η λίστα των δικτύων, ο πελάτης ανά δίκτυο, ο εξερευνητής και το faucet επιλέγονται από το τρέχον δίκτυο του dApp Kit (Κεφάλαιο 5), και η διεύθυνση του πακέτου ανά δίκτυο διαβάζεται από το `Published.toml`.

4.8.2 Το τοπικό δίκτυο στην πράξη

Το τοπικό δίκτυο εκκινείται με την εντολή `sui start --force-regenesis --with-faucet`, που δημιουργεί κάθε φορά μια νέα, άδεια αλυσίδα με δικό της αναγνωριστικό. Αυτό προκάλεσε μια πρακτική δυσκολία: η αλυσίδα εργαλείων «καρφώνει» το αναγνωριστικό αλυσίδας στο `Move.toml` και τη διεύθυνση δημοσίευσης στο `Published.toml`, και μετά από κάθε επανεκκίνηση αρνείται είτε να μεταγλωττίσει, λόγω ασυμφωνίας αναγνωριστικού, είτε να δημοσιεύσει, θεωρώντας το πακέτο ήδη δημοσιευμένο. Η λύση ήταν τέσσερα μικρά σενάρια φλοιού:

`reset-localnet-pins.sh` Διαβάζει το αναγνωριστικό της τρέχουσας αλυσίδας, το γράφει στο `Move.toml` και αφαιρεί την παλιά εγγραφή του τοπικού δικτύου από το `Published.toml`.

`deploy.sh` Εκτελεί το προηγούμενο, δημοσιεύει το πακέτο και καλεί το επόμενο.

`sync-env.sh` Εξάγει τη νέα διεύθυνση του πακέτου από το `Published.toml` και τη γράφει στο αρχείο περιβάλλοντος της εφαρμογής ιστού.

`dev.sh` Εκκινεί το τοπικό δίκτυο, περιμένει να γίνει διαθέσιμο, εκτελεί το `deploy.sh` και εκκινεί την εφαρμογή.

Με αυτά, ολόκληρο το περιβάλλον ανάπτυξης στήνεται με μία εντολή και η εφαρμογή γνωρίζει πάντοτε τη σωστή διεύθυνση του πακέτου χωρίς χειροκίνητη παρέμβαση.

Κεφάλαιο 5

Υλοποίηση της αποκεντρωμένης εφαρμογής

Το κεφάλαιο περιγράφει την εφαρμογή ιστού μέσω της οποίας οι χρήστες αλληλεπιδρούν με το συμβόλαιο. Οι πρώτες ενότητες παρουσιάζουν την τελική υλοποίηση· η Ενότητα 5.6 καταγράφει τις δυσκολίες που προέκυψαν και τις προσεγγίσεις που δοκιμάσθηκαν, στο πνεύμα του αντίστοιχου κεφαλαίου της [9]· και η Ενότητα 5.7 περιγράφει πώς επαληθεύθηκε ότι το σύνολο λειτουργεί.

5.1 Επισκόπηση και τεχνολογίες

Η εφαρμογή είναι μια single-page application που εκτελείται εξ ολοκλήρου στον φυλλομετρητή. Δεν διαθέτει δικό της διακομιστή: τα αρχεία της είναι στατικά, και κάθε δεδομένο που εμφανίζει προέρχεται απευθείας από έναν πλήρη κόμβο του Sui. Αποτελείται από περίπου 4.200 γραμμές TypeScript και CSS σε 57 αρχεία. Ο Πίνακας 5.1 παραθέτει τις τεχνολογίες που χρησιμοποιήθηκαν.

Πίνακας 5.1: Τεχνολογίες της εφαρμογής

Τεχνολογία	Έκδοση	Ρόλος
TypeScript [35]	5.9	Γλώσσα· στατικός έλεγχος τύπων σε όλο τον κώδικα.
React [36]	19	Βιβλιοθήκη διεπαφής χρήστη.
Vite [37]	6	Διακομιστής ανάπτυξης και εργαλείο παραγωγής.
Sui TypeScript SDK [38]	2.31	Κατασκευή συναλλαγών, πελάτες gRPC και JSON-RPC.
dApp Kit React [39]	2.1	Σύνδεση πορτοφολιού, υπογραφή, React hooks.
TanStack Query [40]	5	Ανάκτηση, προσωρινή αποθήκευση και ακύρωση δεδομένων.
Tailwind CSS [41]	4	Μορφοποίηση με βάση design tokens.
Radix Primitives [42]	1.1	Προσβάσιμα παράθυρα διαλόγου, καρτέλες, πλαίσια επιλογής.
React Router	7	Δρομολόγηση και κατάσταση φίλτρων στη διεύθυνση URL.
Geist, Phosphor Icons	5.3, 2.1	Γραμματοσειρές (τοπικά φιλοξενούμενες) και εικονίδια.
sonner	1.7	Ειδοποιήσεις (toasts) με σύνδεσμο στον εξερευνητή.

Ο κώδικας οργανώνεται σε στρώματα που αντιστοιχούν στο Σχήμα 3.5:

```
src/
  dapp-kit.ts          # στιγμιότυπο dApp Kit, δίκτυα
  config/              # διεύθυνσηπακέτου , modules, τύποι, κατάλογος NFT
  lib/                 # πελάτης Sui, ανάλυσηαντικειμένων ,
     εξερευνητέςανάδίκτυο
  hooks/               # ένα hook ανάερώτημακαιανάσυναλλαγή    , faucet,
     υπόλοιπο
  components/
    ui/                # γενικάστοιχεία : Button, Dialog, Tabs, ...
    nft/               # κάρτα NFT, εικόνα, σπανιότητα, φίλτροσπανιότητας
    bid/               # υποβολήπροσφοράς , εισερχόμενες, γραμμήπροσφοράς
    layout/            # γραμμήπλοήγησης , υπόλοιποκαι  faucet
    TxToast.tsx        # ειδοποίησησυναλλαγήςμεσύνδεσμοστονεξερευνητή
    ExplorerLink.tsx   # σύνδεσμοςαντικειμένουήσυναλλαγήςστονεξερευνητή
  pages/               # οιπέντεσελίδες
  types/               # τύποι NFT καιπροσφοράς
```

Απόσπασμα κώδικα 5.1: Δομή του καταλόγου `app/src`

Ο κανόνας που τηρείται είναι ότι οι σελίδες και τα στοιχεία διεπαφής δεν γνωρίζουν τίποτε για το Sui. Καλούν hooks, και οι hooks είναι το μόνο στρώμα που κατασκευάζει συναλλαγές ή διαβάζει την αλυσίδα.

5.2 Σύνδεση με το δίκτυο και το πορτοφόλι

5.2.1 Το στιγμιότυπο dApp Kit

Η σύνδεση με πορτοφόλια και δίκτυα διαμορφώνεται σε ένα σημείο:

```
export const dAppKit = createDAppKit({
  networks: [...NETWORKS],
  defaultNetwork: "localnet",
  enableBurnerWallet: import.meta.env.DEV,
  createClient: createSuiClient,
});
```

Απόσπασμα κώδικα 5.2: Το στιγμιότυπο dApp Kit (`dapp-kit.ts`)

Το dApp Kit ανακαλύπτει αυτόματα κάθε πορτοφόλι που υλοποιεί το Wallet Standard, αποκαθιστά την προηγούμενη σύνδεση όταν ο χρήστης επιστρέφει, και εκθέτει hooks για τον τρέχοντα λογαριασμό και τον τρέχοντα πελάτη. Η επιλογή `enableBurnerWallet` ενεργοποιεί, μόνο σε περιβάλλον ανάπτυξης, ένα προσωρινό πορτοφόλι μέσα στον φυλλομετρητή που υπογράφει χωρίς επιβεβαίωση. Η δυνατότητα αυτή αποδείχθηκε καθοριστική για τον αυτοματοποιημένο έλεγχο της διεπαφής (Ενότητα 5.7).

5.2.2 Πελάτης ανά δίκτυο

Η εφαρμογή δηλώνει τρία δίκτυα, localnet, devnet και testnet (Ενότητα 2.2.8), και ένα από αυτά ως προεπιλογή· η έκδοση ανάπτυξης προεπιλέγει το τοπικό δίκτυο και η τελική έκδοση το testnet (Ενότητα 4.8.1). Το SDK προσφέρει περισσότερους από έναν πελάτες, ανάλογα με το πρωτόκολλο επικοινωνίας με τον κόμβο. Ο συνιστώμενος είναι ο `SuiGrpcClient` [43]· ο παλαιότερος `SuiJsonRpcClient` διατηρείται για συμβατότητα. Και οι δύο υλοποιούν μια κοινή διεπαφή, το Core API, προσβάσιμη ως `client.core`. Η εφαρμογή επιλέγει πελάτη ανά δίκτυο:

```
export const NETWORKS = ["localnet", "devnet", "testnet"] as const;
export type Network = (typeof NETWORKS)[number];
```

```
7
8  export const LOCALNET_URL = "http://127.0.0.1:9000";
9
10 const FULLNODE_URLS: Record<Network, string> = {
11   localnet: LOCALNET_URL,
12   devnet: "https://fullnode.devnet.sui.io:443",
13   testnet: "https://fullnode.testnet.sui.io:443",
14 };
15
16 // Localnet (sui 1.67) lacks gRPC ListEvents, so it gets the JSON-RPC
       client;
17 // every read goes through the transport-agnostic Core API either way.
18 export function createSuiClient(network: Network): ClientWithCoreApi {
19   if (network === "localnet") {
20     return new SuiJsonRpcClient({ network, url: FULLNODE_URLS[network] });
21   }
22   return new SuiGrpcClient({ network, baseUrl: FULLNODE_URLS[network] });
23 }
```

Απόσπασμα κώδικα 5.3: Επιλογή πελάτη ανά δίκτυο (`lib/suiClient.ts`)

Ο λόγος για τον οποίο το τοπικό δίκτυο χρησιμοποιεί διαφορετικό πελάτη εξηγείται στην Ενότητα 5.6.1. Το σημαντικό εδώ είναι ότι ο υπόλοιπος κώδικας δεν το γνωρίζει: όλες οι αναγνώσεις γίνονται μέσω του `client.core`, και η αλλαγή πελάτη για ένα δίκτυο είναι αλλαγή μίας γραμμής.

5.3 Ανάγνωση της κατάστασης χωρίς indexer

Η εφαρμογή χρειάζεται τρία είδη πληροφορίας: ποια NFTs κατέχει ο χρήστης, ποια NFTs υπάρχουν συνολικά, και ποιες προσφορές είναι ενεργές. Μόνο το πρώτο απαντάται με ένα ερώτημα σε πλήρη κόμβο.

5.3.1 NFTs του χρήστη

Ο κόμβος ευρετηριάζει τα αντικείμενα ανά ιδιοκτήτη. Η μέθοδος `listOwnedObjects`, με φίλτρο τον τύπο `MarketplaceNFT`, επιστρέφει σελιδοποιημένα τα NFTs μιας διεύθυνσης. Τα NFTs που ο χρήστης έχει δεσμεύσει σε προσφορές δεν περιλαμβάνονται, και ορθώς: ανήκουν στο αντικείμενο `Bid`. Η σελίδα «My NFTs» τα εμφανίζει παρ' όλα αυτά, με την ένδειξη «In a bid», αντλώντας τα από τις δεσμεύσεις των ενεργών προσφορών του χρήστη

(ερώτημα `useMyBids`, Πίνακας 5.2), ώστε ο χρήστης να βλέπει ολόκληρη την περιουσία του και να μπορεί να ακυρώσει μια προσφορά από το ίδιο το NFT.

5.3.2 Ανακάλυψη μέσω συμβάντων

Για τα υπόλοιπα δύο, η εφαρμογή ξεκινά από τα συμβάντα (Ενότητα 2.2.6). Η βοηθητική `listAllEvents` διαβάζει όλα τα συμβάντα ενός τύπου, από το νεότερο προς το παλαιότερο:

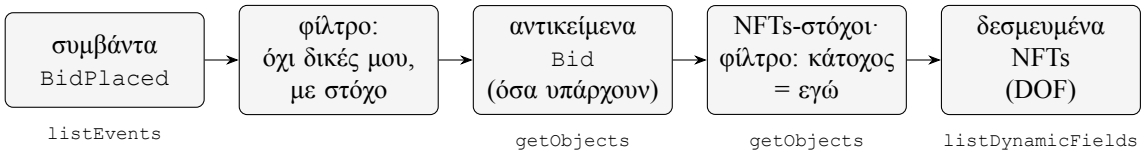
```
14 export async function listAllEvents(
15   client: ClientWithCoreApi,
16   eventType: string,
17 ): Promise<SuiClientTypes.EventEntry[]> {
18   const events: SuiClientTypes.EventEntry[] = [];
19   let before: string | null = null;
20
21   // gRPC pages can come back short or empty with hasNextPage=true; keep
       paging.
22   while (true) {
23     const page = await client.core.listEvents({
24       filter: { eventType },
25       order: "descending",
26       before,
27     });
28     events.push(...page.events);
29     if (!page.hasNextPage || !page.endCursor) break;
30     before = page.endCursor;
31   }
32
33   return events;
34 }
```

Απόσπασμα κώδικα 5.4: Ανάγνωση όλων των συμβάντων ενός τύπου (`lib/bidEnriched.ts`)

Η σελιδοποίηση δεν είναι τυπικότητα. Στη διεπαφή gRPC, ο κόμβος περιορίζει το τμήμα της αλυσίδας που σαρώνει ανά αίτημα, και μια σελίδα μπορεί να επιστραφεί κενή ενώ υπάρχουν παλαιότερα συμβάντα. Ο βρόχος συνεχίζει όσο ο κόμβος δηλώνει ότι υπάρχει επόμενη σελίδα.

Το Σχήμα 5.1 δείχνει πώς από τα συμβάντα προκύπτει η λίστα των εισερχόμενων προσφορών ενός χρήστη, και το Απόσπασμα 5.5 την υλοποίηση.

```
15 export function useReceivedBids(address: string | undefined) {
16   const client = useCurrentClient();
```



Σχήμα 5.1: Ανακάλυψη των εισερχόμενων προσφορών ενός χρήστη από δεδομένα της αλυσίδας.

```
17
18  return useQuery({
19    queryKey: ["receivedBids", address],
20    queryFn: async (): Promise<EnrichedBid[]> => {
21      if (!address) return [];
22
23      const events = await listAllEvents(client, EVENT_BID_PLACED);
24
25      // The bidder is the event sender; untargeted bids are addressed to
            no one.
26      const candidates = events
27        .filter((e) => e.sender !== address)
28        .map((e) => e.json as unknown as BidPlacedEvent | null)
29        .filter((e): e is BidPlacedEvent => !!e && !!parseOptionalId(e.
              target_nft_id));
30
31      if (candidates.length === 0) return [];
32
33      const uniqueBidIds = [...new Set(candidates.map((c) => c.bid_id))];
34
35      const bidObjects = await fetchObjects(client, uniqueBidIds);
36
37      // A missing object means the bid was accepted, rejected or
            cancelled.
38      const bids = bidObjects
39        .map(parseBidObject)
40        .filter((b): b is BidObject => !!b);
41
42      if (bids.length === 0) return [];
43
44      const targetIds = [
45        ...new Set(
46          bids.map((b) => b.targetNftId).filter((id): id is string => !!id
                ),
```

```
        ),
      ];
      const targetLookup = await fetchNfts(client, targetIds);

      const myReceived = bids.filter((b) => {
        if (!b.targetNftId) return false;
        const nft = targetLookup.get(b.targetNftId);
        return !!nft && nft.owner === address;
      });

      if (myReceived.length === 0) return [];

      const offeredByBid = await Promise.all(
        myReceived.map((b) => fetchOfferedNfts(client, b.id)),
      );

      return myReceived.map((bid, i) => ({
        bid,
        targetNft: bid.targetNftId ? targetLookup.get(bid.targetNftId) ??
            null : null,
        offeredNfts: offeredByBid[i],
      }));
    },
    enabled: !!address,
  });
}
```

Απόσπασμα κώδικα 5.5: Το ερώτημα εισερχόμενων προσφορών (`hooks/useReceivedBids.ts`)

Τέσσερα σημεία αξίζουν σχολιασμό.

Ο προσφέρων είναι ο αποστολέας του συμβάντος. Το ωφέλιμο φορτίο του `BidPlaced` δεν περιέχει πεδίο `bidder`· ο κόμβος καταγράφει τον αποστολέα κάθε συμβάντος στα μεταδεδομένα του, και το SDK τον εκθέτει ως `e.sender`. Το φίλτρο «όχι δικές μου» και το αντίστροφό του στο `useMyBids` συγκρίνουν αυτό το πεδίο.

Οι τερματισμένες προσφορές φιλτράρονται από την ύπαρξη του αντικειμένου. Το συμβάν `BidPlaced` παραμένει για πάντα στο ιστορικό. Αν η προσφορά έγινε αποδεκτή, απορρίφθηκε ή ακυρώθηκε, το αντικείμενο `Bid` έχει διαγραφεί, και η `getObjects` επιστρέφει σφάλμα για το συγκεκριμένο ID αντί για αντικείμενο. Η `parseBidObject` μετατρέπει τα σφάλματα σε `null`, και έτσι απομένουν μόνο οι ενεργές προσφορές. Δεν χρειάζεται να διαβασθούν τα συμβάντα τερματισμού.

Ο αποδέκτης προσδιορίζεται από τον τρέχοντα κάτοχο του στόχου. Το συμβάν δεν αναφέρει σε ποιον απευθύνεται η προσφορά, διότι η προσφορά στοχεύει αντικείμενο και όχι πρόσωπο. Η εφαρμογή ανακτά τα στοχευόμενα NFTs και κρατά τις προσφορές των οποίων ο στόχος ανήκει *τώρα* στον χρήστη. Αν ένα NFT αλλάξει κάτοχο, οι προσφορές του ακολουθούν αυτόματα τον νέο.

Τα δεσμευμένα NFTs διαβάζονται ως δυναμικά πεδία. Η `fetchOfferedNfts` απαριθμεί τα δυναμικά πεδία της προσφοράς και, επειδή πρόκειται για πεδία αντικειμένων, λαμβάνει απευθείας τα IDs των παιδιών:

```
121 export async function fetchOfferedNfts(
122   client: ClientWithCoreApi,
123   bidId: string,
124 ): Promise<MarketplaceNFT[]> {
125   const childIds: string[] = [];
126   let cursor: string | null = null;
127   try {
128     do {
129       const page: SuiClientTypes.ListDynamicFieldsResponse =
130         await client.core.listDynamicFields({ parentId: bidId, cursor });
131       for (const entry of page.dynamicFields) {
132         if (entry.$kind === "DynamicObject") childIds.push(entry.childId);
133       }
134       cursor = page.hasNextPage ? page.cursor : null;
135     } while (cursor);
136   } catch {
137     // The bid may have been consumed since it was listed; v2 clients
        throw then.
138     return [];
139   }
140   const nftMap = await fetchNfts(client, childIds);
141   return Array.from(nftMap.values());
142 }
```

Απόσπασμα κώδικα 5.6: Ανάγνωση των δεσμευμένων NFTs μιας προσφοράς (`lib/bidEnriched.ts`)

Με τον ίδιο τρόπο η σελίδα μιας συλλογής προκύπτει από τα συμβάντα `NftMinted`. Εκεί το φίλτρο είναι ο ιδιοκτήτης: εμφανίζονται μόνο τα NFTs που ανήκουν σε διεύθυνση. Όσα είναι δεσμευμένα σε προσφορά ανήκουν σε αντικείμενο και παραλείπονται, ώστε να μην μπορεί κάποιος να υποβάλει προσφορά για ένα NFT που ο κάτοχός του έχει ήδη προσφέρει αλλού. Η σελίδα προσφέρει επιπλέον φίλτρο σπανιότητας, πολλαπλής επιλογής, του οποίου

η κατάσταση ζει στη διεύθυνση URL (`?rarity=common&rarity=rare`): η επιλογή επιβιώνει σε επαναφόρτωση, μοιράζεται ως σύνδεσμος, και ένα ίδιο στοιχείο εξυπηρετεί και τις τρεις συλλογές.

5.3.3 Μεταδεδομένα και προσωρινή αποθήκευση

Για κάθε αντικείμενο ζητούνται δύο όψεις: η απόδοση Display και η αναπαράσταση JSON των πεδίων. Το όνομα, η εικόνα και τα λοιπά στοιχεία λαμβάνονται από το Display όταν είναι διαθέσιμο και, διαφορετικά, από τα πεδία. Η εφεδρική αυτή διαδρομή αποδείχθηκε απαραίτητη (Ενότητα 5.6.2).

Όλα τα ερωτήματα εκτελούνται μέσω του TanStack Query, που τα αποθηκεύει προσωρινά με ένα κλειδί. Ο Πίνακας 5.2 δείχνει τα ερωτήματα και από ποιες συναλλαγές ακυρώνεται το καθένα.

Πίνακας 5.2: Ερωτήματα της εφαρμογής και ακύρωση της προσωρινής μνήμης

Hook	Κλειδί	Ακυρώνεται μετά από
`useOwnedNFTs`	`ownedNFTs`	κάθε συναλλαγή
`useMarketplaceNFTs`	`marketplaceNFTs`	δημιουργία, υποβολή, αποδοχή, ακύρωση
`useReceivedBids`	`receivedBids`	αποδοχή, απόρριψη
`useMyBids`	`myBids`	υποβολή, αποδοχή, ακύρωση
`useSuiBalance`	`suiBalance`	κάθε συναλλαγή, faucet· και περιοδικά ανά 15 s

5.4 Σύνθεση και εκτέλεση συναλλαγών

5.4.1 Κατασκευή των PTBs

Κάθε ενέργεια του χρήστη αντιστοιχεί σε ένα hook που κατασκευάζει ένα PTB. Η υποβολή προσφοράς είναι το πιο σύνθετο:

```
20    const tx = new Transaction();
21
22    const nftArgs = offeredNftIds.map((id) => tx.object(id));
23    const offeredVec = tx.makeMoveVec({ type: NFT_TYPE, elements: nftArgs
          });
24
25    // The contract always takes a Coin<SUI>; a zero split is fine.
26    const [suiCoin] = tx.splitCoins(tx.gas, [offeredSuiMist]);
27
```

```
    tx.moveCall({
      target: `${BARTER_MODULE}::place_bid`,
      typeArguments: [NFT_TYPE, NFT_TYPE],
      arguments: [offeredVec, suiCoin, tx.pure.u64(1), tx.pure.option("id"
          , targetNftId)],
    });
```

Απόσπασμα κώδικα 5.7: Το PTB της υποβολής προσφοράς (`hooks/usePlaceBid.ts`)

Ο κώδικας αντιστοιχεί ένα προς ένα στο Σχήμα 2.2. Η `tx.object(id)` δηλώνει ένα αντικείμενο ως είσοδο με το ID του, και το SDK συμπληρώνει κατά την κατασκευή την έκδοση και το είδος ιδιοκτησίας του. Η `tx.gas` αναφέρεται στο νόμισμα από το οποίο θα πληρωθεί το gas, και το προσφερόμενο ποσό αποκόπτεται από αυτό, ώστε ο χρήστης να μην χρειάζεται να επιλέξει νομίσματα. Τα ορίσματα τύπου δίνονται ρητά στο `typeArguments`. Στην εφαρμογή επίδειξης και οι δύο τύποι είναι ο `MarketplaceNFT`, αφού οι τρεις συλλογές της μοιράζονται έναν τύπο Move· για προσφορά μεταξύ διαφορετικών τύπων αρκεί να αλλάξουν οι δύο αυτές τιμές.

Ο Πίνακας 5.3 συνοψίζει τα PTBs όλων των ροών.

Πίνακας 5.3: Τα PTBs της εφαρμογής

Ροή	Εντολές	Υπογραφές
Δημιουργία n NFTs	$n\times$ `MoveCall mint`, `TransferObjects`	1
Υποβολή προσφοράς	`MakeMoveVec`, `SplitCoins`, `MoveCall place_bid`	1
Αποδοχή	`MakeMoveVec`, `MoveCall accept_bid`	1
Απόρριψη	`MoveCall reject_bid`	1
Ακύρωση	`MoveCall cancel_bid`	1

Κάθε ροή απαιτεί μία υπογραφή (ΜΛΑ8). Η δημιουργία πολλών NFTs γίνεται σε μία συναλλαγή: η `mint` επιστρέφει κάθε αντικείμενο ως αποτέλεσμα εντολής, και μία τελική εντολή `TransferObjects` παραδίδει ολόκληρη την παρτίδα στον αποστολέα (Απόσπασμα 5.8). Η ίδια τεχνική θα επέτρεπε δημιουργία και προσφορά στην ίδια συναλλαγή.

```
    // `mint` returns the NFT; the PTB delivers the batch to the sender.
    const minted = Array.from({ length: quantity }, () =>
      tx.moveCall({
        target: `${NFT_MODULE}::mint`,
        arguments: [
          tx.pure.string(item.name),
          tx.pure.string(item.description),
```

```
            tx.pure.string(item.imageUrl),
            tx.pure.string(item.collection),
            tx.pure.string(item.rarity),
          ],
        }),
    );
    tx.transferObjects(minted, account.address);
```

Απόσπασμα κώδικα 5.8: Δημιουργία παρτίδας NFTs σε ένα PTB (`hooks/useMintNFT.ts`)

5.4.2 Εκτέλεση, αναμονή και ανανέωση

Όλα τα hooks συναλλαγών μοιράζονται έναν βοηθητικό μηχανισμό εκτέλεσης:

```
export function useTransactionMutation() {
  const dAppKit = useDAppKit();
  const client = useCurrentClient();
  const queryClient = useQueryClient();

  return useMutation({
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ["suiBalance"] });
    },
    mutationFn: async (transaction: Transaction) => {
      const result = await dAppKit.signAndExecuteTransaction({ transaction
          });

      if (result.$kind === "FailedTransaction") {
        throw new Error(
          result.FailedTransaction.status.error?.message ??
            "Transaction failed",
        );
      }

      await client.core.waitForTransaction({ digest: result.Transaction.
         digest });
      return result.Transaction;
    },
  });
}
```

Απόσπασμα κώδικα 5.9: Κοινός μηχανισμός εκτέλεσης συναλλαγών (`hooks/useTransactionMutation.ts`)

Ο μηχανισμός κάνει τρία πράγματα που είναι εύκολο να παραλειφθούν.

Πρώτον, ελέγχει το *είδος* του αποτελέσματος. Μια συναλλαγή μπορεί να καταγραφεί στην αλυσίδα και να έχει αποτύχει, για παράδειγμα επειδή μια `assert!` του συμβολαίου δεν ικανοποιήθηκε. Το SDK επιστρέφει τότε αποτέλεσμα τύπου `FailedTransaction` αντί να εγείρει εξαίρεση, και ένας κώδικας που δεν το ελέγχει θα ανακοίνωνε επιτυχία. Στην πράξη, οι περισσότερες τέτοιες αποτυχίες εντοπίζονται νωρίτερα, διότι το SDK προσομοιώνει τη συναλλαγή πριν ζητήσει υπογραφή.

Δεύτερον, περιμένει να *ευρετηριασθεί* η συναλλαγή πριν επιστρέψει. Η ολοκλήρωση μιας συναλλαγής δεν συνεπάγεται ότι ο κόμβος απαντά ήδη στα ερωτήματα με τη νέα κατάσταση. Χωρίς την αναμονή, η άμεση ανανέωση των δεδομένων θα μπορούσε να επιστρέψει την παλιά εικόνα.

Τρίτον, επιτρέπει σε κάθε hook να ακυρώσει, μετά την επιτυχία, ακριβώς τα ερωτήματα που επηρεάζονται (Πίνακας 5.2), ώστε η διεπαφή να ενημερώνεται χωρίς επαναφόρτωση. Το υπόλοιπο SUI ανανεώνεται μετά από κάθε συναλλαγή ανεξαρτήτως έκβασης, αφού και μια αποτυχημένη συναλλαγή καταναλώνει gas.

Κάθε επιτυχής ενέργεια επιστρέφει το ψηφιακό αποτύπωμα (digest) της συναλλαγής, και η ειδοποίηση που βλέπει ο χρήστης το μετατρέπει σε σύνδεσμο «View on …» προς έναν εξερευνητή αλυσίδας. Ο εξερευνητής επιλέγεται ανά δίκτυο (`lib/explorer.ts`): Polymedia Explorer για το τοπικό δίκτυο, επειδή είναι ο μόνος που μπορεί να διαβάσει έναν κόμβο στον υπολογιστή του χρήστη, SuiVision για το testnet και Suiscan για το devnet. Ο ίδιος πίνακας τροφοδοτεί τα εικονίδια δίπλα σε κάθε διεύθυνση και NFT.

Τα σφάλματα παρουσιάζονται στον χρήστη με κατανοητή διατύπωση. Μια βοηθητική συνάρτηση αναγνωρίζει τις συνηθέστερες περιπτώσεις (απόρριψη στο πορτοφόλι, ανεπαρκές υπόλοιπο) και αντιστοιχίζει τους επτά κωδικούς σφαλμάτων του συμβολαίου σε μηνύματα, ώστε ένα «abort code: 0» να εμφανίζεται ως «Only the bidder can do that».

5.5 Υλοποίηση της διεπαφής

Οι αρχές σχεδίασης της Ενότητας 3.9 υλοποιήθηκαν σε τρία επίπεδα, και η διεπαφή πέρασε στη συνέχεια από έναν δεύτερο σχεδιαστικό γύρο.

Design tokens. Τα χρώματα, οι γραμματοσειρές, οι σκιές, οι καμπύλες κίνησης και οι ακτίνες καμπυλότητας των μακετών ορίζονται μία φορά, ως μεταβλητές θέματος του Tailwind. Από αυτές παράγονται οι κλάσεις που χρησιμοποιούν όλα τα στοιχεία. Οι τρεις βαθμίδες

σπανιότητας ορίζονται ως κλάσεις με μεταλλική διαβάθμιση και χρησιμοποιούνται από τρία στοιχεία: ετικέτα, κουκκίδα και πλαίσιο.

Γενικά στοιχεία. Ένα μικρό σύνολο στοιχείων, όπως `Button`, `Dialog`, `Tabs` και `Badge`, ενσωματώνει τις επαναλαμβανόμενες αποφάσεις. Το `Button` διαθέτει ιδιότητα `loading`, που εμφανίζει ένδειξη προόδου και απενεργοποιεί το πάτημα· έτσι η «ορατή εκκρεμότητα» υλοποιείται με τον ίδιο τρόπο παντού. Στη σελίδα δημιουργίας, όσο ένα mint εκκρεμεί, οι υπόλοιπες κάρτες παραμένουν ενεργές: ένα δεύτερο πάτημα μπαίνει σε ουρά («Queued») και εκτελείται μόλις ολοκληρωθεί το πρώτο, διότι δύο ταυτόχρονες συναλλαγές του ίδιου πορτοφολιού θα διεκδικούσαν το ίδιο νόμισμα gas. Το `Dialog` έχει τρεις παραλλαγές: κεντρικό παράθυρο, φύλλο που ανασύρεται από το κάτω μέρος της οθόνης στα κινητά, και πλήρη οθόνη για τη σύνθεση προσφοράς.

Στοιχεία πεδίου. Η κάρτα NFT, η γραμμή προσφοράς, τα «χάπια» σύνοψης και τα πλαίσια «δίνεις / παίρνεις» είναι τα στοιχεία που συνθέτουν τις σελίδες. Η γραμμή προσφοράς δέχεται την κατεύθυνση ως παράμετρο και αντιστρέφει τις δύο πλευρές, ώστε η ίδια προσφορά να παρουσιάζεται σωστά και στον προσφέροντα και στον κάτοχο (ΛΑ10).

Η σύνθεση προσφοράς κάνει το βήμα του SUI ρητά προαιρετικό, με κουμπί «Skip», και η γραμμή πλοήγησης δείχνει το υπόλοιπο του πορτοφολιού και ένα κουμπί faucet που ζητά δοκιμαστικά SUI από το faucet του τρέχοντος δικτύου, ώστε ένα νέο προσωρινό πορτοφόλι να μπορεί να χρησιμοποιηθεί χωρίς τη γραμμή εντολών.

Δεύτερος σχεδιαστικός γύρος. Μετά την υλοποίηση των μακετών, η διεπαφή πέρασε από έναν έλεγχο με βάση ένα δημοσιευμένο σύνολο κανόνων σχεδίασης διεπαφών [44]. Ο έλεγχος δεν άλλαξε τη δομή των σελίδων, που είχε ήδη εγκριθεί, αλλά τα υλικά τους: οι γραμματοσειρές Inter και DM Mono, που φορτώνονταν από εξωτερική υπηρεσία, αντικαταστάθηκαν από τις Geist Sans και Geist Mono, φιλοξενούμενες τοπικά· τα εικονίδια ενοποιήθηκαν σε μία οικογένεια (Phosphor) με ένα πάχος γραμμής· οι κάρτες έχασαν το γκρίζο περίγραμμα υπέρ μιας λεπτής γραμμής και σκιάς χρωματισμένης στην απόχρωση της μάρκας, με ομόκεντρες ακτίνες πλαισίου και εικόνας· όλες οι μεταβάσεις απέκτησαν κοινή καμπύλη κίνησης και τα κουμπιά απόκριση στο πάτημα· και αποκαλύφθηκε ότι τα παράθυρα διαλόγου άνοιγαν ακαριαία, επειδή οι κλάσεις κίνησης αναφέρονταν σε ένα πρόσθετο που δεν είχε εγκατασταθεί. Όλες οι κινήσεις σέβονται τη ρύθμιση `prefers-reduced-motion`.

Η διεπαφή προσαρμόζεται σε πλάτη από 360 εικονοστοιχεία. Στα κινητά η πλοήγηση μεταφέρεται σε δεύτερη γραμμή, τα πλέγματα γίνονται δίστηλα, η σελίδα δημιουργίας εμφανίζει μία συλλογή τη φορά, και οι ενέργειες των προσφορών εμφανίζονται μόνο στην ανεπτυγμένη γραμμή, με κουμπιά ύψους κατάλληλου για αφή.

5.6 Δυσκολίες και εναλλακτικές προσεγγίσεις

5.6.1 Το ListEvents στο τοπικό δίκτυο

Η πρώτη εκδοχή της εφαρμογής χρησιμοποιούσε τον `SuiGrpcClient` σε όλα τα δίκτυα και μεταγλωττιζόταν χωρίς σφάλματα. Το πρόβλημα φάνηκε μόνο όταν εκτελέσθηκε ένα σενάριο επαλήθευσης σε πραγματικό τοπικό δίκτυο: η δημιουργία NFTs, η ανάγνωση αντικειμένων και η εκτέλεση συναλλαγών λειτουργούσαν μέσω gRPC, αλλά κάθε ερώτημα συμβάντων αποτύγχανε με το σφάλμα `UNIMPLEMENTED`. Ο κόμβος που εκκινεί η εντολή `sui start`, στην έκδοση 1.67 της αλυσίδας εργαλείων, δεν υλοποιεί τη μέθοδο `ListEvents` της υπηρεσίας gRPC. Εφόσον τρία από τα ερωτήματα της εφαρμογής ξεκινούν από συμβάντα (Πίνακας 5.2), η εφαρμογή θα εμφανιζόταν άδεια, χωρίς κανένα σφάλμα μεταγλώττισης να έχει προειδοποιήσει.

Εξετάσθηκαν τρεις λύσεις:

1. Εκκίνηση του τοπικού δικτύου μαζί με indexer και υπηρεσία GraphQL. Απαιτεί βάση δεδομένων και αναιρεί την απλότητα του περιβάλλοντος ανάπτυξης.
2. Χρήση του JSON-RPC για όλα τα δίκτυα. Λειτουργεί, αλλά δεσμεύει την εφαρμογή σε διεπαφή που έχει χαρακτηρισθεί παρωχημένη.
3. Επιλογή πελάτη ανά δίκτυο, με όλες τις αναγνώσεις μέσω του Core API, το οποίο υλοποιούν και οι δύο πελάτες.

Επιλέχθηκε η τρίτη (Απόσπασμα 5.3). Το τοπικό δίκτυο χρησιμοποιεί JSON-RPC, τα δημόσια δίκτυα gRPC, και όταν ο τοπικός κόμβος αποκτήσει τη μέθοδο που λείπει, η αλλαγή θα είναι μία γραμμή. Το δίδαγμα είναι γενικότερο: ο στατικός έλεγχος τύπων επαληθεύει ότι ο κώδικας χρησιμοποιεί σωστά μια διεπαφή, όχι ότι ο διακομιστής στην άλλη άκρη την υλοποιεί.

5.6.2 Λοιπές δυσκολίες

Display μέσω gRPC. Η διεπαφή gRPC επιστρέφει μεταδεδομένα προβολής μόνο για τη νεότερη μορφή του προτύπου Display. Για τον `MarketplaceNFT`, του οποίου το Display ορίζεται με την κλασική μορφή, η απόκριση ήταν κενή. Η εφεδρική ανάγνωση από τα πεδία του αντικειμένου, που είχε προβλεφθεί εξαρχής, κάλυψε την περίπτωση χωρίς αλλαγή στο συμβόλαιο.

Μορφοποίηση του κουμπιού σύνδεσης. Το κουμπί σύνδεσης και το παράθυρο επιλογής πορτοφολιού του dApp Kit είναι web components με shadow DOM, και δεν μορφοποιούνται με συνήθεις κανόνες CSS. Προσαρμόζονται μόνο μέσω μεταβλητών που κληρονομούν από τη σελίδα. Επειδή το παράθυρο αποδίδεται μέσα στο κουμπί, μια μεταβλητή ακτίνας καμπυ-

λότητας που έκανε το κουμπί στρογγυλό έκανε στρογγυλό και το παράθυρο. Διατηρήθηκε μία κοινή τιμή και για τα δύο.

Εμφωλευμένα κουμπιά. Στην πρώτη υλοποίηση, ολόκληρη η γραμμή μιας προσφοράς ήταν ένα στοιχείο `button` που την ανέπτυσσε, και μέσα της βρίσκονταν τα κουμπιά αποδοχής και απόρριψης. Η εμφώλευση αυτή δεν είναι έγκυρη HTML και, όπως έδειξε ο αυτοματοποιημένος έλεγχος, καθιστούσε διφορούμενο το ποιο στοιχείο δέχεται το πάτημα. Στον επανασχεδιασμό η κεφαλίδα της γραμμής έγινε στοιχείο με ρόλο κουμπιού και υποστήριξη πληκτρολογίου, και τα κουμπιά ενεργειών είναι αδελφά στοιχεία.

Εξερευνητής και τοπικός κόμβος. Ο σύνδεσμος της ειδοποίησης προς τον Polymedia Explorer εμφάνιζε αρχικά «Data could not be extracted» για κάθε συναλλαγή, παρότι ο κόμβος την επέστρεφε κανονικά. Ο εξερευνητής είναι σελίδα HTTPS στο διαδίκτυο που, για το τοπικό δίκτυο, καλεί απευθείας τον κόμβο στη διεύθυνση `127.0.0.1:9000` από τον φυλλομετρητή, και οι πρόσφατες εκδόσεις του Chrome απαιτούν ρητή άδεια «πρόσβασης σε τοπικό δίκτυο» για τέτοιες κλήσεις. Το πρόβλημα δεν έχει λύση από την πλευρά της εφαρμογής· η άδεια δίνεται μία φορά από τον χρήστη και καταγράφεται στην τεκμηρίωση του αποθετηρίου.

5.7 Επαλήθευση

Η ορθή λειτουργία του συστήματος επαληθεύθηκε σε τρία επίπεδα, από το στενότερο προς το ευρύτερο.

Έλεγχοι μονάδας του συμβολαίου. Οι 21 έλεγχοι της Ενότητας 4.7 επαληθεύουν τη λογική των δύο modules σε προσομοιωμένο περιβάλλον.

Σενάρια συμβολαίου μέσω SDK. Ένα σενάριο TypeScript δημιουργεί τρεις λογαριασμούς σε πραγματικό τοπικό δίκτυο, τους χρηματοδοτεί από το faucet και εκτελεί 20 σενάρια απευθείας στο συμβόλαιο, μετά από κάθε βήμα ελέγχοντας την κατάσταση της αλυσίδας (ιδιοκτησία, υπόλοιπα, ύπαρξη του `Bid`). Πέρα από τις κανονικές ροές και τους επτά κωδικούς σφαλμάτων, καλύπτει περιπτώσεις που δεν εκφράζονται εύκολα σε προσομοίωση: δύο προσφορές για το ίδιο NFT και τι συμβαίνει στη δεύτερη όταν γίνει αποδεκτή η πρώτη, δύο ταυτόχρονες αποδοχές της ίδιας προσφοράς από διαφορετικούς λογαριασμούς, προσφορά της οποίας ο στόχος είναι ο ίδιος δεσμευμένος σε άλλη προσφορά, αποδοχή της δικής του προσφοράς από τον προσφέροντα, ανταλλαγή NFT με νόμισμα μέσω των γενικών παραμέτρων, και λανθασμένα ορίσματα τύπου. Τα αποτελέσματα συζητούνται στην Ενότητα 7.4. Αυτό το επίπεδο αποκάλυψε το πρόβλημα της Ενότητας 5.6.1.

Αυτοματοποιημένη πλοήγηση της διεπαφής. Με το Playwright [45] ένας πραγματικός φυλλομετρητής ανοίγει την εφαρμογή, συνδέεται με το προσωρινό πορτοφόλι, και εκτελεί τις ροές πατώντας τα ίδια κουμπιά που θα πατούσε ένας χρήστης. Ένας δεύτερος και ένας τρίτος χρήστης, οδηγούμενοι από το SDK, παίζουν τον ρόλο των αντισυμβαλλομένων. Μετά από κάθε ενέργεια ελέγχεται το μήνυμα που εμφανίζεται και, ανεξάρτητα, η κατάσταση της αλυσίδας. Περιλαμβάνει 17 ροές, από τις σελίδες χωρίς σύνδεση και το faucet μέχρι την αποδοχή, την απόρριψη και την ακύρωση από οποιαδήποτε πλευρά, μια προσφορά που ακυρώθηκε ενώ ο χρήστης την έβλεπε, και τη διάταξη σε πλάτος 390 εικονοστοιχείων. Ένα συγγενές σενάριο παρήγαγε τα στιγμιότυπα του Κεφαλαίου 6.

Η επιλογή του προσωρινού πορτοφολιού αξίζει μια εξήγηση. Τα πραγματικά πορτοφόλια είναι επεκτάσεις φυλλομετρητή που ζητούν χειροκίνητη επιβεβαίωση, και ο αυτοματισμός τους είναι εύθραυστος. Το προσωρινό πορτοφόλι υλοποιεί το ίδιο Wallet Standard, άρα η εφαρμογή ακολουθεί ακριβώς την ίδια διαδρομή κώδικα, και επιτρέπει η πλοήγηση να φτάσει μέχρι την αλυσίδα και πίσω χωρίς ανθρώπινη παρέμβαση. Ενεργοποιείται μόνο στο περιβάλλον ανάπτυξης.

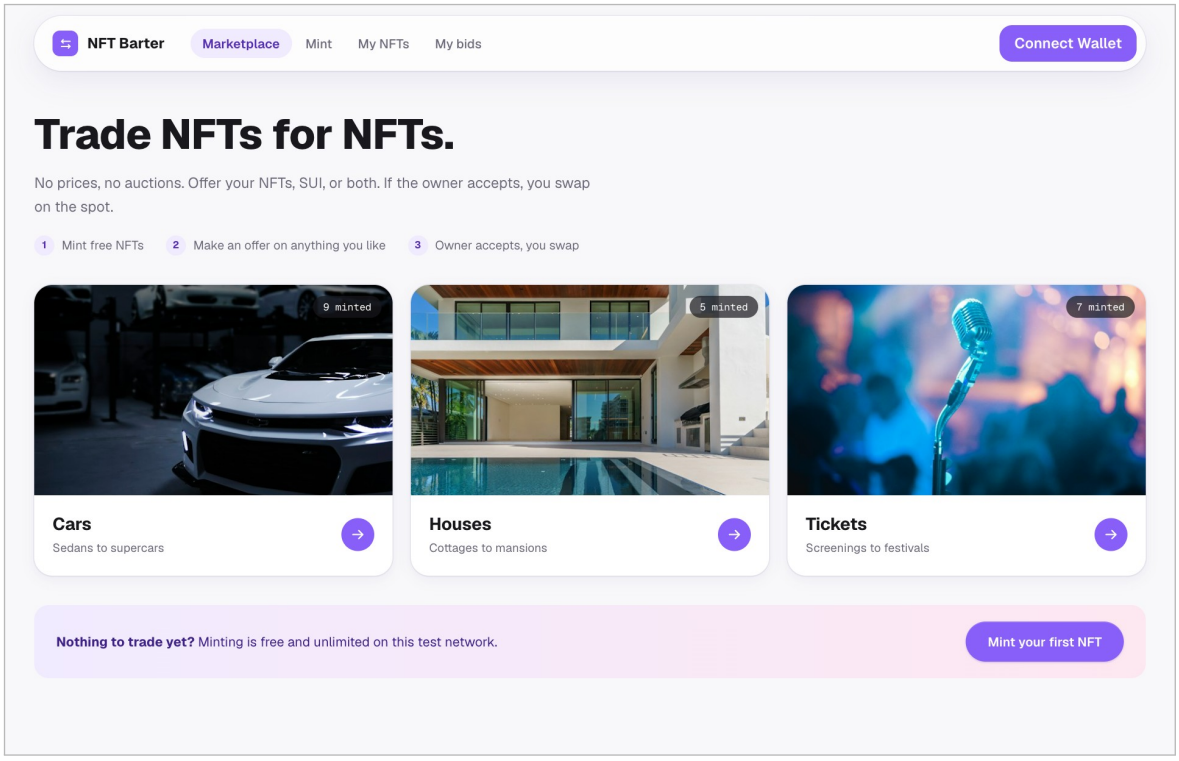
Κεφάλαιο 6

Επίδειξη λειτουργίας

Το κεφάλαιο παρουσιάζει το σύστημα σε λειτουργία, ακολουθώντας έναν χρήστη από τη σύνδεση έως την ολοκλήρωση μιας ανταλλαγής, κατά το πρότυπο του αντίστοιχου κεφαλαίου της [9]. Όλα τα στιγμιότυπα ελήφθησαν από την εφαρμογή συνδεδεμένη σε τοπικό δίκτυο Sui, με πραγματικές συναλλαγές. Στο σενάριο συμμετέχουν τρεις διευθύνσεις: ο χρήστης που παρακολουθούμε, του οποίου η διεύθυνση εμφανίζεται επάνω δεξιά σε κάθε στιγμιότυπο μετά τη σύνδεση, και δύο αντισυμβαλλόμενοι που κατέχουν NFTs και υποβάλλουν προσφορές. Τα στιγμιότυπα παρήχθησαν από το σενάριο αυτοματοποιημένης πλοήγησης της Ενότητας 5.7, ώστε να αναπαράγονται πανομοιότυπα μετά από κάθε αλλαγή της διεπαφής.

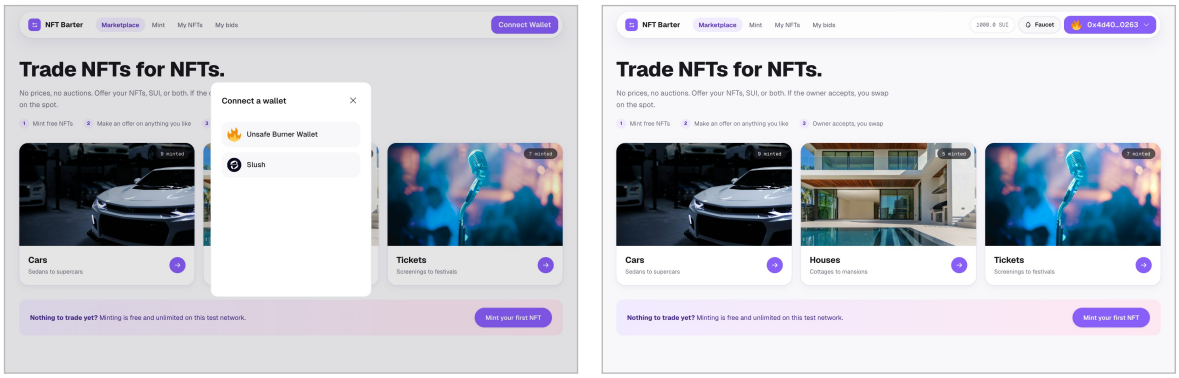
6.1 Αρχική σελίδα και σύνδεση πορτοφολιού

Η αρχική σελίδα (Σχήμα 6.1) εξηγεί σε μία πρόταση τι κάνει η εφαρμογή, συνοψίζει τη διαδικασία σε τρία βήματα και παρουσιάζει τις τρεις συλλογές με το πλήθος των NFTs που έχουν δημιουργηθεί σε καθεμία. Είναι προσβάσιμη χωρίς πορτοφόλι.



Σχήμα 6.1: Η αρχική σελίδα πριν από τη σύνδεση πορτοφολιού.

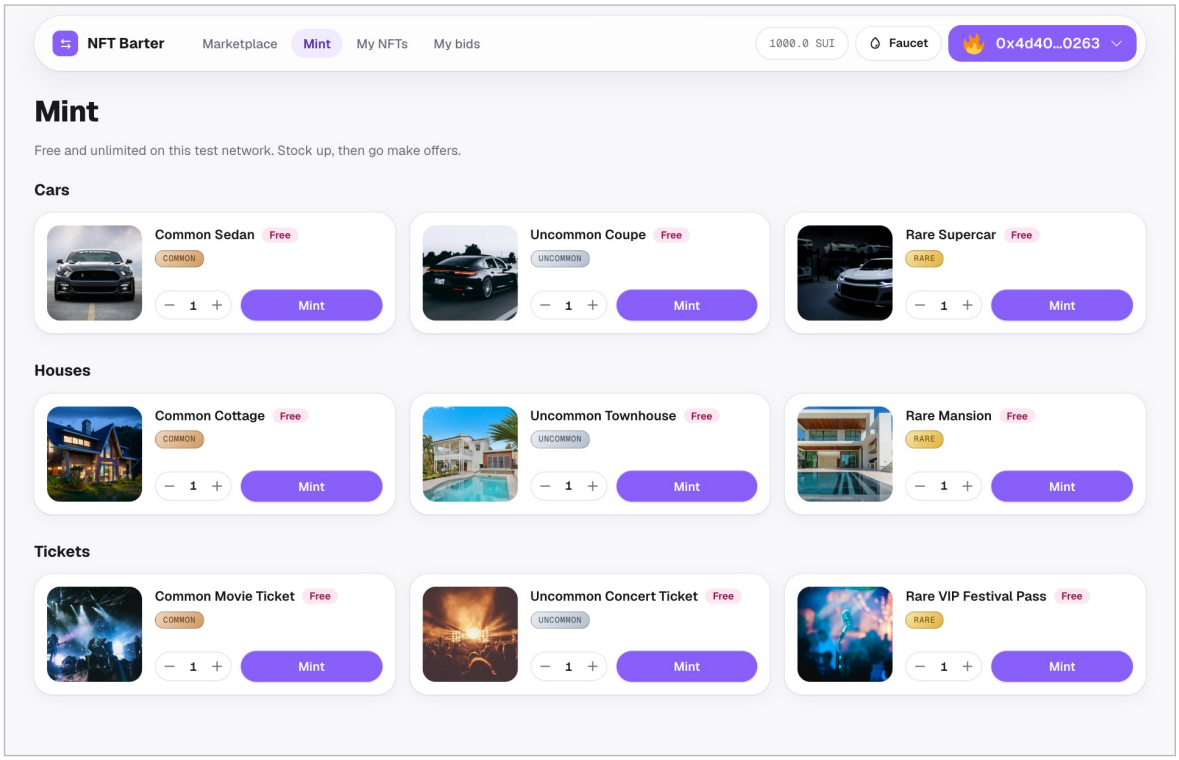
Με το κουμπί «Connect Wallet» ανοίγει το παράθυρο επιλογής πορτοφολιού (Σχήμα 6.2, αριστερά), όπου εμφανίζονται όσα πορτοφόλια υλοποιούν το Wallet Standard. Μετά τη σύνδεση, το κουμπί δείχνει τη διεύθυνση του χρήστη (δεξιά), δίπλα του εμφανίζονται το υπόλοιπο SUI και το κουμπί «Faucet» για δοκιμαστικά νομίσματα, και οι σελίδες «Mint», «My NFTs» και «My bids» ενεργοποιούνται.



Σχήμα 6.2: Αριστερά: επιλογή πορτοφολιού. Δεξιά: η αρχική σελίδα με συνδεδεμένο πορτοφόλι και NFTs ήδη στην αγορά.

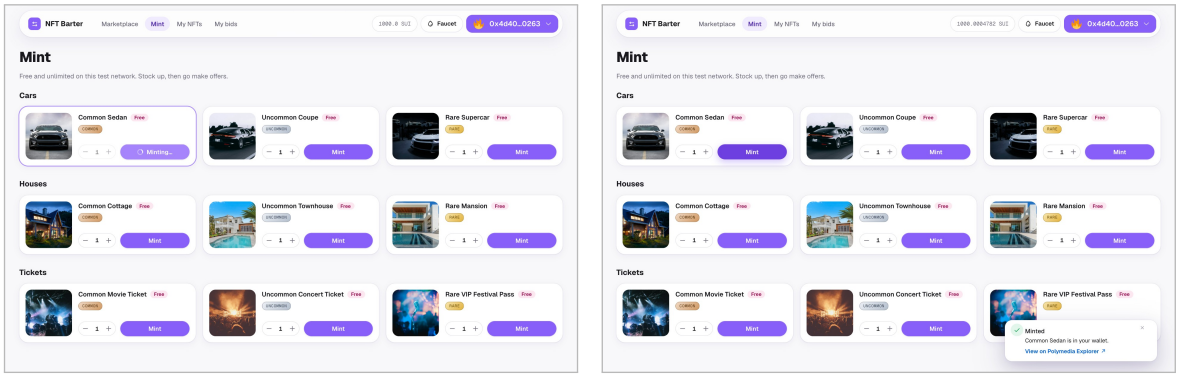
6.2 Δημιουργία NFT

Η σελίδα «Mint» (Σχήμα 6.3) παρουσιάζει τα εννέα είδη του καταλόγου, τρία ανά συλλογή, με τη βαθμίδα σπανιότητάς τους. Ο χρήστης ορίζει ποσότητα και πατά «Mint».



Σχήμα 6.3: Η σελίδα δημιουργίας NFT.

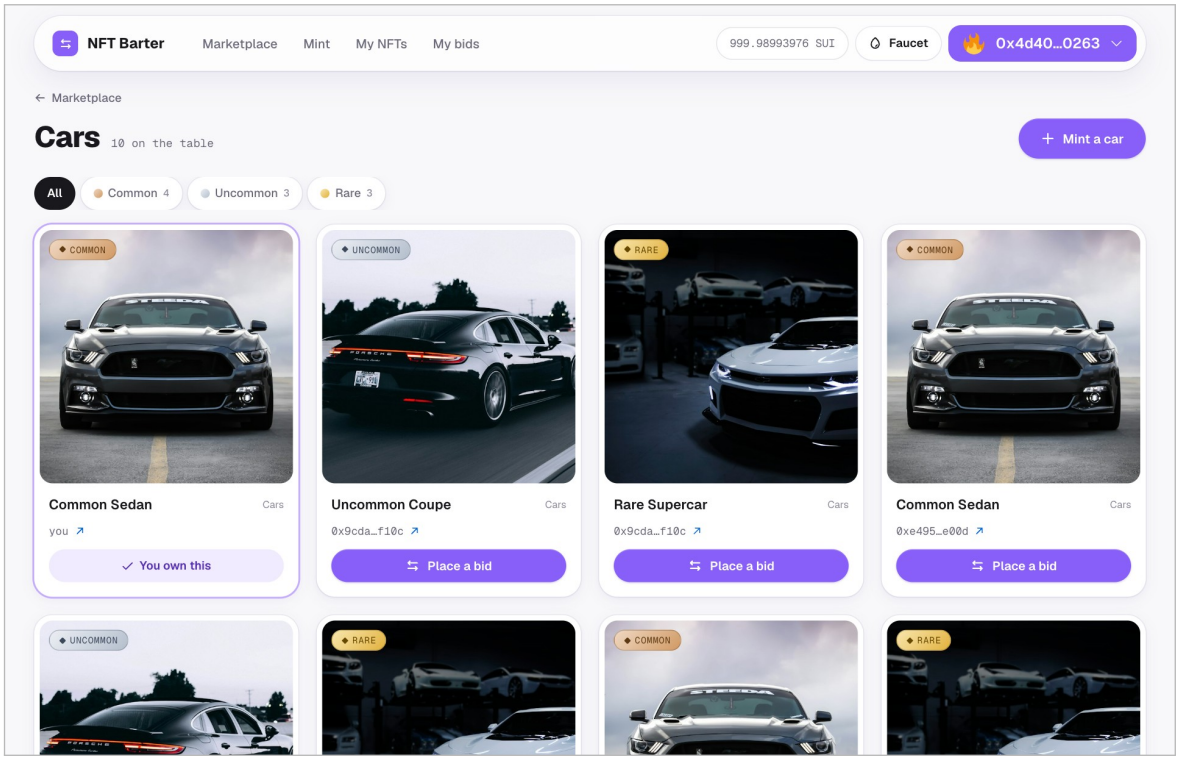
Όσο η συναλλαγή εκκρεμεί (Σχήμα 6.4, αριστερά), η κάρτα που την εκκίνησε επισημαίνεται και το κουμπί της δείχνει «Minting…». Οι υπόλοιπες κάρτες παραμένουν ενεργές: ένα πάτημα σε άλλη κάρτα μπαίνει σε ουρά και εκτελείται μόλις ολοκληρωθεί η τρέχουσα συναλλαγή. Με την ολοκλήρωση εμφανίζεται ειδοποίηση (δεξιά) με σύνδεσμο προς τη συναλλαγή στον εξερευνητή, και το υπόλοιπο στη γραμμή πλοήγησης ενημερώνεται. Στο παράδειγμα δημιουργήθηκε ένα Common Sedan.



Σχήμα 6.4: Αριστερά: εκκρεμής δημιουργία. Δεξιά: ειδοποίηση επιτυχίας.

6.3 Περιήγηση σε συλλογή

Επιλέγοντας μια συλλογή από την αρχική σελίδα, ο χρήστης βλέπει όλα τα NFTs της που βρίσκονται σε πορτοφόλια (Σχήμα 6.5). Κάθε κάρτα δείχνει την εικόνα, τη σπανιότητα ως μεταλλική ετικέτα, το όνομα, και τη διεύθυνση του κατόχου με σύνδεσμο προς explorer. Τα NFTs άλλων χρηστών φέρουν το κουμπί «Place a bid». Το NFT που ανήκει στον ίδιο τον χρήστη διακρίνεται με περίγραμμα και την ένδειξη «You own this». Τα τρία πλήκτρα κάτω από τον τίτλο φιλτράρουν τη συλλογή ανά βαθμίδα σπανιότητας, με το πλήθος κάθε βαθμίδας, και η επιλογή αποτυπώνεται στη διεύθυνση της σελίδας ώστε να μπορεί να μοιρασθεί.



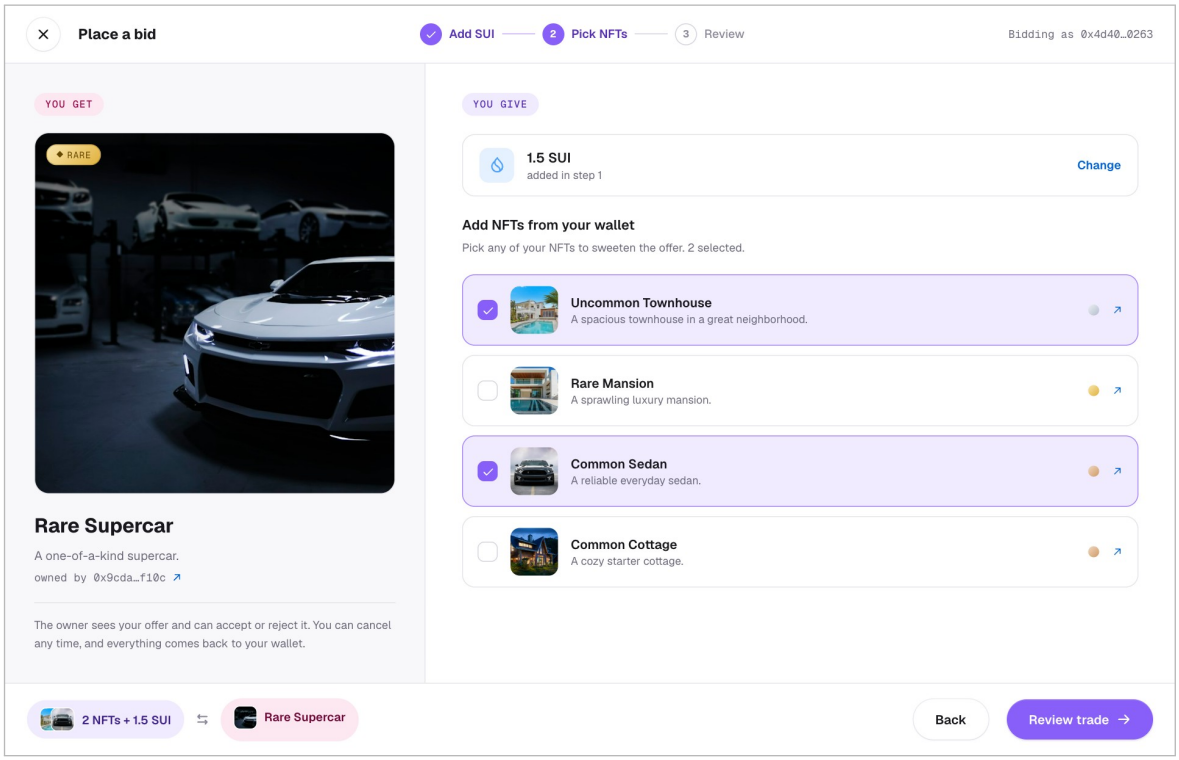
Σχήμα 6.5: Η σελίδα της συλλογής Cars. Πρώτη αριστερά η κάρτα NFT του ίδιου του χρήστη.

6.4 Υποβολή προσφοράς

Το «Place a bid» ανοίγει τη σύνθεση προσφοράς σε πλήρη οθόνη. Η αριστερή στήλη δείχνει σταθερά τι θα *λάβει* ο χρήστης· η δεξιά, τι *δίνει*. Η γραμμή προόδου στο επάνω μέρος δείχνει τα τρία βήματα.

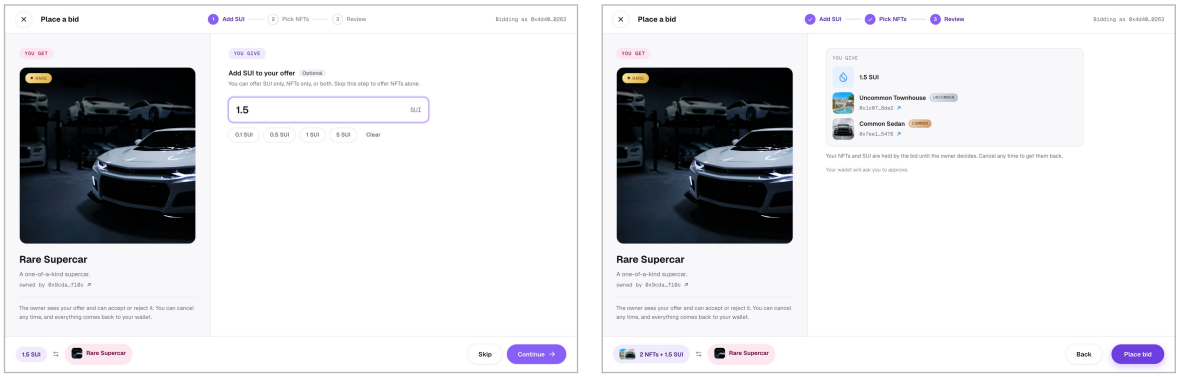
Στο πρώτο βήμα (Σχήμα 6.7, αριστερά) ο χρήστης ορίζει προαιρετικά ένα ποσό SUI, είτε πληκτρολογώντας το είτε με ένα από τα προτεινόμενα ποσά, ή παραλείπει το βήμα με το «Skip». Στο δεύτερο (Σχήμα 6.6) επιλέγει NFTs από το πορτοφόλι του. Στο παράδειγμα προσφέρονται 1,5 SUI, ένα Uncommon Townhouse και ένα Common Sedan για ένα Rare Supercar: μια προσφορά που συνδυάζει νόμισμα με NFTs δύο διαφορετικών συλλογών για

ένα NFT τρίτης βαθμίδας. Η σύνοψη στο κάτω μέρος της οθόνης ενημερώνεται με κάθε επιλογή.



Σχήμα 6.6: Σύνθεση προσφοράς, βήμα 2: επιλογή NFTs. Κάτω αριστερά η σύνοψη «δίνεις ⇄ παίρνεις».

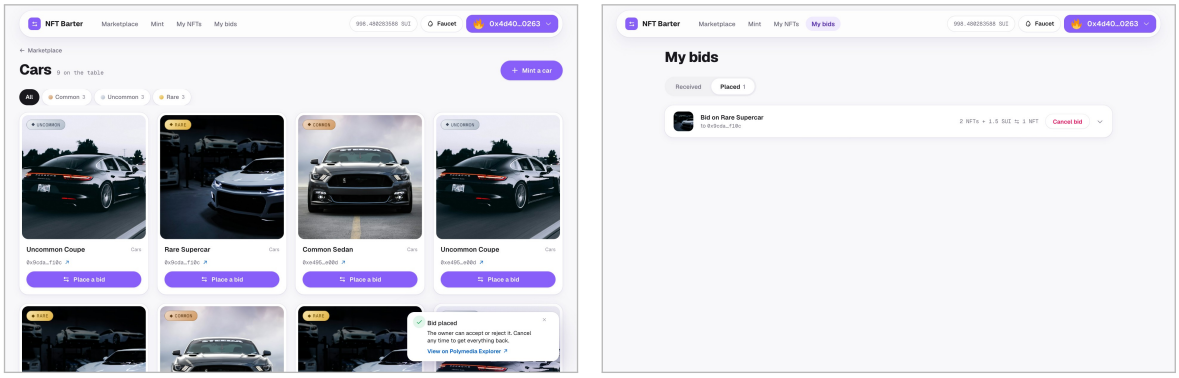
Στο τρίτο βήμα (Σχήμα 6.7, δεξιά) ο χρήστης βλέπει την τελική μορφή της ανταλλαγής και μια υπενθύμιση ότι τα στοιχεία του θα δεσμευθούν έως ότου αποφασίσει ο κάτοχος. Με το «Place bid» το πορτοφόλι ζητά έγκριση.



Σχήμα 6.7: Αριστερά: βήμα 1, ποσό SUI. Δεξιά: βήμα 3, επισκόπηση.

Μετά την επιτυχή υποβολή ο χρήστης επιστρέφει στη συλλογή με σχετική ειδοποίηση, η οποία, όπως κάθε ειδοποίηση επιτυχούς συναλλαγής, περιέχει σύνδεσμο προς τη συναλλαγή στον εξερευνητή του τρέχοντος δικτύου. Η προσφορά εμφανίζεται στην καρτέλα «Placed»

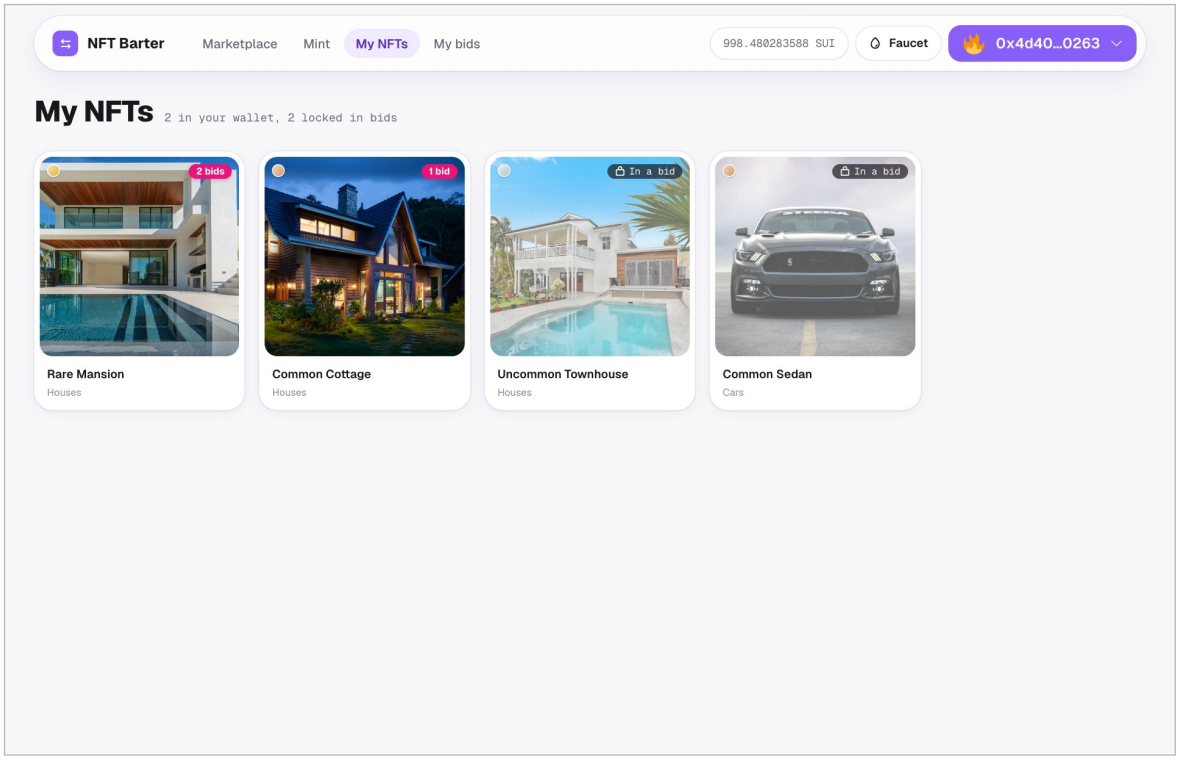
της σελίδας «My bids» (Σχήμα 6.8). Η γραμμή συνοψίζει την ανταλλαγή, δύο NFTs και 1,5 SUI για ένα NFT, και, όταν αναπτυχθεί, την αναλύει στα δύο πλαίσια. Από εδώ ο χρήστης μπορεί να την ακυρώσει.



Σχήμα 6.8: Αριστερά: ειδοποίηση επιτυχούς υποβολής, πάνω από τη συλλογή. Δεξιά: η προσφορά στην καρτέλα «Placed».

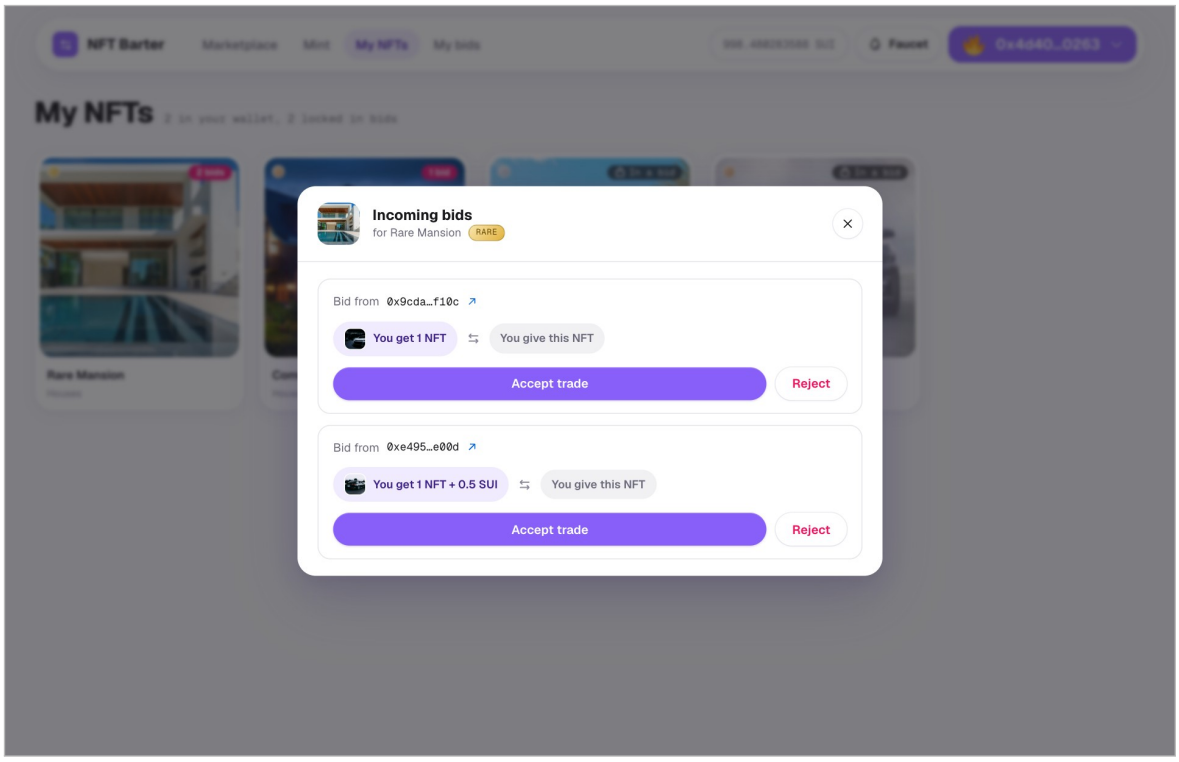
6.5 Εισερχόμενες προσφορές

Η σελίδα «My NFTs» (Σχήμα 6.9) δείχνει τα NFTs του χρήστη. Όσα έχουν λάβει προσφορές φέρουν ένδειξη με το πλήθος τους, ώστε ο χρήστης να βλέπει με μια ματιά πού υπάρχει ενδιαφέρον. Όσα έχει δεσμεύσει ο ίδιος σε προσφορά του εμφανίζονται επίσης, αχνότερα και με την ένδειξη «In a bid»· πατώντας τα βλέπει την προσφορά που τα κρατά και μπορεί να την ακυρώσει από εκεί. Η σπανιότητα αποδίδεται εδώ με τη μορφή κουκκίδας, που διαβάζεται και σε μικρό μέγεθος.



Σχήμα 6.9: Τα NFTs του χρήστη: τα δύο πρώτα με ένδειξη εισερχόμενων προσφορών, τα δύο επόμενα δεσμευμένα στη δική του προσφορά («In a bid»).

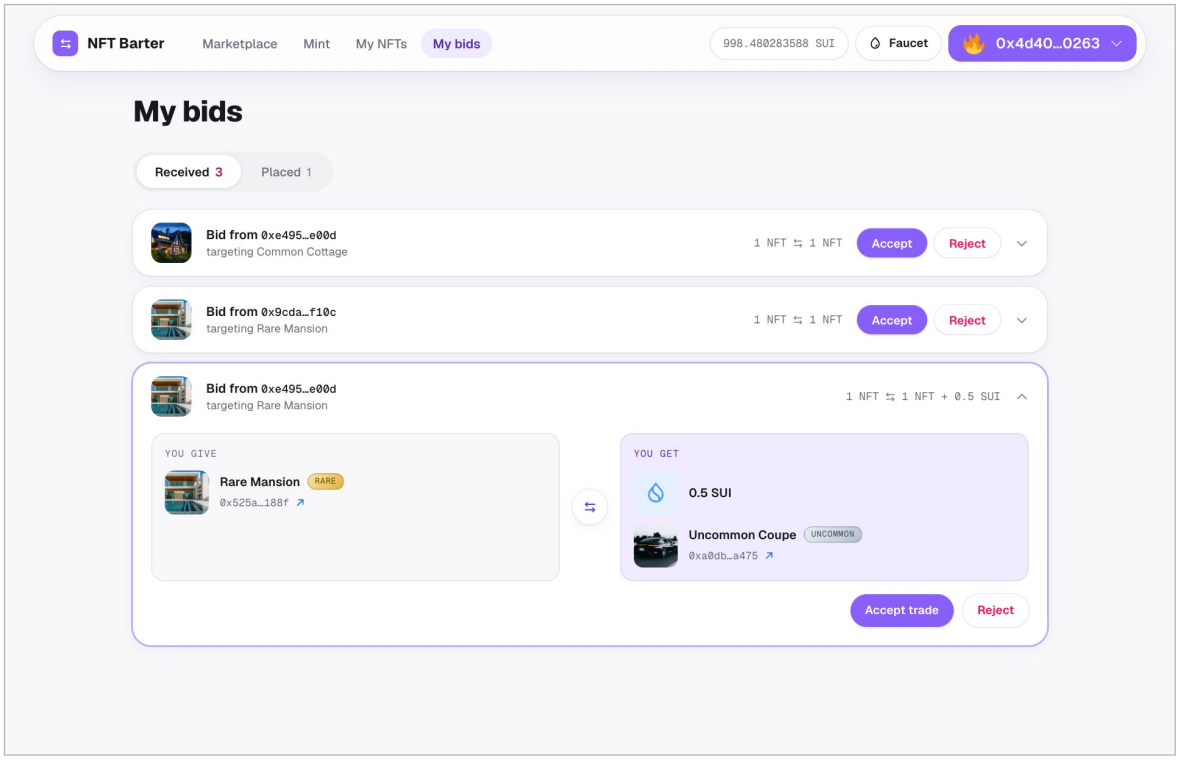
Πατώντας σε ένα NFT ανοίγει το παράθυρο με τις προσφορές του (Σχήμα 6.10). Κάθε προσφορά παρουσιάζεται από την οπτική του κατόχου: τι θα λάβει, με μικρογραφίες των προσφερόμενων NFTs, και ότι θα δώσει το συγκεκριμένο NFT. Στο παράδειγμα το Rare Mansion έχει δύο προσφορές από διαφορετικούς προσφέροντες: μία με ένα NFT μόνο (ένα Rare Supercar) και μία με ένα NFT και 0,5 SUI (ένα Uncommon Coupe).



Σχήμα 6.10: Οι εισερχόμενες προσφορές για το Rare Mansion.

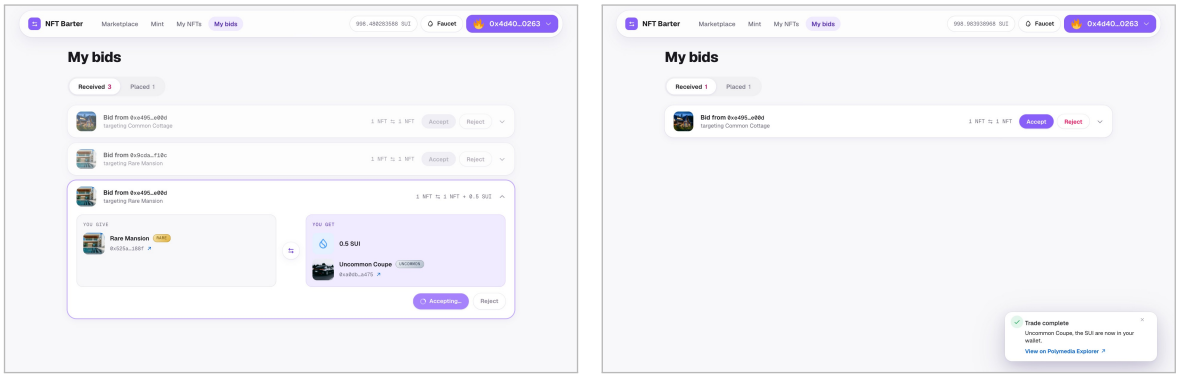
6.6 Αποδοχή, απόρριψη και ακύρωση

Όλες οι εισερχόμενες προσφορές του χρήστη, ανεξαρτήτως NFT, συγκεντρώνονται στην καρτέλα «Received» της σελίδας «My bids», εδώ τρεις: δύο για το Rare Mansion και μία για το Common Cottage. Αναπτύσσοντας μια γραμμή (Σχήμα 6.11) ο χρήστης βλέπει την ανταλλαγή αναλυτικά: αριστερά το NFT που θα δώσει, δεξιά το SUI και τα NFTs που θα λάβει, το καθένα με τη σπανιότητα και το ID του.



Σχήμα 6.11: Η καρτέλα «Received» με τρεις εισερχόμενες προσφορές, η τρίτη ανεπτυγμένη.

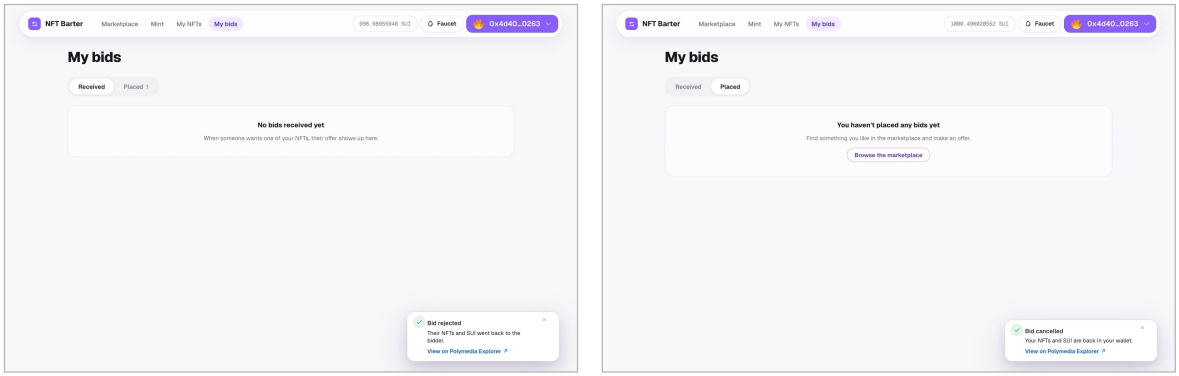
Με το «Accept trade» και την έγκριση στο πορτοφόλι, η ανταλλαγή εκτελείται. Όσο εκκρεμεί (Σχήμα 6.12, αριστερά), το κουμπί δείχνει «Accepting…» και οι υπόλοιπες προσφορές απενεργοποιούνται, ώστε να μην υποβληθούν ανταγωνιστικές ενέργειες. Με την ολοκλήρωση (δεξιά), η ειδοποίηση αναφέρει τι προστέθηκε στο πορτοφόλι, το υπόλοιπο αυξάνεται κατά τα 0,5 SUI της προσφοράς, και η λίστα ανανεώνεται. Αξίζει να παρατηρηθεί ότι από τη λίστα αφαιρέθηκε όχι μόνο η προσφορά που έγινε αποδεκτή αλλά και η δεύτερη προσφορά για το ίδιο NFT: το NFT άλλαξε κάτοχο, και η προσφορά εκείνη απευθύνεται πλέον στον νέο. Απομένει μόνο η προσφορά για το Common Cottage.



Σχήμα 6.12: Αριστερά: αποδοχή σε εξέλιξη. Δεξιά: η ανταλλαγή ολοκληρώθηκε.

Η απόρριψη μιας προσφοράς και η ακύρωση μιας προσφοράς που έχει υποβάλει ο ίδιος ο

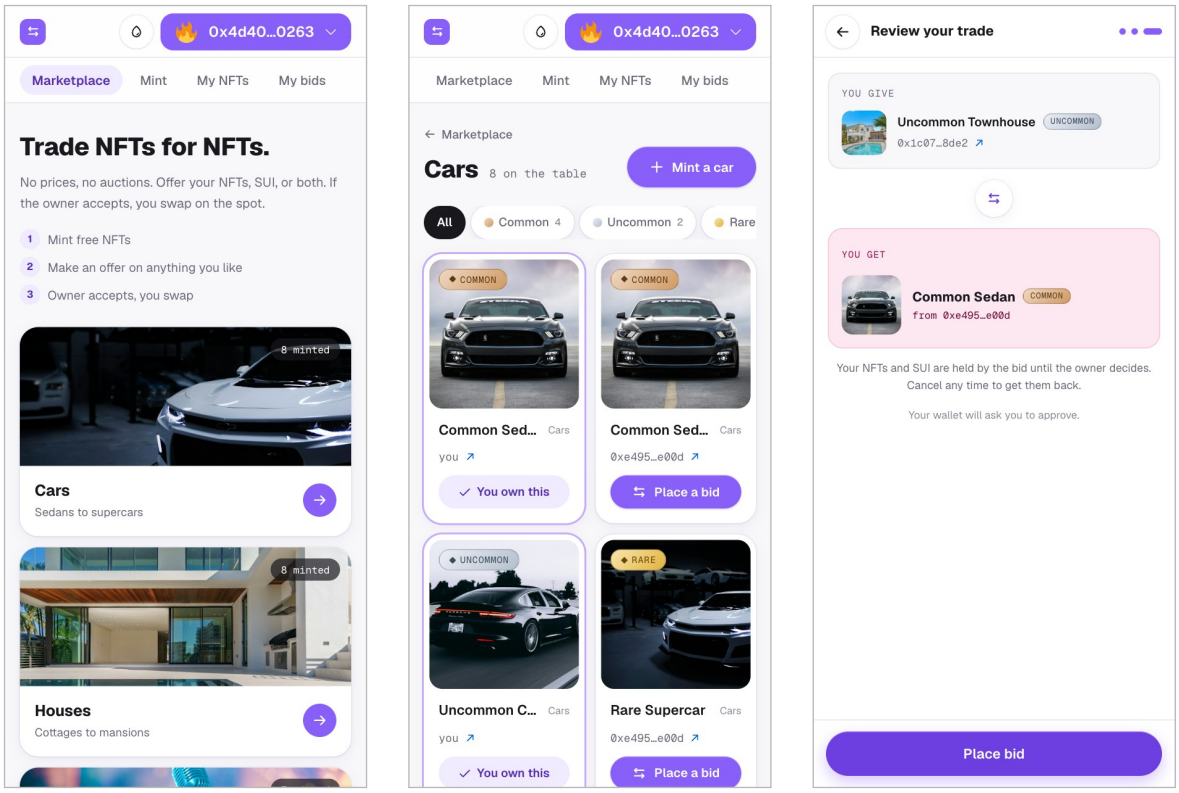
χρήστης ακολουθούν το ίδιο πρότυπο: ένα πάτημα, μία έγκριση, και ειδοποίηση που δηλώνει πού κατέληξαν τα στοιχεία (Σχήμα 6.13). Στο παράδειγμα απορρίπτεται η προσφορά για το Common Cottage, οπότε η καρτέλα «Received» αδειάζει, και έπειτα ακυρώνεται η προσφορά του ίδιου του χρήστη για το Rare Supercar: τα δύο NFTs επιστρέφουν στο πορτοφόλι του και το υπόλοιπο ανεβαίνει κατά το 1,5 SUI που είχε δεσμεύσει.



Σχήμα 6.13: Αριστερά: απόρριψη εισερχόμενης προσφοράς. Δεξιά: ακύρωση υποβληθείσας προσφοράς.

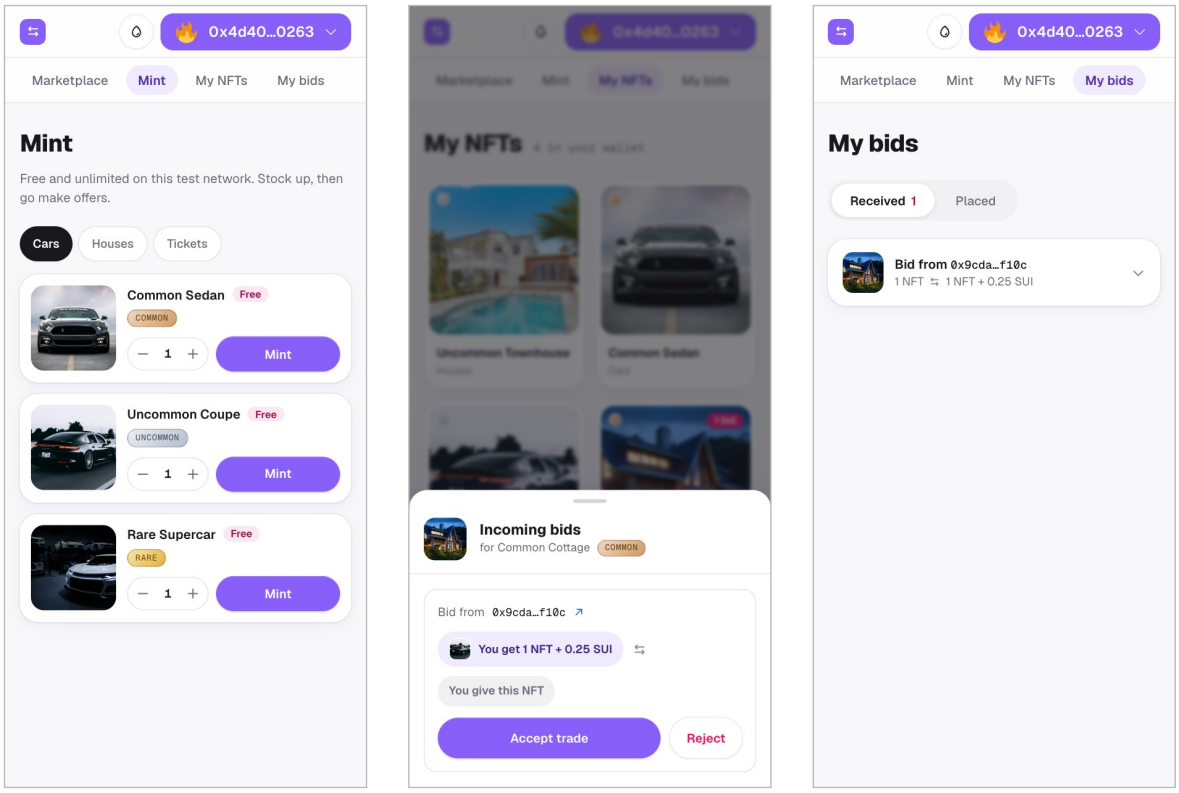
6.7 Η εφαρμογή σε κινητή οθόνη

Η διεπαφή προσαρμόζεται σε πλάτος 360 εικονοστοιχείων χωρίς απώλεια λειτουργιών· τα στιγμιότυπα που ακολουθούν ελήφθησαν σε πλάτος 390. Η πλοήγηση μεταφέρεται σε δεύτερη γραμμή και το κουμπί του faucet γίνεται εικονίδιο. Το Σχήμα 6.14 δείχνει την αρχική σελίδα, μια συλλογή σε δίστηλο πλέγμα με τα φίλτρα σπανιότητας να κυλούν οριζόντια, και την επισκόπηση προσφοράς. Στη στενή οθόνη τα δύο πλαίσια της ανταλλαγής στοιβάζονται κατακόρυφα, και το κουμπί υποβολής καταλαμβάνει όλο το πλάτος.



Σχήμα 6.14: Κινητή οθόνη: αρχική σελίδα, συλλογή, επισκόπηση προσφοράς.

Το Σχήμα 6.15 δείχνει τη σελίδα δημιουργίας, όπου οι συλλογές επιλέγονται με ετικέτες αντί να εμφανίζονται όλες μαζί· τις εισερχόμενες προσφορές, που παρουσιάζονται ως φύλλο που ανασύρεται από το κάτω μέρος της οθόνης· και τη σελίδα προσφορών, όπου κάθε γραμμή συνοψίζει την ανταλλαγή και τα κουμπιά ενεργειών εμφανίζονται μόνο όταν αναπτυχθεί, σε μέγεθος κατάλληλο για αφή.



Σχήμα 6.15: Κινητή οθόνη: δημιουργία, εισερχόμενες προσφορές, σελίδα προσφορών.

Κεφάλαιο 7

Αξιολόγηση

Οι δύο εργασίες αναφοράς [9, 8] αξιολογούν το σύστημά τους κυρίως μέσω επίδειξης. Το παρόν κεφάλαιο προσθέτει τέσσερις ακόμη οπτικές: το κόστος των λειτουργιών σε gas, δοκιμές αντοχής και κλιμάκωσης, μια προβολή του κόστους στο mainnet, και την αντιστοίχιση των απαιτήσεων σε ελέγχους. Κλείνει με τους περιορισμούς της υλοποίησης. Όλες οι μετρήσεις του κεφαλαίου έγιναν σε τοπικό δίκτυο, για τους λόγους που εξηγεί η Ενότητα 7.2.1· τα ποσά gas μεταφέρονται σε κάθε δίκτυο με απλή αλλαγή τιμών, οι χρόνοι όχι.

7.1 Κόστος συναλλαγών

7.1.1 Μεθοδολογία

Το κόστος μετρήθηκε σε τοπικό δίκτυο με ένα σενάριο που εκτελεί κάθε λειτουργία ως χωριστή συναλλαγή και καταγράφει τη σύνοψη gas από τα αποτελέσματά της. Η τιμή αναφοράς του gas ήταν 1.000 MIST ανά μονάδα. Όλα τα ποσά του κεφαλαίου δίνονται σε SUI με έξι δεκαδικά ψηφία· 1 SUI $= 10^9$ MIST, οπότε το έκτο δεκαδικό ψηφίο αντιστοιχεί σε 1.000 MIST. Για να είναι οι μετρήσεις καθαρές ελήφθησαν δύο μέτρα. Το faucet παραδίδει το ποσό σε περισσότερα του ενός νομίσματα, και η πρώτη συναλλαγή ενός λογαριασμού, αν αφεθεί στην προεπιλογή του SDK, τα συγχωνεύει όλα στο νόμισμα που πληρώνει το gas· η διαγραφή των υπόλοιπων νομισμάτων παράγει επιστροφή αποθήκευσης που δεν έχει σχέση με τη λειτουργία που μετράται. Γι' αυτό κάθε λογαριασμός εκτέλεσε πρώτα μια ασήμαντη συναλλαγή εκτός μέτρησης, ώστε η συγχώνευση να γίνει εκεί, και σε κάθε μετρούμενη συναλλαγή το gas πληρώθηκε ρητά από ένα μόνο, προκαθορισμένο νόμισμα. Τα ποσά είναι ντετερμινιστικά: επανάληψη της μέτρησης έδωσε πανομοιότυπες τιμές.

7.1.2 Αποτελέσματα

Ο Πίνακας 7.1 παρουσιάζει τις μετρήσεις. Οι τέσσερις στήλες του επεξηγούνται αμέσως μετά.

Πίνακας 7.1: Κόστος gas ανά λειτουργία, σε SUI

Λειτουργία	Υπολογισμός	Αποθήκευση	Επιστροφή	Καθαρό
`mint` ×1	0,001000	0,003321	0,000978	0,003343
`mint` ×3	0,001000	0,007988	0,000978	0,008009
`mint` ×10	0,001000	0,024320	0,000978	0,024342
`place_bid`: μόνο SUI	0,001000	0,003914	0,000978	0,003936
`place_bid`: 1 NFT	0,001000	0,009082	0,003288	0,006794
`place_bid`: 3 NFTs	0,001000	0,019418	0,007908	0,012510
`place_bid`: 3 NFTs + SUI	0,001000	0,019418	0,007908	0,012510
`accept_bid`: μόνο SUI	0,001000	0,004309	0,006185	−0,000876
`accept_bid`: 1 NFT	0,001000	0,005654	0,011301	−0,004647
`accept_bid`: 3 NFTs	0,001000	0,010321	0,021534	−0,010213
`accept_bid`: 3 NFTs + SUI	0,001000	0,011309	0,021534	−0,009225
`reject_bid`: 1 NFT	0,001000	0,005654	0,011301	−0,004647
`cancel_bid`: 1 NFT	0,001000	0,003321	0,008991	−0,004670

Οι στήλες του πίνακα αντιστοιχούν στα σκέλη του μοντέλου gas της Ενότητας 2.2.5:

Υπολογισμός. Το κόστος εκτέλεσης της συναλλαγής: οι μονάδες υπολογισμού που κατανάλωσαν οι εντολές της, επί την τιμή αναφοράς του gas της εποχής. Στο τοπικό δίκτυο η τιμή αυτή είναι 1.000 MIST ανά μονάδα, και όλες οι λειτουργίες του συμβολαίου πέφτουν στο κατώτατο κλιμάκιο των 1.000 μονάδων: 1.000×1.000 MIST $= 0{,}001$ SUI.

Αποθήκευση. Το ποσό που δεσμεύεται ως εγγύηση για τα bytes που γράφει η συναλλαγή στην αλυσίδα: κάθε αντικείμενο που δημιουργείται, καθώς και κάθε υπάρχον αντικείμενο που τροποποιείται, συμπεριλαμβανομένου του νομίσματος που πληρώνει το gas. Χρεώνεται με σταθερή τιμή ανά byte.

Επιστροφή. Το μέρος της εγγύησης αποθήκευσης που επιστρέφεται στον αποστολέα επειδή η συναλλαγή διαγράφει ή ξαναγράφει αντικείμενα για τα οποία είχε καταβληθεί εγγύηση στο παρελθόν, από οποιονδήποτε. Το 99% της εγγύησης επιστρέφεται· το 1% παρακρατείται. Η επιστροφή πιστώνεται σε όποιον υποβάλλει τη συναλλαγή της διαγραφής, όχι κατ' ανάγκην σε όποιον είχε πληρώσει την εγγύηση.

Καθαρό. Το ποσό που αφαιρέθηκε τελικά από το νόμισμα gas του αποστολέα: υπολογισμός συν αποθήκευση μείον επιστροφή. Αρνητική τιμή σημαίνει ότι ο αποστολέας εισέπραξε περισσότερα από όσα πλήρωσε, επειδή απελευθέρωσε εγγύηση που είχε καταβάλει κάποιος άλλος· αυτό συμβαίνει σε όλες τις λειτουργίες που κλείνουν μια προσφορά.

7.1.3 Ανάλυση

Ο υπολογισμός είναι σταθερός και αμελητέος. Όλες οι λειτουργίες χρεώνονται το ελάχιστο κόστος υπολογισμού, 0,001 SUI, ανεξάρτητα από το πλήθος των NFTs. Η λογική του συμβολαίου είναι απλή, και ακόμη και οι βρόχοι πάνω από τρία ή δέκα αντικείμενα δεν ξεπερνούν το κατώτατο κλιμάκιο χρέωσης.

Το κόστος καθορίζεται από την αποθήκευση. Από τις διαφορές μεταξύ των γραμμών του πίνακα προκύπτει το κόστος αποθήκευσης κάθε είδους αντικειμένου (Πίνακας 7.2). Κάθε γραμμή του Πίνακα 7.1 αναλύεται ακριβώς σε άθροισμα τέτοιων όρων. Για παράδειγμα, η υποβολή προσφοράς με ένα NFT αποθηκεύει το `Bid`, ένα δυναμικό πεδίο, το NFT με νέο ιδιοκτήτη και το νόμισμα του gas: $0{,}002926 + 0{,}002835 + 0{,}002333 + 0{,}000988 = 0{,}009082$ SUI.

Πίνακας 7.2: Κόστος αποθήκευσης ανά αντικείμενο, όπως προκύπτει από τις μετρήσεις

Αντικείμενο	Αποθήκευση (SUI)
`MarketplaceNFT` (με διεύθυνση εικόνας 80 χαρακτήρων)	0,002333
`Bid`	0,002926
Δυναμικό πεδίο αντικειμένου (ανά δεσμευμένο NFT)	0,002835
`Coin<SUI>`	0,000988

Το μέγεθος του `Bid` αξίζει μια σύγκριση. Σε προγενέστερη έκδοση του συμβολαίου η δομή περιείχε επιπλέον τα ονόματα των δύο τύπων ως συμβολοσειρές και είχε την ικανότητα `store`, και η αποθήκευσή της κόστιζε 0,004294 SUI. Η αφαίρεση των δύο πεδίων, που επαναλάμβαναν πληροφορία ήδη διαθέσιμη στον τύπο του αντικειμένου (Ενότητα 4.2), μείωσε το κόστος κατά 32%. Η αποθήκευση στο Sui χρεώνεται ανά byte, και κάθε πλεονάζον πεδίο έχει τίμημα.

Η δέσμευση είναι εγγύηση, όχι δαπάνη. Το κόστος αποθήκευσης επιστρέφεται κατά 99% όταν το αντικείμενο διαγραφεί (Ενότητα 2.2.5). Επειδή κάθε προσφορά καταλήγει σε διαγραφή του `Bid` και των πεδίων του, το μεγαλύτερο μέρος όσων καταβάλλονται κατά την υποβολή ανακτάται κατά τον τερματισμό. Ο Πίνακας 7.3 δείχνει το συνολικό κόστος ενός πλήρους κύκλου ζωής για προσφορά με ένα NFT.

Πίνακας 7.3: Καθαρό κόστος ανά κύκλο ζωής προσφοράς με ένα NFT, σε SUI

Κύκλος ζωής	Προσφέρων	Κάτοχος	Σύνολο
Υποβολή και αποδοχή	0,006794	−0,004647	0,002147
Υποβολή και απόρριψη	0,006794	−0,004647	0,002147
Υποβολή και ακύρωση	0,002124	0	0,002124

Μια πλήρης ανταλλαγή κοστίζει συνολικά περίπου 0,002 SUI: δύο φορές το ελάχιστο κόστος υπολογισμού και το μη επιστρεφόμενο 1% της αποθήκευσης. Το κόστος είναι ανεξάρτητο από την αξία των στοιχείων που ανταλλάσσονται, και το πρωτόκολλο δεν εισπράττει τίποτε (ΜΛΑ4).

Η εγγύηση επιστρέφεται σε όποιον κλείνει την προσφορά. Ο Πίνακας 7.3 αναδεικνύει μια ασυμμετρία που δεν είχε προβλεφθεί στον σχεδιασμό. Την εγγύηση αποθήκευσης την καταβάλλει ο προσφέρων, αλλά η επιστροφή πηγαίνει στον αποστολέα της συναλλαγής που διαγράφει τα αντικείμενα. Στην ακύρωση αυτός είναι ο ίδιος ο προσφέρων. Στην αποδοχή και στην απόρριψη είναι ο κάτοχος, ο οποίος εισπράττει περίπου 0,005 SUI που είχε καταβάλει ο προσφέρων. Το ποσό είναι μικρό και η συνέπεια έχει και θετική όψη: ο κάτοχος έχει ελαφρύ κίνητρο να απορρίπτει ρητά τις προσφορές που δεν τον ενδιαφέρουν, αποδεσμεύοντας έτσι τα στοιχεία του προσφέροντος. Παραμένει ωστόσο μια μεταφορά αξίας που οι χρήστες δεν βλέπουν, και θα έπρεπε είτε να εμφανίζεται στη διεπαφή είτε να αντισταθμίζεται από το συμβόλαιο.

Κλιμάκωση με το πλήθος των NFTs. Κάθε επιπλέον δεσμευμένο NFT προσθέτει στην υποβολή περίπου 0,0029 SUI καθαρά, σχεδόν εξ ολοκλήρου ως εγγύηση για το δυναμικό πεδίο. Η σχέση είναι γραμμική, και δεν παρατηρήθηκε αύξηση του κόστους υπολογισμού έως τα δέκα αντικείμενα ανά συναλλαγή.

7.1.4 Χρόνος απόκρισης

Στο τοπικό δίκτυο, ο χρόνος από την υποβολή μιας συναλλαγής έως την επιστροφή του αποτελέσματος ήταν περίπου 0,20 δευτερόλεπτα, και έως ότου η νέα κατάσταση γίνει ορατή στα ερωτήματα περίπου 0,50 δευτερόλεπτα, με μικρή διακύμανση μεταξύ λειτουργιών. Οι τιμές αυτές αφορούν δίκτυο που εκτελείται σε έναν υπολογιστή, με έναν επικυρωτή, και δεν αντιπροσωπεύουν δημόσιο δίκτυο (Ενότητα 2.2.8). Δείχνουν πάντως ότι η εφαρμογή δεν προσθέτει δική της καθυστέρηση: ο χρόνος που βιώνει ο χρήστης καθορίζεται από το πορτοφόλι και το δίκτυο.

7.2 Δοκιμές αντοχής και κλιμάκωσης

Οι μετρήσεις της προηγούμενης ενότητας αφορούν τη συνήθη χρήση: μία προσφορά, λίγα NFTs, ένας χρήστης τη φορά. Η παρούσα ενότητα πιέζει το σύστημα σε τέσσερις διαστάσεις, το μέγεθος μιας προσφοράς, το μέγεθος μιας παρτίδας δημιουργίας, το πλήθος ταυτόχρονων χρηστών και το μέγεθος του ιστορικού, και καταγράφει πού αλλάζει η συμπεριφορά του και πού βρίσκονται τα όριά του. Όλες οι μετρήσεις έγιναν σε τοπικό δίκτυο· η Ενότητα 7.3 τις προβάλλει στο mainnet.

7.2.1 Μεθοδολογία και επιλογή δικτύου

Οι δοκιμές εκτελέσθηκαν από ένα σενάριο TypeScript πάνω στο SDK, στο ίδιο τοπικό δίκτυο ενός επικυρωτή της Ενότητας 7.1 (sui 1.67.1, πρωτόκολλο 114), με ένα μόνο πακέτο δημοσιευμένο και χωρίς άλλο φορτίο. Ένας πλούσιος λογαριασμός, χρηματοδοτημένος από το τοπικό faucet, μοίρασε νομίσματα σε όσους λογαριασμούς χρειάστηκε κάθε δοκιμή, έως 250 ταυτόχρονα, με ένα PTB ανά 100 παραλήπτες. Κάθε λογαριασμός υπογράφει τις δικές του συναλλαγές με δικό του νόμισμα gas, ώστε οι ταυτόχρονες συναλλαγές να μην συγκρούονται μεταξύ τους παρά μόνο εκεί που η δοκιμή το επιδιώκει. Οι χρόνοι μετρούνται στον πελάτη, από την αποστολή έως την επιστροφή των αποτελεσμάτων.

Η επιλογή του τοπικού δικτύου αντί του testnet ήταν συνειδητή, για τέσσερις λόγους. Πρώτον, η χρηματοδότηση: οι δοκιμές χρειάσθηκαν εκατοντάδες λογαριασμούς και περισσότερες από δύο χιλιάδες συναλλαγές, ενώ το faucet του testnet διαθέτει μικρό ποσό ανά αίτημα με όριο ρυθμού ανά διεύθυνση δικτύου (Ενότητα 2.2.8). Δεύτερον, οι δημόσιοι πλήρεις κόμβοι περιορίζουν τον ρυθμό αιτημάτων, οπότε μια ριπή εκατό ταυτόχρονων συναλλαγών θα μετρούσε τον περιορισμό του κόμβου και όχι το σύστημα. Τρίτον, η επαναληψιμότητα: το τοπικό δίκτυο ξεκινά από καθαρή κατάσταση και δεν μοιράζεται το φορτίο του με κανέναν, ώστε δύο εκτελέσεις να δίνουν συγκρίσιμους χρόνους. Τέταρτον, το κόστος σε gas δεν εξαρτάται από το δίκτυο: οι μονάδες υπολογισμού και τα bytes αποθήκευσης είναι ίδια παντού για την ίδια συναλλαγή, και μόνο οι τιμές ανά μονάδα διαφέρουν. Έτσι οι μετρήσεις κόστους μεταφέρονται με ακρίβεια σε οποιοδήποτε δίκτυο, όπως γίνεται στην Ενότητα 7.3, ενώ οι χρόνοι και ο ρυθμός διεκπεραίωσης αφορούν μόνο το περιβάλλον που μετρήθηκε. Το τίμημα είναι ότι η εικόνα για την καθυστέρηση ενός δημόσιου δικτύου μπορεί να είναι μόνο προσεγγιστική.

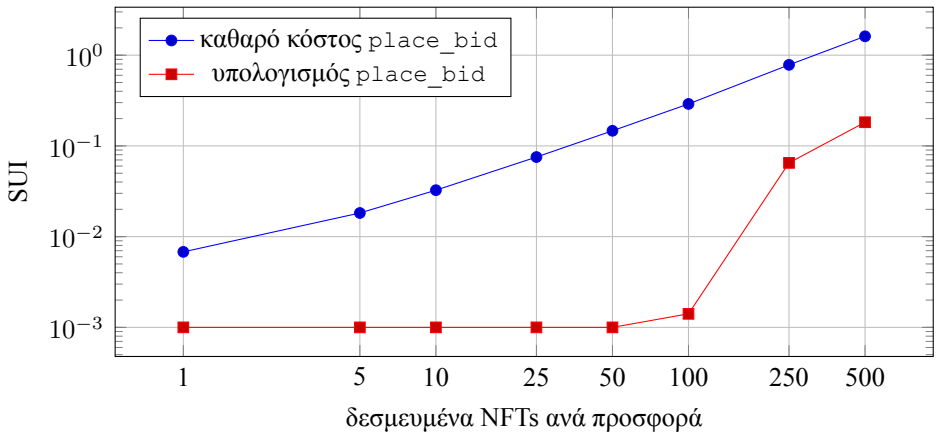
7.2.2 Μέγεθος προσφοράς

Η πρώτη δοκιμή υποβάλλει και αποδέχεται προσφορές με 1 έως 500 δεσμευμένα NFTs (Πίνακας 7.4, Σχήμα 7.1). Και οι 500 δεσμεύσεις πέρασαν· τα όρια του πρωτοκόλλου που

θα σταματούσαν την κλιμάκωση είναι τα 512 ορίσματα ανά εντολή και τα 2.048 αντικείμενα εισόδου ανά συναλλαγή, και δεν προσεγγίσθηκαν διαφορετικά.

Πίνακας 7.4: Κόστος και χρόνος προσφοράς με n δεσμευμένα NFTs, σε SUI (τοπικό δίκτυο)

	`place_bid`			`accept_bid`		
n	Υπολογισμός	Καθαρό	ms	Υπολογισμός	Καθαρό	ms
1	0,001000	0,006794	156	0,001000	−0,004647	161
5	0,001000	0,018226	210	0,001000	−0,015780	213
10	0,001000	0,032516	217	0,001000	−0,029696	209
25	0,001000	0,075386	210	0,001000	−0,071444	209
50	0,001000	0,146838	213	0,001000	−0,141024	212
100	0,001410	0,290151	208	0,001000	−0,280184	214
250	0,065000	0,782460	204	0,012900	−0,685752	202
500	0,182400	1,614393	293	0,055700	−1,338732	276



Σχήμα 7.1: Κόστος υποβολής προσφοράς ως συνάρτηση του πλήθους δεσμευμένων NFTs (λογαριθμικοί άξονες). Το καθαρό κόστος είναι γραμμικό· ο υπολογισμός μένει στο ελάχιστο κλιμάκιο έως τα 50 και εκτοξεύεται πάνω από τα 100.

Τρία πράγματα φαίνονται. Το *καθαρό* κόστος είναι γραμμικό στο n, περίπου 0,0029 SUI ανά NFT, και σχεδόν εξ ολοκλήρου εγγύηση αποθήκευσης που επιστρέφεται κατά την αποδοχή: η αποδοχή προσφοράς 500 NFTs επιστρέφει 1,34 SUI στον αποδέκτη. Ο *υπολογισμός* όμως δεν είναι γραμμικός. Μένει στο ελάχιστο κλιμάκιο των 0,001 SUI έως τα 50 NFTs, ανεβαίνει ελαφρά στα 100, και στα 250 και 500 γίνεται 65 και 182 φορές το ελάχιστο: η χρέωση ανά byte για την ανάγνωση και εγγραφή εκατοντάδων αντικειμένων και την προσάρτηση ισάριθμων δυναμικών πεδίων κυριαρχεί. Η αποδοχή είναι τρεις φορές φθηνότερη σε υπολογισμό από την υποβολή στο ίδιο n, επειδή αφαιρεί πεδία αντί να τα δημιουργεί. Τέλος, ο χρόνος εκτέλεσης δεν αλλάζει ουσιαστικά, από 0,16 έως 0,29 δευτερόλεπτα· σε τοπικό

δίκτυο η καθυστέρηση καθορίζεται από τη συναίνεση, όχι από το μέγεθος της συναλλαγής. Για τη χρήση που προβλέπει η εφαρμογή, λίγα NFTs ανά προσφορά, το σύστημα βρίσκεται βαθιά μέσα στην επίπεδη περιοχή.

7.2.3 Παρτίδες δημιουργίας

Η δεύτερη δοκιμή δημιουργεί 1 έως 200 NFTs σε ένα PTB, με μία εντολή `mint` ανά αντικείμενο και μία κοινή `TransferObjects` (Πίνακας 7.5). Ο υπολογισμός μένει στο ελάχιστο κλιμάκιο ακόμη και για 200 κλήσεις, και το κόστος ανά NFT συγκλίνει στα 0,00233 SUI, την αποθήκευση του ίδιου του αντικειμένου· το σταθερό κόστος της συναλλαγής μοιράζεται σε όλο και περισσότερα αντικείμενα. Ο χρόνος παραμένει κάτω από 0,2 δευτερόλεπτα. Το όριο εδώ είναι οι 1.024 εντολές ανά PTB και τα 512 ορίσματα της τελικής μεταβίβασης.

Πίνακας 7.5: Δημιουργία n NFTs σε μία συναλλαγή, σε SUI (τοπικό δίκτυο)

n	Υπολογισμός	Καθαρό	Καθαρό ανά NFT	ms
1	0,001000	0,003328	0,003328	128
10	0,001000	0,024190	0,002419	162
50	0,001000	0,117214	0,002344	172
100	0,001000	0,233494	0,002335	151
200	0,001000	0,466814	0,002334	174

7.2.4 Ταυτόχρονοι χρήστες

Η τρίτη δοκιμή προσομοιώνει αγορά με φορτίο: 100 προσφέροντες υποβάλλουν ταυτόχρονα από μία προσφορά ο καθένας προς 100 διαφορετικούς κατόχους, και έπειτα οι 100 κάτοχοι αποδέχονται ταυτόχρονα. Και οι 200 συναλλαγές ολοκληρώθηκαν επιτυχώς (Πίνακας 7.6). Κάθε ριπή των 100 χρειάσθηκε κάτω από ένα δευτερόλεπτο συνολικά, δηλαδή περίπου 104 συναλλαγές ανά δευτερόλεπτο από έναν μόνο πελάτη, και ο διάμεσος χρόνος ανά συναλλαγή μέσα στη ριπή ήταν 0,95 δευτερόλεπτα, σχεδόν όσο και η ριπή ολόκληρη: ο κόμβος παραλαμβάνει τις 100 συναλλαγές σχεδόν μαζί και τις διεκπεραιώνει μαζί, όχι σειριακά. Το μέτρο αυτό αφορά τον ρυθμό που μπορεί να παραγάγει ένας πελάτης σε έναν τοπικό κόμβο, και δεν είναι μέτρηση της χωρητικότητας του δικτύου. Το ουσιώδες εύρημα είναι η απουσία αποτυχιών: επειδή κάθε προσφορά είναι χωριστό κοινόχρηστο αντικείμενο και κάθε αποστολέας έχει δικό του νόμισμα, εκατό ανεξάρτητες ανταλλαγές δεν διεκδικούν κανέναν κοινό πόρο.

Πίνακας 7.6: Εκατό ταυτόχρονες συναλλαγές ανά ροή (τοπικό δίκτυο)

Ροή	Επιτυχείς	Αποτυχίες	Σύνολο	Συν./s	Διάμεσος
100 × `place_bid`	100	0	0,96 s	104	0,95 s
100 × `accept_bid`	100	0	0,96 s	104	0,95 s

7.2.5 Ανταγωνισμός για την ίδια προσφορά

Η τέταρτη δοκιμή είναι η μόνη που επιδιώκει σύγκρουση: k λογαριασμοί, από 2 έως 20, προσπαθούν ταυτόχρονα να αποδεχθούν την ίδια ανοικτή προσφορά, σε τρεις γύρους για κάθε k (Πίνακας 7.7). Σε κάθε έναν από τους δώδεκα γύρους ακριβώς ένας πέτυχε και όλοι οι υπόλοιποι απέτυχαν με τον ίδιο λόγο, `InputObjectDeleted`: η συναίνεση διέταξε τις k συναλλαγές, η πρώτη διέγραψε το `Bid`, και οι επόμενες βρήκαν την είσοδό τους να μην υπάρχει. Καμία αποτυχημένη δεν άφησε ίχνος πέραν του gas της. Αυτή είναι η πειραματική επιβεβαίωση του επιχειρήματος της Ενότητας 4.6 για την ταυτόχρονη πρόσβαση, και δείχνει ότι το πρωτόκολλο δεν χρειάζεται κανένα δικό του κλείδωμα: το παρέχει το μοντέλο αντικειμένων.

Πίνακας 7.7: k ταυτόχρονες αποδοχές της ίδιας προσφοράς, τρεις γύροι ανά k

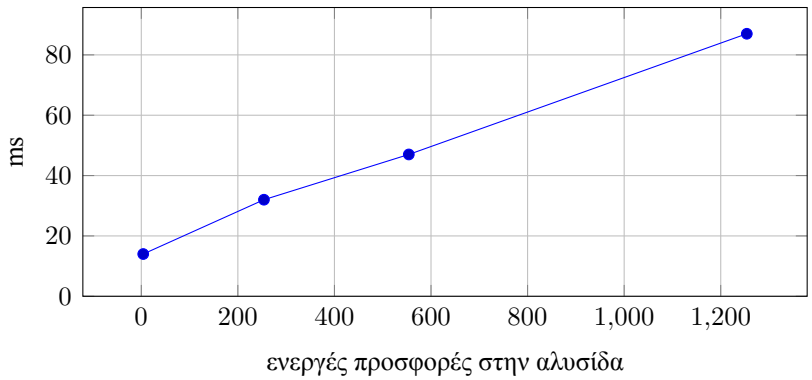
k	Γύροι	Επιτυχείς ανά γύρο	Αποτυχίες συνολικά	Αιτία αποτυχίας
2	3	1	3	`InputObjectDeleted`
5	3	1	12	`InputObjectDeleted`
10	3	1	27	`InputObjectDeleted`
20	3	1	57	`InputObjectDeleted`

7.2.6 Κόστος ανακάλυψης με το ιστορικό

Η τελευταία δοκιμή δεν πιέζει το συμβόλαιο αλλά την εφαρμογή, και ποσοτικοποιεί τον περιορισμό της ανακάλυψης μέσω συμβάντων (Ενότητα 5.3). Μετρήθηκε ο χρόνος του ερωτήματος εισερχόμενων προσφορών ενός κατόχου, όπως το εκτελεί η εφαρμογή (ανάγνωση όλων των `BidPlaced`, ανάκτηση των αντικειμένων, ανάκτηση των στόχων), ενώ πενήντα λογαριασμοί γέμιζαν την αλυσίδα με ενεργές προσφορές (Πίνακας 7.8, Σχήμα 7.2).

Πίνακας 7.8: Χρόνος του ερωτήματος εισερχόμενων προσφορών ως συνάρτηση του ιστορικού

Συμβάντα `BidPlaced`	Ενεργές προσφορές	Διάμεσος χρόνος (3 επαναλήψεις)
237	4	14 ms
487	254	32 ms
787	554	47 ms
1.487	1.254	87 ms



Σχήμα 7.2: Ο χρόνος του ερωτήματος εισερχόμενων προσφορών αυξάνει γραμμικά με το πλήθος των προσφορών στην αλυσίδα, περίπου 58 ms ανά χίλιες προσφορές σε τοπικό κόμβο.

Η σχέση είναι γραμμική, περίπου 58 χιλιοστά του δευτερολέπτου ανά χίλιες ενεργές προσφορές, και ο χρόνος παραμένει κάτω από το ένα δέκατο του δευτερολέπτου στις 1.250. Ο κόμβος όμως είναι τοπικός· σε δημόσιο κόμβο κάθε σελίδα συμβάντων και κάθε παρτίδα αντικειμένων είναι ένα αίτημα δικτύου, και το ίδιο ερώτημα θα κόστιζε δεκάδες τέτοια αιτήματα. Η καμπύλη επιβεβαιώνει αυτό που η Ενότητα 7.6 διατυπώνει ποιοτικά: η προσέγγιση αρκεί για επίδειξη και μικρή κοινότητα, και χρειάζεται indexer πέρα από μερικές χιλιάδες προσφορές.

7.3 Προβολή στο mainnet

Το σύστημα δεν δοκιμάσθηκε στο mainnet, και δεν θα έπρεπε (Ενότητα 4.8.1). Το κόστος του εκεί μπορεί ωστόσο να υπολογισθεί με ακρίβεια, επειδή το gas του Sui είναι γινόμενο μετρημένων ποσοτήτων επί δημοσιευμένες τιμές (Ενότητα 2.2.5): οι μονάδες υπολογισμού και τα bytes αποθήκευσης κάθε συναλλαγής προκύπτουν από τις μετρήσεις, και οι τιμές ανά μονάδα διαβάζονται από το ίδιο το δίκτυο. Ο Πίνακας 7.9 δείχνει τις παραμέτρους των τριών δικτύων, όπως διαβάστηκαν από τους πλήρεις κόμβους τους στις 16 Οκτωβρίου 2026.

Πίνακας 7.9: Παράμετροι gas ανά δίκτυο (16 Οκτωβρίου 2026)

Παράμετρος	Localnet (μετρήσεις)	Testnet	Mainnet
Έκδοση πρωτοκόλλου	114	139	138
Τιμή αναφοράς gas (MIST/μονάδα)	1.000	1.000	100
Τιμή αποθήκευσης (MIST/μονάδα)	76	76	76
Ποσοστό επιστροφής αποθήκευσης	99%	99,99%	99,99%

Δύο διαφορές έχουν σημασία. Η τιμή αναφοράς του mainnet είναι δέκα φορές χαμηλότερη από του τοπικού δικτύου, οπότε ο υπολογισμός κοστίζει το ένα δέκατο. Και το ποσοστό επιστροφής αποθήκευσης έχει αυξηθεί στις νεότερες εκδόσεις του πρωτοκόλλου από 99% σε 99,99%, οπότε το μη επιστρεφόμενο μέρος της εγγύησης, που στο τοπικό δίκτυο ήταν το 1%, γίνεται το ένα εκατοστό του. Η τιμή αποθήκευσης είναι ίδια. Η προβολή υπολογίζει λοιπόν, για κάθε μετρημένη συναλλαγή, υπολογισμό ίσο με τις μετρημένες μονάδες επί 100, αποθήκευση αμετάβλητη, και επιστροφή ίση με το 99,99% της αποθήκευσης που είχαν καταβάλει τα αντικείμενα που διαγράφονται ή τροποποιούνται. Ο Πίνακας 7.10 δίνει το αποτέλεσμα, μαζί με το ισοδύναμο σε δολάρια με την ισοτιμία της ίδιας ημέρας [46].

Πίνακας 7.10: Προβολή του καθαρού κόστους ανά λειτουργία στο mainnet

Λειτουργία	Τοπικό (SUI)	Mainnet (SUI)	Mainnet (USD)
`mint` ×1	0,003343	0,002433	0,0025
`place_bid`: μόνο SUI	0,003936	0,003026	0,0031
`place_bid`: 1 NFT	0,006794	0,005861	0,0061
`place_bid`: 3 NFTs	0,012510	0,011531	0,0120
`accept_bid`: 1 NFT	−0,004647	−0,005660	−0,0059
`accept_bid`: 3 NFTs	−0,010213	−0,011328	−0,0118
`reject_bid`: 1 NFT	−0,004647	−0,005660	−0,0059
`cancel_bid`: 1 NFT	−0,004670	−0,005660	−0,0059
Πλήρης κύκλος: υποβολή και αποδοχή	0,002147	0,000201	0,0002
Πλήρης κύκλος: υποβολή και ακύρωση	0,002124	0,000201	0,0002

Η εικόνα που προκύπτει είναι ακόμη ευνοϊκότερη από αυτή του τοπικού δικτύου. Μια πλήρης ανταλλαγή με ένα NFT κοστίζει στο mainnet περίπου 0,0002 SUI, δηλαδή δύο εκατοστά του σεντ: σχεδόν εξ ολοκλήρου οι δύο ελάχιστες χρεώσεις υπολογισμού των 0,0001 SUI, αφού η εγγύηση αποθήκευσης επιστρέφεται πλέον στο σύνολό της πλην αμελητέου υπολοίπου. Η ασυμμετρία της Ενότητας 7.1 παραμένει και μεγαλώνει ελαφρά: ο κάτοχος που αποδέχεται ή απορρίπτει εισπράττει περίπου 0,0057 SUI (0,006 δολάρια) που είχε καταβάλει ο προσφέρων. Για τις μεγάλες προσφορές της Ενότητας 7.2.2, η υποβολή με 100 NFTs δεσμεύει προσωρινά 0,29 SUI (0,30 δολάρια) και με 500 NFTs 1,44 SUI (1,49 δολάρια),

ποσά που επιστρέφονται σχεδόν ολόκληρα κατά τον τερματισμό· το μόνιμο κόστος των 500 NFTs είναι ο υπολογισμός, 0,0182 SUI στην υποβολή και 0,0056 στην αποδοχή.

Για τον ρυθμό διεκπεραίωσης και την καθυστέρηση δεν υπάρχει αντίστοιχος ακριβής υπολογισμός, και η εκτίμηση πρέπει να είναι ποιοτική. Οι 100 ταυτόχρονες ανταλλαγές της Ενότητας 7.2.4 είναι αμελητέο φορτίο για το δίκτυο, που σχεδιάσθηκε για τάξεις μεγέθους περισσότερες συναλλαγές ανά δευτερόλεπτο [2, 16]· ο περιορισμός σε πραγματική χρήση θα ήταν ο ρυθμός αιτημάτων που δέχεται ο δημόσιος κόμβος από έναν πελάτη. Κάθε συναλλαγή του πρωτοκόλλου αγγίζει ένα κοινόχρηστο αντικείμενο, το `Bid`, άρα περνά από τη συναίνεση (Ενότητα 2.2.2), της οποίας η καθυστέρηση σε δημόσιο δίκτυο με δεκάδες γεωγραφικά διάσπαρτους επικυρωτές είναι της τάξης του μισού δευτερολέπτου κατά τη δημοσιευμένη αξιολόγηση του Mysticeti [16], στην οποία προστίθεται ο χρόνος του πορτοφολιού και η απόσταση του χρήστη από τον κόμβο. Ο χρήστης της εφαρμογής θα δει επομένως τα αποτελέσματα των ενεργειών του σε ένα έως δύο δευτερόλεπτα αντί για το μισό του τοπικού δικτύου, και η συμπεριφορά υπό ανταγωνισμό θα είναι η ίδια με την Ενότητα 7.2.5, αφού την επιβάλλει το ίδιο μοντέλο εκτέλεσης. Η προβολή αυτή είναι εκτίμηση και όχι μέτρηση· η επαλήθευσή της στο testnet, που τρέχει το ίδιο πρωτόκολλο με το mainnet, είναι το φυσικό επόμενο βήμα.

7.4 Αποτελέσματα των σεναρίων

Τα δύο εξωτερικά επίπεδα επαλήθευσης της Ενότητας 5.7 εκτελέσθηκαν στο τοπικό δίκτυο μετά την τελική έκδοση του συμβολαίου και της εφαρμογής. Και τα 20 σενάρια συμβολαίου και οι 17 ροές διεπαφής ολοκληρώθηκαν επιτυχώς, χωρίς σφάλμα εκτέλεσης της εφαρμογής στην κονσόλα του φυλλομετρητή. Ο Πίνακας 7.11 συνοψίζει τα σενάρια που προσθέτουν γνώση πέρα από τους ελέγχους μονάδας· τα υπόλοιπα επαναλαμβάνουν στο πραγματικό δίκτυο τις κανονικές ροές και τους επτά κωδικούς σφαλμάτων.

Οι ροές διεπαφής ανέδειξαν δύο σημεία που δεν είναι σφάλματα αλλά όρια της τρέχουσας υλοποίησης. Οι λίστες δεν ανανεώνονται περιοδικά: μια προσφορά που φθάνει ενώ ο χρήστης βρίσκεται ήδη στη σελίδα «My bids» εμφανίζεται μόλις ο χρήστης αλλάξει σελίδα ή επιστρέψει στο παράθυρο. Και όταν ο χρήστης επιχειρεί να αποδεχθεί προσφορά που ο προσφέρων ακύρωσε στο μεταξύ, η εφαρμογή εμφανίζει το ωμό μήνυμα του κόμβου («Object … has been deleted») αντί για φιλική διατύπωση, και η παρωχημένη γραμμή παραμένει μέχρι την επόμενη ανάγνωση. Και τα δύο αντιμετωπίζονται με περιοδική ανανέωση των ερωτημάτων και καταγράφονται στις μελλοντικές επεκτάσεις.

Πίνακας 7.11: Σενάρια συμβολαίου στο τοπικό δίκτυο που υπερβαίνουν τους ελέγχους μονάδας

Σενάριο	Παρατηρούμενη συμπεριφορά
Δύο προσφορές για το ίδιο NFT· αποδοχή της πρώτης	Η δεύτερη παραμένει στην αλυσίδα. Ο παλιός κάτοχος δεν μπορεί ούτε να την αποδεχθεί ούτε να την απορρίψει, διότι δεν κατέχει πλέον τον στόχο· ο νέος κάτοχος την αποδέχθηκε.
Δύο ταυτόχρονες αποδοχές της ίδιας ανοικτής προσφοράς από διαφορετικούς λογαριασμούς	Ακριβώς μία επιτυγχάνει. Η άλλη απορρίπτεται με `InputObjectDeleted`, χωρίς καμία μεταβολή κατάστασης πέραν του gas.
Ο στόχος είναι δεσμευμένος σε άλλη προσφορά	Η προσφορά δεν μπορεί να διεκπεραιωθεί από κανέναν, αφού κανείς δεν μπορεί να δώσει τον στόχο ως είσοδο· η ακύρωση από τον προσφέροντα λειτουργεί, και μετά την απελευθέρωση του στόχου η αποδοχή επίσης.
Αποδοχή από τον ίδιο τον προσφέροντα	Επιτρέπεται και είναι ακίνδυνη: όλα τα στοιχεία, μαζί με το SUI, καταλήγουν ξανά σε αυτόν.
Προσφορά `Bid<MarketplaceNFT, Coin<SUI>>`	Έγινε αποδεκτή με ένα αντικείμενο νομίσματος ως αντάλλαγμα, χωρίς καμία αλλαγή στο συμβόλαιο: οι γενικές παράμετροι δέχονται οποιονδήποτε τύπο `key + store`.
Λανθασμένα ορίσματα τύπου στην αποδοχή	Απορρίπτονται από την εικονική μηχανή με `TypeMismatch` πριν την εκτέλεση.
Δεύτερη αποδοχή ήδη καταναλωμένης προσφοράς	Απορρίπτεται κατά τον έλεγχο εισόδων, πριν καν ζητηθεί υπογραφή, επειδή το αντικείμενο `Bid` έχει διαγραφεί.

7.5 Κάλυψη απαιτήσεων

Ο Πίνακας 7.12 αντιστοιχίζει τις απαιτήσεις του Κεφαλαίου 3 στους ελέγχους που τις επαληθεύουν. Οι αριθμοί στη στήλη «Έλεγχοι Move» παραπέμπουν στον Πίνακα 4.6· οι αριθμοί με πρόθεμα N στον Πίνακα 4.7. Οι στήλες «SDK» και «Διεπαφή» δείχνουν αν η απαίτηση ασκείται από τα σενάρια συμβολαίου μέσω SDK και από την αυτοματοποιημένη πλοήγηση, αντίστοιχα (Ενότητα 5.7).

Πίνακας 7.12: Αντιστοίχιση απαιτήσεων σε ελέγχους

Απαίτηση	Περιγραφή	Έλεγχοι Move	SDK	Διεπαφή
ΛΑ1	Δημιουργία NFT	N1, N3	✓	✓
ΛΑ2	Προσφορά με NFTs ή/και SUI	1, 2, 3, 7, 8	✓	✓
ΛΑ3	Στοχευμένη προσφορά	8, 9, 11, 18	✓	✓
ΛΑ4	Αποδοχή	6 έως 9, N3	✓	✓
ΛΑ5	Απόρριψη	15, 16, 17	✓	✓
ΛΑ6	Ακύρωση	13, 14	✓	✓
ΛΑ7 έως ΛΑ10	Προβολές της εφαρμογής	---	✓	✓
ΜΛΑ1	Ατομικότητα	10, 11, 12	✓	---
ΜΛΑ2	Διατήρηση στοιχείων	1, 7, 13, 15	✓	✓
ΜΛΑ3	Εξουσιοδότηση	14, 16, 17, 18	✓	---
ΜΛΑ5	Γενικότητα τύπων	1 έως 18	✓	---
ΜΛΑ6	Ταυτόχρονη πρόσβαση	---	✓	---
ΜΛΑ8	Μία υπογραφή ανά ενέργεια	---	---	✓

Τρεις παρατηρήσεις. Η γενικότητα ως προς τους τύπους (ΜΛΑ5) επαληθεύεται από τους ελέγχους Move, που χρησιμοποιούν δύο διαφορετικούς τύπους, και από το σενάριο ανταλλαγής NFT με νόμισμα· η εφαρμογή επίδειξης διαθέτει έναν μόνο τύπο NFT, και οι συλλογές της διαφέρουν ως προς ένα πεδίο. Η ορθότητα υπό ταυτόχρονη πρόσβαση (ΜΛΑ6) ελέγχεται πειραματικά μόνο στο τοπικό δίκτυο, με δύο λογαριασμούς· κατά τα λοιπά στηρίζεται στο μοντέλο εκτέλεσης του Sui, όπως αναλύθηκε στην Ενότητα 4.6. Η απουσία έμπιστου τρίτου (ΜΛΑ4) και η διαφάνεια (ΜΛΑ7) είναι ιδιότητες του σχεδιασμού που επαληθεύονται με επιθεώρηση του κώδικα: κανένα από τα δύο modules δεν περιέχει συνάρτηση με προνομιακό καλούντα.

7.6 Περιορισμοί

Ένας τύπος προσφερομένων ανά προσφορά. Τα NFTs που προσφέρονται σε μία προσφορά πρέπει να είναι του ίδιου τύπου Move. Προσφορά που συνδυάζει αντικείμενα δύο

διαφορετικών τύπων δεν εκφράζεται με την τρέχουσα υπογραφή.

Η εφαρμογή επίδειξης χρησιμοποιεί έναν τύπο. Οι τρεις συλλογές της εφαρμογής είναι τιμές ενός πεδίου του ίδιου τύπου, και η διεπαφή περνά τον `MarketplaceNFT` και στις δύο παραμέτρους. Η ανταλλαγή μεταξύ διαφορετικών τύπων επιδεικνύεται από τους ελέγχους του συμβολαίου, όχι από τη διεπαφή. Η υποστήριξη αυθαίρετων τύπων στη διεπαφή απαιτεί ανακάλυψη των NFTs του χρήστη ανεξαρτήτως τύπου και ανάγνωση του Display τους.

Ανακάλυψη μέσω συμβάντων. Κάθε ερώτημα της εφαρμογής διαβάζει όλα τα συμβάντα ενός τύπου από την αρχή της αλυσίδας και ανακτά όλα τα αντίστοιχα αντικείμενα. Το κόστος αυξάνει γραμμικά με το συνολικό ιστορικό, όχι με το πλήθος των ενεργών προσφορών. Για μια επίδειξη είναι επαρκές· για πραγματική χρήση απαιτείται indexer.

Απουσία λήξης και αντιπρότασης. Οι προσφορές δεν λήγουν, και ο κάτοχος δεν μπορεί να απαντήσει με διαφορετικούς όρους· μπορεί μόνο να δεχθεί ή να απορρίψει.

Επιστροφή της εγγύησης αποθήκευσης. Όπως έδειξε η Ενότητα 7.1, η εγγύηση που καταβάλλει ο προσφέρων επιστρέφεται στον κάτοχο όταν αυτός κλείνει την προσφορά.

Μη επιβολή πολιτικών δημιουργών. Το πρωτόκολλο μεταβιβάζει αντικείμενα με `public_transfer` και δεν γνωρίζει για royalties.

Ανανέωση δεδομένων. Οι λίστες της εφαρμογής ανανεώνονται μετά από ενέργεια του ίδιου του χρήστη ή αλλαγή σελίδας, όχι περιοδικά· ενέργειες τρίτων γίνονται ορατές με καθυστέρηση (Ενότητα 7.4).

Κάλυψη ελέγχων. Η ταυτόχρονη πρόσβαση ελέγχεται μόνο σε τοπικό δίκτυο, με δύο λογαριασμούς· η συμπεριφορά υπό πραγματικό φόρτο δικτύου δεν έχει μετρηθεί. Δεν έχει γίνει ανεξάρτητος έλεγχος ασφάλειας.

Περιβάλλον αξιολόγησης. Οι μετρήσεις έγιναν σε τοπικό δίκτυο ενός επικυρωτή, με παλαιότερη έκδοση πρωτοκόλλου από το mainnet. Τα ποσά gas μεταφέρονται με ακρίβεια (Ενότητα 7.3), οι χρόνοι και ο ρυθμός διεκπεραίωσης όχι· η εκτίμησή τους για δημόσιο δίκτυο είναι ποιοτική έως ότου επαναληφθούν στο testnet.

Κεφάλαιο 8

Συμπεράσματα και μελλοντικές επεκτάσεις

8.1 Σύνοψη

Η εργασία ξεκίνησε από μια παρατήρηση για τις αγορές NFT: είναι οργανωμένες γύρω από το νόμισμα, και ο χρήστης που θέλει να ανταλλάξει ένα αντικείμενο με ένα άλλο, ιδίως διαφορετικής συλλογής, δεν εξυπηρετείται. Το Sui, του οποίου το μοντέλο αντικειμένων προσφέρεται για ασφαλή δέσμευση χωρίς διαρκείς εγκρίσεις προς συμβόλαια, δεν διέθετε τέτοια λειτουργία.

Σχεδιάστηκε ένα πρωτόκολλο στο οποίο η προσφορά είναι η ίδια ένα αντικείμενο της αλυσίδας που κρατά δεσμευμένα τα προσφερόμενα στοιχεία, με τέσσερις πράξεις: υποβολή, αποδοχή, απόρριψη και ακύρωση. Το πρωτόκολλο υλοποιήθηκε σε δύο modules Move συνολικού μήκους 323 γραμμών, με γενικούς τύπους για τις συλλογές και δυναμικά πεδία αντικειμένων για τη δέσμευση, γραμμένα σύμφωνα με δημοσιευμένα πρότυπα ύφους και αρχιτεκτονικής για Sui Move, και συνοδεύεται από 21 ελέγχους μονάδας. Αναπτύχθηκε εφαρμογή ιστού που λειτουργεί χωρίς διακομιστή, ανακαλύπτει NFTs και προσφορές από δεδομένα της αλυσίδας, και παρουσιάζει κάθε προσφορά ως ανταλλαγή «δίνεις ⇄ παίρνεις». Η εφαρμογή επαληθεύθηκε από άκρη σε άκρη σε τρία επίπεδα: ελέγχους μονάδας, 20 σενάρια συμβολαίου σε τοπικό δίκτυο και 17 ροές αυτοματοποιημένης πλοήγησης.

Οι δοκιμές αντοχής έδειξαν ότι το κόστος μιας προσφοράς κλιμακώνεται γραμμικά έως εκατοντάδες δεσμευμένα NFTs, ότι εκατό ταυτόχρονες ανταλλαγές περνούν χωρίς καμία αποτυχία, και ότι είκοσι ταυτόχρονες αποδοχές της ίδιας προσφοράς καταλήγουν πάντοτε σε ακριβώς μία επιτυχία. Η προβολή στο mainnet, με τις τρέχουσες τιμές του δικτύου, φέρνει το κόστος μιας πλήρους ανταλλαγής στα 0,0002 SUI.

8.2 Συμπεράσματα

Το μοντέλο αντικειμένων απλοποιεί ουσιαστικά το πρόβλημα. Η δέσμευση στοιχείων, που σε μοντέλο λογαριασμών απαιτεί εγκρίσεις και προσεκτική λογιστική, στο Sui είναι μια μεταβίβαση κυριότητας σε αντικείμενο. Η εξουσιοδότηση της απόρριψης υλοποιήθηκε χωρίς κανένα μητρώο κατόχων, απαιτώντας απλώς μια αναφορά στο αντικείμενο. Η ορθότητα υπό ταυτόχρονη πρόσβαση δεν απαίτησε κώδικα: προκύπτει από το ότι η προσφορά είναι κοινόχρηστο αντικείμενο που διαγράφεται στην πρώτη επιτυχή ενέργεια, και το σενάριο των δύο ταυτόχρονων αποδοχών το επιβεβαίωσε στην πράξη. Η ίδια ιδιότητα έχει μια συνέπεια που αξίζει να γίνεται ρητή στους χρήστες: η προσφορά δεσμεύεται στο αντικείμενο, όχι στον κάτοχό του, και τον ακολουθεί αν αυτό αλλάξει χέρια.

Το σύστημα τύπων της Move λειτουργεί ως έλεγχος ορθότητας. Τα πιο επικίνδυνα σφάλματα ενός πρωτοκόλλου δέσμευσης, η απώλεια ή ο εγκλωβισμός στοιχείων, είναι σε μεγάλο βαθμό σφάλματα μεταγλώττισης. Ένα NFT ή ένα υπόλοιπο που δεν διατίθεται ρητά εμποδίζει τη μεταγλώττιση. Το ένα σημείο όπου η γλώσσα δεν βοηθά, η διαγραφή αντικειμένου με εναπομείναντα δυναμικά πεδία, ήταν ακριβώς εκείνο που χρειάστηκε επιχειρηματολογία και ειδικούς ελέγχους.

Οι γενικοί τύποι αρκούν για τις διαφορετικές συλλογές. Η υποστήριξη αυθαίρετων συλλογών δεν απαίτησε μητρώο, διαχειριστή ή ενδιάμεσο token. Δύο παράμετροι τύπου στη δομή της προσφοράς μεταφέρουν τον έλεγχο στην εικονική μηχανή. Το τίμημα είναι ο περιορισμός σε έναν τύπο προσφερομένων ανά προσφορά.

Μια εφαρμογή χωρίς διακομιστή είναι εφικτή, με όρια. Τα συμβάντα επαρκούν για να ανακαλύψει η εφαρμογή ό,τι χρειάζεται από έναν πλήρη κόμβο. Η προσέγγιση αυτή δίνει ισχυρές εγγυήσεις διαφάνειας και μηδενικό κόστος λειτουργίας, αλλά δεν κλιμακώνεται, διότι κάθε ερώτημα διατρέχει όλο το ιστορικό.

Το κόστος είναι χαμηλό και ανεξάρτητο από την αξία. Μια πλήρης ανταλλαγή κοστίζει περίπου 0,002 SUI σε gas, χωρίς τέλη. Η μέτρηση ανέδειξε και δύο πράγματα που δεν είχαν προβλεφθεί: η εγγύηση αποθήκευσης επιστρέφεται σε όποιον κλείνει την προσφορά και όχι σε όποιον την κατέβαλε, και κάθε πλεονάζον πεδίο σε ένα αντικείμενο κοστίζει. Η αφαίρεση δύο πεδίων που επαναλάμβαναν πληροφορία διαθέσιμη αλλού μείωσε το κόστος αποθήκευσης της προσφοράς κατά σχεδόν ένα τρίτο.

Η επαλήθευση από άκρη σε άκρη αποκαλύπτει ό,τι ο έλεγχος τύπων δεν μπορεί. Το σοβαρότερο πρόβλημα που συναντήθηκε στην εφαρμογή, η απουσία μιας μεθόδου από τον

τοπικό κόμβο, δεν προκαλούσε κανένα σφάλμα μεταγλώττισης και εντοπίστηκε μόνο όταν οι ροές εκτελέστηκαν σε πραγματικό δίκτυο. Η επένδυση σε ένα προσωρινό πορτοφόλι και σε αυτοματοποιημένη πλοήγηση αποδόθηκε πολλαπλά: εντόπισε σφάλματα, και παρήγαγε τα στιγμιότυπα της εργασίας.

8.3 Μελλοντικές επεκτάσεις

Το σύστημα μπορεί να επεκταθεί προς πολλές κατευθύνσεις. Παρατίθενται κατά σειρά εγγύτητας προς την τρέχουσα υλοποίηση.

1. **Λήξη προσφορών.** Προσθήκη προαιρετικής χρονικής σήμανσης λήξης, που ελέγχεται κατά την αποδοχή με το αντικείμενο `Clock` του συστήματος. Μια ληγμένη προσφορά θα μπορούσε να κλείνεται από οποιονδήποτε, με επιστροφή των στοιχείων στον προσφέροντα.

2. **Απόδοση της εγγύησης στον προσφέροντα.** Το συμβόλαιο θα μπορούσε να δέχεται κατά την υποβολή ένα μικρό ποσό που επιστρέφεται στον προσφέροντα κατά τον τερματισμό, αντισταθμίζοντας την επιστροφή αποθήκευσης που εισπράττει ο κάτοχος. Εναλλακτικά, η διεπαφή θα μπορούσε απλώς να κάνει το ποσό ορατό.

3. **Αντιπροτάσεις.** Ο κάτοχος να μπορεί να απαντήσει σε μια προσφορά με τροποποιημένους όρους. Υλοποιείται ως νέα προσφορά με αντίστροφους ρόλους που παραπέμπει στην αρχική, ώστε η διεπαφή να παρουσιάζει τη διαπραγμάτευση ως ενιαίο νήμα.

4. **Ετερογενή καλάθια.** Άρση του περιορισμού ενός τύπου προσφερομένων ανά προσφορά. Τα δυναμικά πεδία αντικειμένων δέχονται ήδη τιμές διαφορετικών τύπων· η δυσκολία βρίσκεται στις συναρτήσεις τερματισμού, που πρέπει να γνωρίζουν τον τύπο κάθε στοιχείου για να το αφαιρέσουν. Μια λύση είναι η σταδιακή σύνθεση και αποσύνθεση της προσφοράς με μία κλήση ανά τύπο μέσα στο ίδιο PTB.

5. **Πολλαπλοί στόχοι ανά προσφορά.** Αντικατάσταση του `target_nft_id` με λίστα αποδεκτών στόχων, ώστε ο ίδιος δεσμευμένος συνδυασμός να προσφέρεται για οποιοδήποτε από περισσότερα NFTs, με την πρώτη αποδοχή να καταναλώνει την προσφορά. Η απόρριψη θα αφαιρούσε μόνο τον στόχο του απορρίπτοντος από τη λίστα. Η ανάγκη προέκυψε από τη χρήση: με το τρέχον σχήμα, όποιος θέλει τα ίδια δύο NFTs για δύο διαφορετικούς στόχους πρέπει να διαλέξει έναν.

6. **Περιοδική ανανέωση και φιλικότερα σφάλματα.** Ανανέωση των ερωτημάτων ανά τακτά διαστήματα, ώστε ενέργειες τρίτων να εμφανίζονται χωρίς αλλαγή σελίδας, και

αντιστοίχιση των σφαλμάτων ανύπαρκτου αντικειμένου σε μήνυμα της μορφής «η προσφορά δεν είναι πλέον διαθέσιμη» (Ενότητα 7.4).

7. **Indexer και διεπαφή GraphQL.** Αντικατάσταση της ανακάλυψης μέσω συμβάντων με ερωτήματα σε ευρετηριασμένη βάση, ώστε το κόστος ανάγνωσης να εξαρτάται από το πλήθος των ενεργών προσφορών. Τα συμβάντα που ήδη εκπέμπει το συμβόλαιο επαρκούν για την κατασκευή του ευρετηρίου.

8. **Αυθαίρετοι τύποι στη διεπαφή.** Ανακάλυψη όλων των NFTs του χρήστη ανεξαρτήτως τύπου και απόδοσή τους μέσω του προτύπου Display, ώστε η εφαρμογή να χρησιμοποιείται με πραγματικές συλλογές του οικοσυστήματος.

9. **Τυπική επαλήθευση.** Διατύπωση των αναλλοίωτων A1 έως A3 ως προδιαγραφών και απόδειξή τους με τον Move Prover, και ανεξάρτητος έλεγχος ασφάλειας, ως προϋποθέσεις για ανάπτυξη σε mainnet.

10. **Ευκολότερη είσοδος χρηστών.** Υποστήριξη zkLogin, ώστε ένας νέος χρήστης να συνδέεται με λογαριασμό που ήδη διαθέτει αντί να εγκαθιστά πορτοφόλι, και χορηγούμενων συναλλαγών, ώστε να μην απαιτείται SUI για την πρώτη ενέργεια.

Τέλος, αν και η εργασία αφορά NFTs, το πρωτόκολλο δεν κάνει καμία υπόθεση για το τι αναπαριστούν τα αντικείμενα που ανταλλάσσονται. Με τον ίδιο κώδικα μπορούν να ανταλλαγούν αντικείμενα παιχνιδιών, εισιτήρια, ή οποιοδήποτε περιουσιακό στοιχείο αναπαρίσταται ως αντικείμενο του Sui.

Βιβλιογραφία

[1] William Entriken, Dieter Shirley, Jacob Evans και Nastassia Sachs. *EIP-721: Non-Fungible Token Standard*. Πρόσβαση: Οκτώβριος 2026. 2018. URL: https://eips.ethereum.org/EIPS/eip-721.

[2] The MystenLabs Team. *The Sui Smart Contracts Platform*. Πρόσβαση: Οκτώβριος 2026. 2023. URL: https://docs.sui.io/paper/sui.pdf.

[3] OpenSea. *OpenSea: NFT Marketplace*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://opensea.io/.

[4] Blur Foundation. *Blur: NFT Marketplace for Pro Traders*. Πρόσβαση: Οκτώβριος 2026. 2023. URL: https://blur.io/.

[5] Tradeport. *Tradeport: Sui NFT Marketplace*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://www.tradeport.xyz/.

[6] BlueMove. *BlueMove: Multi-chain NFT Marketplace*. Αρχειοθετημένο στιγμιότυπο (Internet Archive, 05/01/2026)· ο ιστότοπος sui.bluemove.net δεν αποκρίνεται πλέον. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://web.archive.org/web/20260105192003/https://sui.bluemove.net/.

[7] Sam Blackshear κ.ά. *Move: A Language With Programmable Resources*. Πρόσβαση: Οκτώβριος 2026. 2019. URL: https://developers.diem.com/papers/diem-move-a-language-with-programmable-resources/2019-06-18.pdf.

[8] Ioannis D. Papias. «Blockchain Tracker: Διαδικτυακή Εφαρμογή για την Καταγραφή και Απεικόνιση Πρωτοβουλιών στην Τεχνολογία Blockchain». Στα ελληνικά. Αγγλικός τίτλος: Blockchain Tracker: Web Application for Recording and Visualising Blockchain Initiatives. Diploma Thesis. National Technical University of Athens, School of Electrical και Computer Engineering, Μαρ. 2023.

[9] Christos Hadjichristofi. «Implementation of Blockchain Application for managing University grades». Diploma Thesis. National Technical University of Athens, School of Electrical και Computer Engineering, 2022. URL: https://dspace.lib.ntua.gr/xmlui/handle/123456789/55872.

[10] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Πρόσβαση: Οκτώβριος 2026. 2008. URL: https://bitcoin.org/bitcoin.pdf.

[11] Nick Szabo. «Formalizing and Securing Relationships on Public Networks». Στο: *First Monday* 2.9 (1997). DOI: 10.5210/fm.v2i9.548.

[12] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. Πρόσβαση: Οκτώβριος 2026. 2014. URL: https://ethereum.org/en/whitepaper/.

[13] Mysten Labs. *Sui Documentation*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/.

[14] Mysten Labs. *Sui Documentation: Object Model*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/concepts/object-model.

[15] Sam Blackshear κ.ά. *Sui Lutris: A Blockchain Combining Broadcast and Consensus*. arXiv:2310.18042. 2023. URL: https://arxiv.org/abs/2310.18042.

[16] Kushal Babel κ.ά. *Mysticeti: Reaching the Limits of Latency with Uncertified DAGs*. arXiv:2310.14821. 2023. URL: https://arxiv.org/abs/2310.14821.

[17] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino και Alexander Spiegelman. «Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus». Στο: *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*. 2022, σσ. 34–50. DOI: 10.1145/3492321.3519594.

[18] Mysten Labs. *Sui Documentation: Sui Full Node Configuration*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/guides/operator/sui-full-node.

[19] Mysten Labs. *Sui Documentation: Programmable Transaction Blocks*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/concepts/transactions/prog-txn-blocks.

[20] Mysten Labs. *Sui Documentation: Gas in Sui*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/concepts/tokenomics/gas-in-sui.

[21] Mysten Labs. *Sui Object Display*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/standards/display.

[22] Mysten Labs. *Sui Wallet Standard*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/standards/wallet-standard.

[23] Mysten Labs. *Sui Documentation: Networks*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/develop/sui-architecture/networks.

[24] Mysten Labs. *Sui Documentation: Connect to a Local Network*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/guides/developer/getting-started/local-network.

[25] Damir Shamanaev κ.ά. *The Move Book*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://move-book.com/.

[26] Mysten Labs. *Sui Documentation: Dynamic Fields*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/concepts/dynamic-fields.

[27] Qin Wang, Rujia Li, Qi Wang και Shiping Chen. *Non-Fungible Token (NFT): Overview, Evaluation, Opportunities and Challenges*. arXiv:2105.07447. 2021. URL: https://arxiv.org/abs/2105.07447.

[28] Witek Radomski, Andrew Cooke, Philippe Castonguay, James Therien, Eric Binet και Ronan Sandford. *EIP-1155: Multi Token Standard*. Πρόσβαση: Οκτώβριος 2026. 2018. URL: https://eips.ethereum.org/EIPS/eip-1155.

[29] Matthieu Nadini, Laura Alessandretti, Flavio Di Giacinto, Mauro Martino, Luca Maria Aiello και Andrea Baronchelli. «Mapping the NFT revolution: market trends, trade networks, and visual features». Στο: *Scientific Reports* 11.20902 (2021). DOI: 10.1038/s41598-021-00053-8.

[30] Henning Pagnia και Felix C. Gärtner. *On the Impossibility of Fair Exchange without a Trusted Third Party*. Αδημοσίευτη ερευνητική εργασία TUD-BS-1999-02. Darmstadt University of Technology, 1999.

[31] N. Asokan, Matthias Schunter και Michael Waidner. «Optimistic Protocols for Fair Exchange». Στο: *Proceedings of the 4th ACM Conference on Computer and Communications Security (CCS)*. 1997, σσ. 7–17. DOI: 10.1145/266420.266426.

[32] Maurice Herlihy. «Atomic Cross-Chain Swaps». Στο: *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing (PODC)*. 2018, σσ. 245–254. DOI: 10.1145/3212734.3212736.

[33] Object Management Group. *Unified Modeling Language, Version 2.5.1*. Πρόσβαση: Οκτώβριος 2026. 2017. URL: https://www.omg.org/spec/UML/2.5.1.

[34] José Cerqueira. *Sui Move Engineering Skills: architecture, source style, security and testing standards for Sui Move*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://github.com/josemvcerqueira/sui-move-engineering-skills.

[35] Microsoft. *TypeScript*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://www.typescriptlang.org/.

[36] Meta Open Source. *React: The library for web and native user interfaces*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://react.dev/.

[37] Vite contributors. *Vite: Next Generation Frontend Tooling*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://vite.dev/.

[38] Mysten Labs. *Sui TypeScript SDK*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://sdk.mystenlabs.com/sui.

[39] Mysten Labs. *Sui dApp Kit*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://sdk.mystenlabs.com/dapp-kit.

[40] TanStack. *TanStack Query*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://tanstack.com/query.

[41] Tailwind Labs. *Tailwind CSS*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://tailwindcss.com/.

[42] WorkOS. *Radix Primitives*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://www.radix-ui.com/primitives.

[43] Mysten Labs. *Sui Documentation: gRPC Overview*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://docs.sui.io/concepts/grpc-overview.

[44] Leonxlnx. *taste-skill: anti-slop frontend design standards*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://github.com/Leonxlnx/taste-skill.

[45] Microsoft. *Playwright: Fast and reliable end-to-end testing for modern web apps*. Πρόσβαση: Οκτώβριος 2026. 2026. URL: https://playwright.dev/.

[46] CoinGecko. *Sui (SUI) price, simple price API*. Τιμή 1,039 USD ανά SUI στις 16 Οκτωβρίου 2026. 2026. URL: https://api.coingecko.com/api/v3/simple/price?ids=sui&vs_currencies=usd.

Συντομογραφίες

ABI	Application Binary Interface
API	Application Programming Interface
CSS	Cascading Style Sheets
dApp	Decentralized Application
DOF	Dynamic Object Field
DOM	Document Object Model
ERC	Ethereum Request for Comments
gRPC	gRPC Remote Procedure Calls
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
ID	Identifier
JSON	JavaScript Object Notation
JSON-RPC	JSON Remote Procedure Call
MIST	Υποδιαίρεση του SUI (1 SUI $= 10^9$ MIST)
NFT	Non-Fungible Token
OTW	One-Time Witness
PTB	Programmable Transaction Block
SDK	Software Development Kit
UID	Unique Identifier
UML	Unified Modeling Language
URL	Uniform Resource Locator
ΛΑ	Λειτουργική απαίτηση
ΜΛΑ	Μη λειτουργική απαίτηση
ΠΧ	Περίπτωση χρήσης

Παράρτημα Α′

Πηγαίος κώδικας του έξυπνου συμβολαίου

Παρατίθενται τα δύο modules του πακέτου `nft_barter`, όπως δημοσιεύθηκαν στο δίκτυο, καθώς και το σκηνικό των ελέγχων. Οι ίδιοι οι 21 έλεγχοι περιλαμβάνονται στο αποθετήριο, στα αρχεία `barter_tests.move` και `marketplace_nft_tests.move` του καταλόγου `tests`· ένας από αυτούς παρατίθεται στο Απόσπασμα 4.10.

Α′.1 marketplace_nft.move

```
/// Freely mintable demo collectible with wallet `Display`. Metadata is caller
/// supplied and deliberately unvalidated: the dApp curates collections.
module nft_barter::marketplace_nft;

use std::string::String;
use sui::{display, event, package};

// === Public Structs ===

/// One-time witness; claims the `Publisher` that owns the `Display`.
public struct MARKETPLACE_NFT has drop {}

/// A tradeable collectible. Any `key + store` type can be bartered; this one
/// exists so the marketplace has something to mint.
public struct MarketplaceNFT has key, store {
    id: UID,
```

```
    name: String,
    description: String,
    image_url: String,
    collection: String,
    rarity: String,
}

/// A new NFT exists. Lets clients discover NFTs without an indexer.
public struct NftMinted has copy, drop {
    nft_id: ID,
}

// === Private Initialization ===

fun init(otw: MARKETPLACE_NFT, ctx: &mut TxContext) {
    let publisher = package::claim(otw, ctx);

    let mut nft_display = display::new_with_fields<MarketplaceNFT>(
        &publisher,
        vector[b"name", b"description", b"image_url", b"collection", b"
            rarity"].map!(|key| key.to_string()),
        vector[b"{name}", b"{description}", b"{image_url}", b"{collection}
            ", b"{rarity}"].map!(
            |template| template.to_string(),
        ),
        ctx,
    );
    nft_display.update_version();

    transfer::public_transfer(publisher, ctx.sender());
    transfer::public_transfer(nft_display, ctx.sender());
}

// === Public Functions ===

/// Mints an NFT and returns it; the transaction decides where it goes, so
    a
/// PTB can mint and bid in one go.
public fun mint(
    name: String,
```

```
    description: String,
    image_url: String,
    collection: String,
    rarity: String,
    ctx: &mut TxContext,
): MarketplaceNFT {
    let nft = MarketplaceNFT {
        id: object::new(ctx),
        name,
        description,
        image_url,
        collection,
        rarity,
    };

    event::emit(NftMinted { nft_id: nft.id.to_inner() });

    nft
}

// === Test-Only Functions ===

#[test_only]
public fun init_for_testing(ctx: &mut TxContext) {
    init(MARKETPLACE_NFT {}, ctx);
}

#[test_only]
/// Field snapshot in declaration order, for asserting minted metadata.
public fun metadata(self: &MarketplaceNFT): (String, String, String,
    String, String) {
    (self.name, self.description, self.image_url, self.collection, self.
        rarity)
}
```

Απόσπασμα κώδικα Α′.1: Το module `marketplace_nft`

Α′.2 barter.move

```
/// Escrowed NFT-for-NFT barter. A bid locks the offered assets inside a shared
/// `Bid`; accepting it swaps both sides in one transaction, so nobody sends first.
module nft_barter::barter;

use std::{ascii::String, type_name};
use sui::{balance::Balance, coin::Coin, dynamic_object_field as dof, event, sui::SUI};

// === Errors ===

#[error(code = 0)]
const ENotBidder: vector<u8> = b"Only the bidder can cancel this bid.";
#[error(code = 1)]
const EInvalidCounterCount: vector<u8> = b"The counter offer must hold exactly the requested number of NFTs.";
#[error(code = 2)]
const ETargetNftNotProvided: vector<u8> = b"The counter offer must include the targeted NFT.";
#[error(code = 3)]
const EEmptyBid: vector<u8> = b"A bid must offer at least one NFT or some SUI.";
#[error(code = 4)]
const EInvalidRequestedCount: vector<u8> = b"A bid must request at least one NFT.";
#[error(code = 5)]
const EBidNotTargeted: vector<u8> = b"Only a targeted bid can be rejected.";
#[error(code = 6)]
const ENotTargetNftOwner: vector<u8> = b"Only the owner of the targeted NFT can reject this bid.";

// === Public Structs ===

/// An open offer of `offered_count` NFTs of type `Offered` plus `offered_sui`,
/// escrowed as dynamic object fields, for `requested_count` NFTs of type `Requested`.
```

```
29 public struct Bid<phantom Offered: key + store, phantom Requested: key +
       store> has key {
30     id: UID,
31     bidder: address,
32     offered_count: u64,
33     offered_sui: Balance<SUI>,
34     requested_count: u64,
35     /// When set, the counter offer must include this NFT and only its
           owner may reject.
36     target_nft_id: Option<ID>,
37 }
38
39 /// Dynamic object field key of the `n`-th escrowed NFT.
40 public struct EscrowKey(u64) has copy, drop, store;
41
42 /// A bid opened. The type names identify the bartered domains after the `
       Bid` is deleted.
43 public struct BidPlaced has copy, drop {
44     bid_id: ID,
45     offered_type: String,
46     requested_type: String,
47     offered_count: u64,
48     offered_sui: u64,
49     requested_count: u64,
50     target_nft_id: Option<ID>,
51 }
52
53 /// The escrow went to the sender and the counter NFTs to the bidder.
54 public struct BidAccepted has copy, drop {
55     bid_id: ID,
56 }
57
58 /// The bidder reclaimed the escrow.
59 public struct BidCancelled has copy, drop {
60     bid_id: ID,
61 }
62
63 /// The targeted NFT's owner declined; the escrow returned to the bidder.
64 public struct BidRejected has copy, drop {
65     bid_id: ID,
```

```
}

// === Public Functions ===

/// Escrows `offered` and `offered_coin` in a new shared `Bid` owned by
    the sender.
public fun place_bid<Offered: key + store, Requested: key + store>(
    offered: vector<Offered>,
    offered_coin: Coin<SUI>,
    requested_count: u64,
    target_nft_id: Option<ID>,
    ctx: &mut TxContext,
) {
    assert!(!offered.is_empty() || offered_coin.value() > 0, EEmptyBid);
    assert!(requested_count > 0, EInvalidRequestedCount);

    let mut bid = Bid<Offered, Requested> {
        id: object::new(ctx),
        bidder: ctx.sender(),
        offered_count: offered.length(),
        offered_sui: offered_coin.into_balance(),
        requested_count,
        target_nft_id,
    };
    bid.escrow(offered);

    event::emit(BidPlaced {
        bid_id: bid.id.to_inner(),
        offered_type: type_name::with_defining_ids<Offered>().into_string
            (),
        requested_type: type_name::with_defining_ids<Requested>().
            into_string(),
        offered_count: bid.offered_count,
        offered_sui: bid.offered_sui.value(),
        requested_count,
        target_nft_id,
    });

    transfer::share_object(bid);
}
```

```
103
104 /// Settles the swap: the escrow goes to the sender and `counter_nfts` go
        to the bidder.
105 public fun accept_bid<Offered: key + store, Requested: key + store>(
106     mut self: Bid<Offered, Requested>,
107     counter_nfts: vector<Requested>,
108     ctx: &mut TxContext,
109 ) {
110     self.assert_counter_offer(&counter_nfts);
111
112     let bid_id = self.id.to_inner();
113     let bidder = self.bidder;
114     let acceptor = ctx.sender();
115     self.release_escrow(acceptor);
116     counter_nfts.do!(|nft| transfer::public_transfer(nft, bidder));
117     self.destroy(acceptor, ctx);
118
119     event::emit(BidAccepted { bid_id });
120 }
121
122 /// Returns the escrow to the bidder. Only the bidder may cancel.
123 public fun cancel_bid<Offered: key + store, Requested: key + store>(
124     mut self: Bid<Offered, Requested>,
125     ctx: &mut TxContext,
126 ) {
127     assert!(self.bidder == ctx.sender(), ENotBidder);
128
129     let bid_id = self.id.to_inner();
130     let bidder = self.bidder;
131     self.release_escrow(bidder);
132     self.destroy(bidder, ctx);
133
134     event::emit(BidCancelled { bid_id });
135 }
136
137 /// Declines a targeted bid and returns the escrow to the bidder.
138 /// Presenting `target_nft` proves the sender owns it.
139 public fun reject_bid<Offered: key + store, Requested: key + store>(
140     mut self: Bid<Offered, Requested>,
141     target_nft: &Requested,
```

```
142     ctx: &mut TxContext,
143 ) {
144     assert!(self.target_nft_id.is_some(), EBidNotTargeted);
145     assert!(self.target_nft_id.contains(&object::id(target_nft)),
            ENotTargetNftOwner);
146 
147     let bid_id = self.id.to_inner();
148     let bidder = self.bidder;
149     self.release_escrow(bidder);
150     self.destroy(bidder, ctx);
151 
152     event::emit(BidRejected { bid_id });
153 }
154 
155 // === Private Functions ===
156 
157 fun assert_counter_offer<Offered: key + store, Requested: key + store>(
158     self: &Bid<Offered, Requested>,
159     counter_nfts: &vector<Requested>,
160 ) {
161     assert!(counter_nfts.length() == self.requested_count,
            EInvalidCounterCount);
162     if (self.target_nft_id.is_none()) return;
163 
164     let target_nft_id = *self.target_nft_id.borrow();
165     assert!(counter_nfts.any!(|nft| object::id(nft) == target_nft_id),
            ETargetNftNotProvided);
166 }
167 
168 /// Attaches each offered NFT under `EscrowKey(n)`, in offer order.
169 fun escrow<Offered: key + store, Requested: key + store>(
170     self: &mut Bid<Offered, Requested>,
171     offered: vector<Offered>,
172 ) {
173     let mut index = 0;
174     offered.do!(|nft| {
175         dof::add(&mut self.id, EscrowKey(index), nft);
176         index = index + 1;
177     });
178 }
```

```
179
180 /// Detaches every escrowed NFT and transfers it to `recipient`.
181 fun release_escrow<Offered: key + store, Requested: key + store>(
182     self: &mut Bid<Offered, Requested>,
183     recipient: address,
184 ) {
185     self.offered_count.do!(|index| {
186         let nft = dof::remove<EscrowKey, Offered>(&mut self.id, EscrowKey(
                index));
187         transfer::public_transfer(nft, recipient);
188     });
189 }
190
191 /// Deletes the emptied bid and pays its SUI, if any, to `recipient`.
192 fun destroy<Offered: key + store, Requested: key + store>(
193     self: Bid<Offered, Requested>,
194     recipient: address,
195     ctx: &mut TxContext,
196 ) {
197     let Bid { id, offered_sui, .. } = self;
198     id.delete();
199
200     if (offered_sui.value() > 0) {
201         transfer::public_transfer(offered_sui.into_coin(ctx), recipient);
202     } else {
203         offered_sui.destroy_zero();
204     }
205 }
206
207 // === Test-Only Functions ===
208
209 #[test_only]
210 /// The object ID of the `n`-th escrowed NFT, for custody assertions.
211 public fun escrowed_nft_id<Offered: key + store, Requested: key + store>(
212     self: &Bid<Offered, Requested>,
213     index: u64,
214 ): ID {
215     dof::id(&self.id, EscrowKey(index)).destroy_some()
216 }
217
```

```
#[test_only]
public fun bid_placed_fields(e: &BidPlaced): (ID, String, String, u64, u64
    , u64, Option<ID>) {
    (
        e.bid_id,
        e.offered_type,
        e.requested_type,
        e.offered_count,
        e.offered_sui,
        e.requested_count,
        e.target_nft_id,
    )
}

#[test_only]
public fun bid_accepted_id(e: &BidAccepted): ID { e.bid_id }

#[test_only]
public fun bid_cancelled_id(e: &BidCancelled): ID { e.bid_id }

#[test_only]
public fun bid_rejected_id(e: &BidRejected): ID { e.bid_id }
```

Απόσπασμα κώδικα Α′.2: Το module `barter`

Α′.3 barter_fixture.move

```
/// Shared `test_scenario` fixture: two mock asset types plus wallet
    helpers,
/// so each test states only its own delta.
#[test_only]
module nft_barter::barter_fixture;

use std::unit_test::destroy;
use sui::{coin::{Self, Coin}, sui::SUI, test_scenario::{Self as ts,
    Scenario}};
use nft_barter::barter::Bid;

// === Constants ===
```

```
11
12 const ALICE: address = @0xA11CE;
13 const BOB: address = @0xB0B;
14
15 public fun alice(): address { ALICE }
16
17 public fun bob(): address { BOB }
18
19 // === Test-Only Types ===
20
21 /// A `key + store` asset offered in bids.
22 public struct Sword has key, store { id: UID }
23
24 /// A `key + store` asset requested by bids.
25 public struct Shield has key, store { id: UID }
26
27 /// Hot-potato fixture; `end` is the only way to drop it.
28 public struct BarterFixture { scenario: Scenario }
29
30 // === Test-Only Functions ===
31
32 public fun start(sender: address): BarterFixture {
33     BarterFixture { scenario: ts::begin(sender) }
34 }
35
36 public fun end(self: BarterFixture) {
37     let BarterFixture { scenario } = self;
38     scenario.end();
39 }
40
41 /// Advances to a transaction from `sender` and runs `body` inside it.
42 public macro fun next_tx($self: &mut BarterFixture, $sender: address,
    $body: |&mut BarterFixture|) {
43     let fixture = $self;
44     fixture.scenario_mut().next_tx($sender);
45     $body(fixture);
46 }
47
48 public macro fun ctx($self: &mut BarterFixture): &mut TxContext {
49     let fixture = $self;
```

```
50      fixture.scenario_mut().ctx()
51  }
52
53  public fun scenario_mut(self: &mut BarterFixture): &mut Scenario {
54      &mut self.scenario
55  }
56
57  public fun sender(self: &BarterFixture): address {
58      self.scenario.sender()
59  }
60
61  public fun new_sword(self: &mut BarterFixture): Sword {
62      Sword { id: object::new(self.scenario.ctx()) }
63  }
64
65  public fun new_shield(self: &mut BarterFixture): Shield {
66      Shield { id: object::new(self.scenario.ctx()) }
67  }
68
69  public fun swords(self: &mut BarterFixture, count: u64): vector<Sword> {
70      vector::tabulate!(count, |_| self.new_sword())
71  }
72
73  /// Mints a shield into `owner`'s wallet and returns its ID.
74  public fun shield_owned_by(self: &mut BarterFixture, owner: address): ID {
75      let shield = self.new_shield();
76      let shield_id = shield.id.to_inner();
77      transfer::public_transfer(shield, owner);
78
79      shield_id
80  }
81
82  public fun sui(self: &mut BarterFixture, amount: u64): Coin<SUI> {
83      coin::mint_for_testing(amount, self.scenario.ctx())
84  }
85
86  public fun take_bid(self: &BarterFixture): Bid<Sword, Shield> {
87      self.scenario.take_shared()
88  }
89
```

```
90  public fun take_shield(self: &BarterFixture): Shield {
91      self.scenario.take_from_sender()
92  }
93
94  /// Number of swords in the current sender's wallet.
95  public fun sword_count(self: &BarterFixture): u64 {
96      self.scenario.ids_for_sender<Sword>().length()
97  }
98
99  public fun shield_count(self: &BarterFixture): u64 {
100     self.scenario.ids_for_sender<Shield>().length()
101 }
102
103 public fun has_bid(_self: &BarterFixture): bool {
104     ts::has_most_recent_shared<Bid<Sword, Shield>>()
105 }
106
107 /// Takes the sender's SUI coin, returns its value and destroys it.
108 public fun sui_received(self: &BarterFixture): u64 {
109     let received: Coin<SUI> = self.scenario.take_from_sender();
110     let amount = received.value();
111     destroy(received);
112
113     amount
114 }
115
116 public fun has_sui(self: &BarterFixture): bool {
117     self.scenario.has_most_recent_for_sender<Coin<SUI>>()
118 }
```

Απόσπασμα κώδικα Α′.3: Το σκηνικό των ελέγχων